

Resident/Relative App – Care Services

Completed Work

AI Discovery

30 01 2025

Business Use Only



Purpose of this Document

01

What does this document contain?

This document contains **information on completed work during the feasibility phase of this initiative.**

Details are grouped by sprint to provide evidence for sprint reviews and planning.

02

Who is it for?

This document can be used by the **AI Squad** to share updates on completed work internally and with **Stakeholders and Sponsors.**

This can also used to identify next steps/development goals.

03

When and how do we use it?

This should be populated **towards the end of every sprint.**

Once completed, it can be **presented during sprint reviews.**

This document also acts as a **long-term record** of work completed.

AI Solution 1: Technical Feasibility

To test the feasibility of solution 1 (see feasibility study), we created an NLP solution to extract foods from nutrition notes to be added into notification templates (10 hours of work).

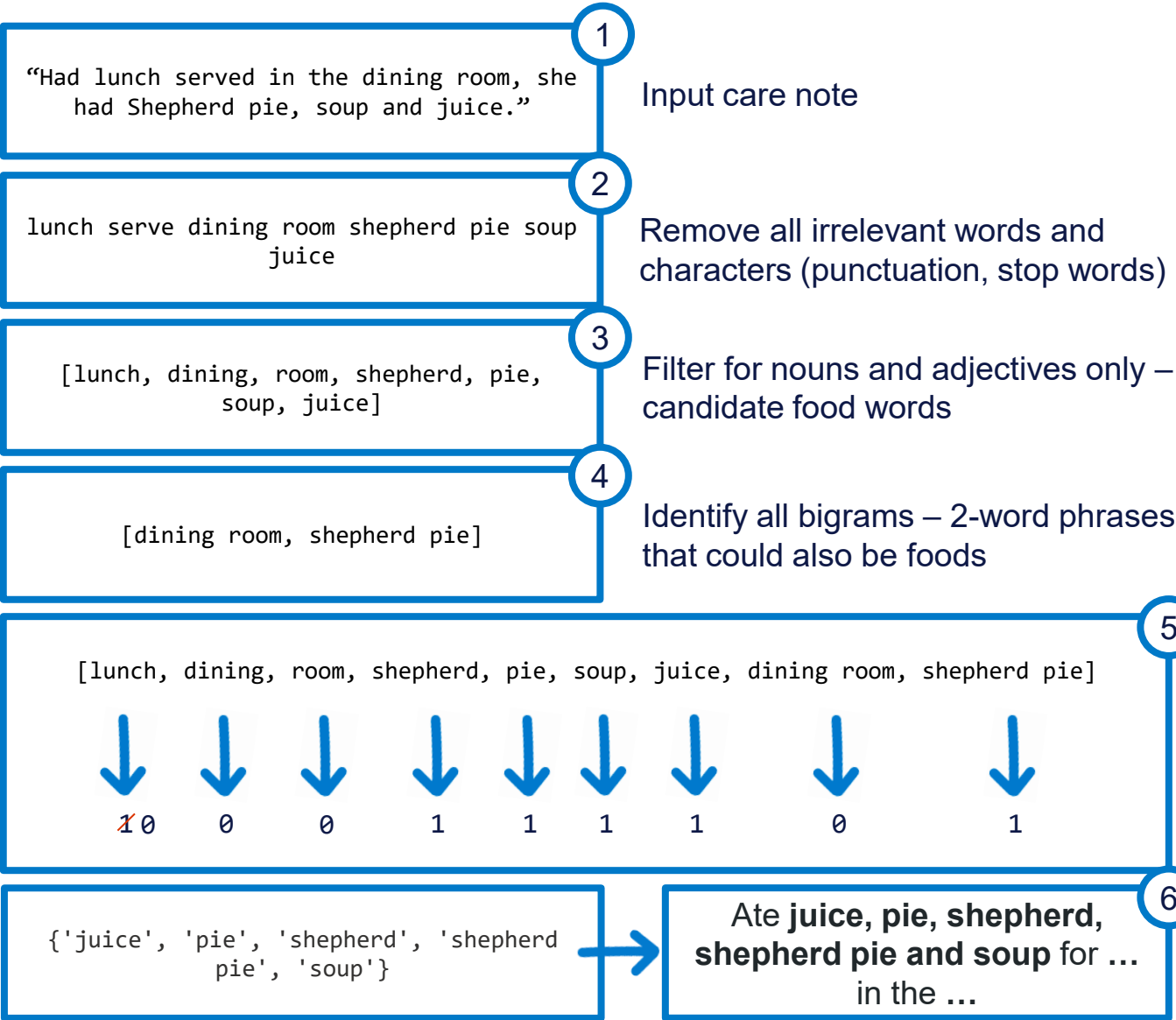
Code

Term	Food
Banana	1
Beef	1
Cereal	1
Bicycle	0
Window	0
...	...

Dataset of nouns to be compared to candidate words to identify food words (K-Nearest Neighbours).

Set of ignore words that are similar to food words but should not be classified as such.

```
ignore_words = {"meal", "breakfast", "lunch", "dinner", "snack", "food", "dish", "eat", "ate", "drink", "glass", "plate", "fork", "knife", "spoon", "cup", "ml", "nutrition", "texture", "puree", "pureed", "ingredient", "diet"}
```



Shepherd was identified as a food word as it's similar to "pie", "lamb", and "beef"

Didn't extract any other information about meal-times

AI Alternative Solution

LLM Technical Feasibility

To demonstrate the difference in complexity between using traditional NLP and LLMs, a similar (redacted and anonymised) note was passed into an LLM (2 hours of work).

Code

Template could also be populated with an LLM to create a more coherent structure

Hasn't fixed any typing errors

Ate a small plate of shepherd pie, soup, juice, pudding, and lamb dish for ... in their bedroom

Didn't request mealtime in the prompt

“Had supper in their room, they had Shepherd pie, soup, juice and pudding. had meal served in their bedroom, the lamb dish, had a small plate, was content.”

Input care note

We have resident care notes containing free text detailing all aspects of their residential or nursing care. We need to extract key information from a patient's nutrition updates to communicate to relatives their recent diet habits. The care notes can be very short or untidy so may or may not contain the information we are looking for.

From the care note, you will:

1. Determine whether the meal was eaten.
2. If specified in the note, determine how much of the meal was eaten.
3. If specified in the note, identify which foods were eaten.
4. If specified in the note, identify where the resident was when they ate the meal.

You must return the result in a JSON structure. Here's the output schema:

```
...
{
  "Eaten": true/false/"unknown",
  "Proportion": "",
  "Food(s)": [],
  "Location": "",
}
```

Do not respond with any other text. Thank you!

Instructions for GPT model – what information we want to extract and in what format.

```
{
  "Eaten": True
  "Proportion": "a small plate",
  "Food(s)": ['Shepherd pie', 'soup', 'juice', 'pudding', 'lamb dish'],
  "Location": "bedroom",
}
```

GPT response

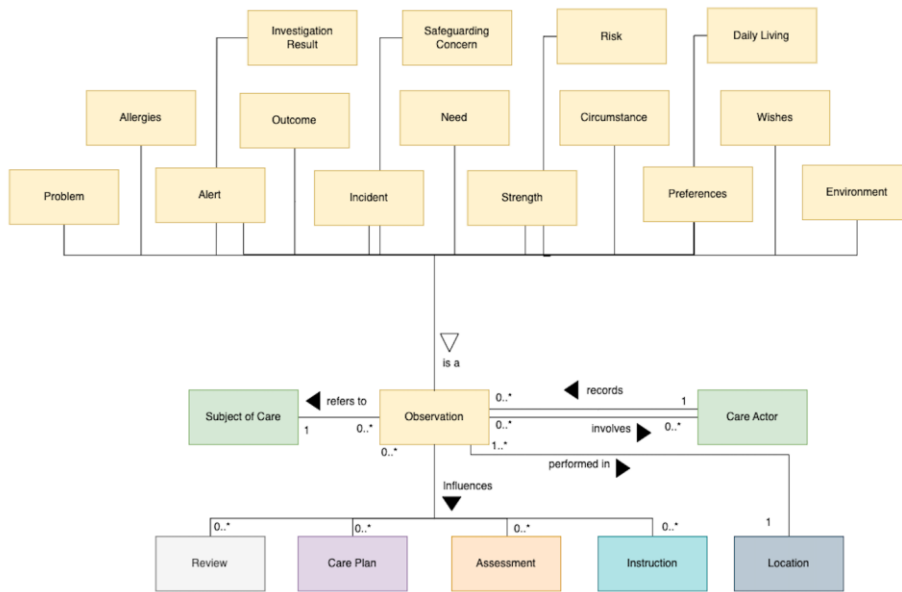
Advantages

- Utilises traditional machine learning techniques to process text – can be used within our existing environments
- Controls the structure of the notes sent to relatives, ensuring only relevant information is communicated
- Explainable – easy to understand why a certain summary was generated
- Lightweight and cheaper than an LLM (e.g. GPT)

Disadvantages

- Bespoke solution required for each KPI – one method of extracting information may not apply to another group of notes
- Time consuming to construct – took two days to build a food extraction model. Text needs cleaning and pre-processing before modelling (unlike with an LLM)
- We would need to define the templates for all notifications – time consuming
- We'd need to create a labelled dataset to test the success of the solution (out of n care notes, how many times did it successfully identify the food item)
- Very rigid – can't create a notification outside of the defined templates
- Not 100% accurate – risk of sending incorrect note to a relative (reputational damage); would require LLM to validate the note accuracy

Tolerances – “Exclude data where specific criteria is met”



Digital Social Care Standards Framework

To answer, “what information should be shared with a relative?”, we can use the Digital Social Care Observation framework.

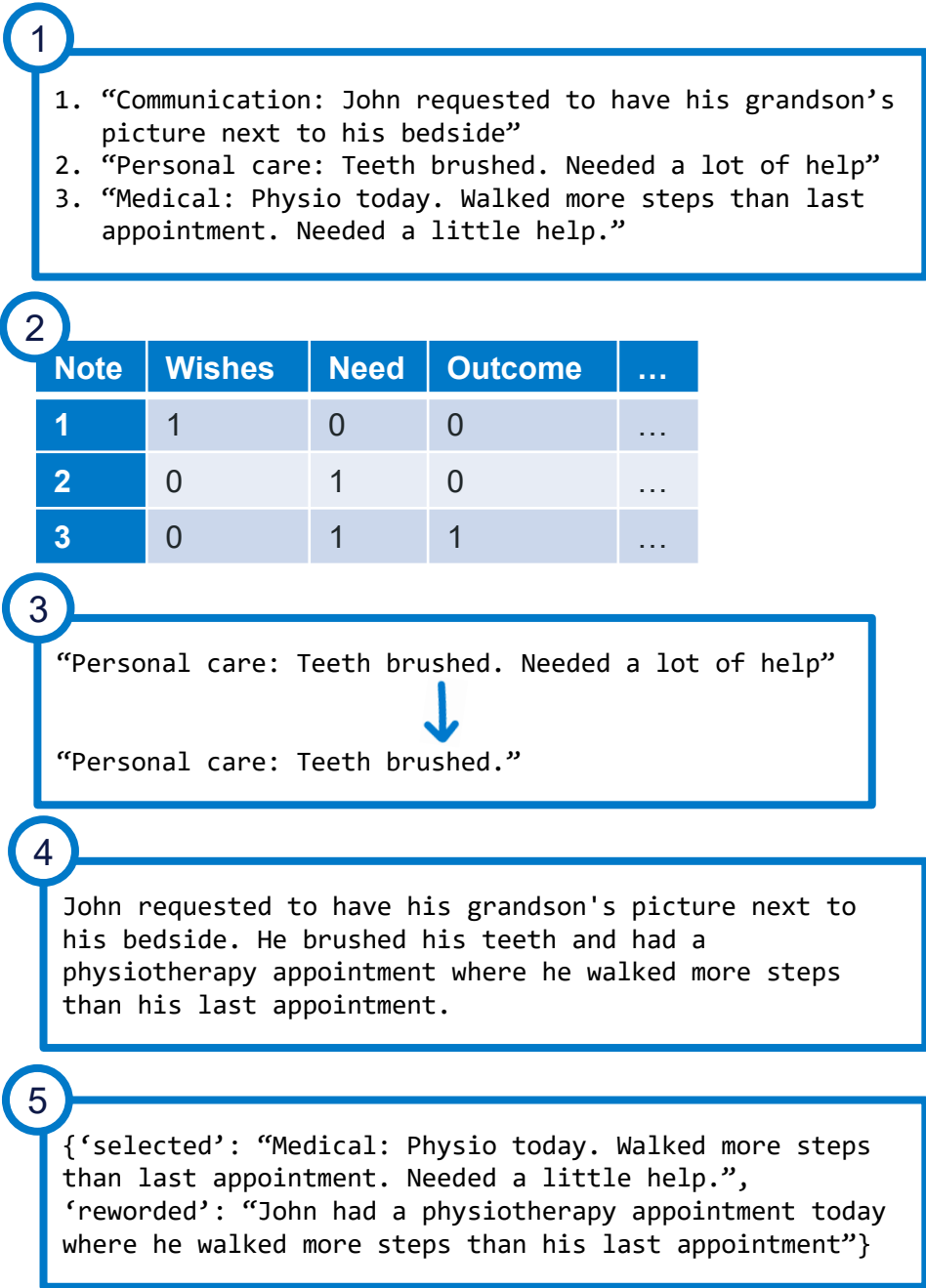
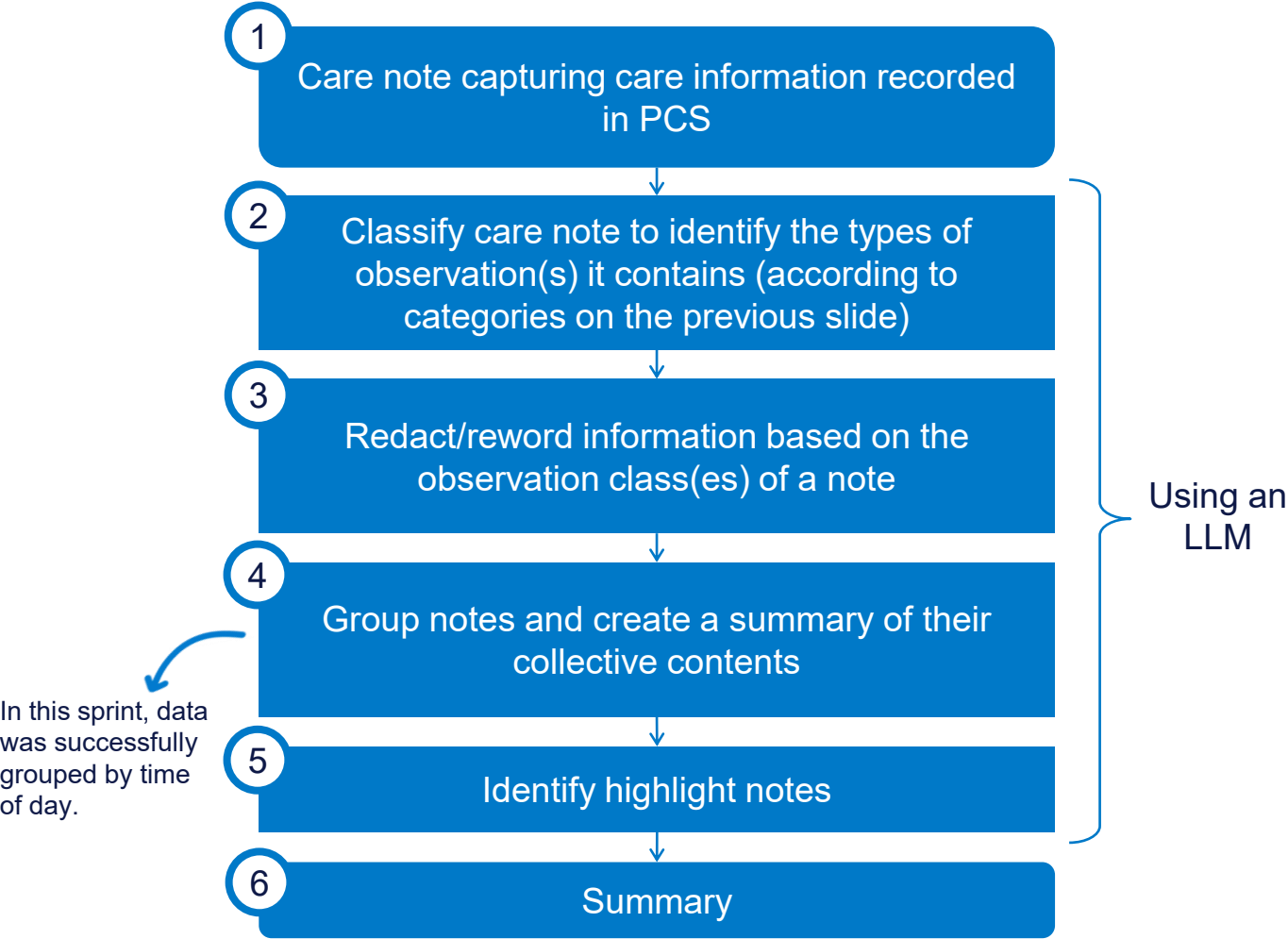
The framework defines a series of “observation” types, detailing the types of information a note can contain.

The framework is particularly useful when surfacing notifications to relatives because it provides a structured, person-centred capture of care notes information.

Thus enabling “Tolerance-Based Filtering” and supporting smart, compassionate communication, keeping relatives informed without overwhelming them with routine details.

Snowflake Cortex Exploration

With LLMs in Snowflake now available, I explored its ability to process and transform care note into a summary format.



Front End Demo

Using cortex, data was transformed and visualised in Power BI to demonstrate a potential front-end layout.

This dashboard contains **real** data – a day of notes has been processed.

Notes with “allergies” and “need” observations have been redacted. Data has been grouped by time of day.

The screenshot shows a Bupa Care Story dashboard. At the top is a dark blue header with the Bupa logo, 'Care' and 'Help' tabs, and a 'My Account' link with a dropdown arrow. Below the header, the main content area is divided into two columns. The left column contains a patient profile section with a circular profile picture placeholder and a 'View profile' link. Below this are tabs for 'Overview' and 'Care Story', with 'Care Story' being the active tab. A date selector shows '03 April 2025'. The main text area is divided into three sections: 'Morning', 'Afternoon', and 'Evening', each followed by a paragraph of text. The 'Night' section is partially visible at the bottom. The right column is titled 'Highlights' and contains three bullet points, each preceded by a redacted name. Annotations with blue arrows point to the 'View profile' link and the date selector, with text stating 'This button doesn't work' and 'This button works' respectively.

Bupa Care Help My Account

View profile

Overview **Care Story** 03 April 2025

Morning

■■■■ had a comfortable night's sleep, being checked regularly and repositioned on her left and right sides. She was administered medications as prescribed and had her pad checked, which was damp but didn't need changing. ■■■■ had a thickened juice and a pureed breakfast consisting of weetabix and yogurt, which she ate all of. She also had a glass of water, fruit juice, and tea, all prepared to a mildly thick consistency. ■■■■ had a pudding for dessert and was content throughout the day.

Afternoon

■■■■ had a busy day. She chatted with staff and residents for 30 minutes and seemed content. Her personal care included washing hands, cleaning teeth, doing laundry, and general care, all of which she was content with. Her environment was checked multiple times, ensuring her mattress, bed rails, room temperature, and pendant alarm were functioning correctly. ■■■■ had her fingernails and toenails checked and was repositioned to avoid pressure ulcers. She had cream applied as prescribed and received her COVID-19 spring booster vaccine. ■■■■ had a cup of tea and a glass of fruit juice, both prepared to a mild...

Evening

■■■■ was checked and found to be safe and content. Her pad was changed and she was repositioned to avoid pressure ulcers. She had mac and cheese puree, 2 pots of yogurt, and a cup of tea for lunch, which she ate all of and enjoyed. Later, she had a pudding and another cup of tea. She was positioned on her left side using a slide sheet and appeared content. Her pad was changed again and bed rails were checked and fitted safely.

Night

■■■■ rang the bell for cranberry juice and had a glass of fruit juice, drinking 115 ml of the offered 150 ml. She was seen by a nurse for medication and appeared comfortable in bed watching TV. ■■■■ was checked multiple times to ensure her safety and comfort, and was found to be asleep and content. She was also repositioned on her right side using a slide sheet. ■■■■'s pad was checked and found to be clean, with a BSS reading indicating bowels were not opened.

Highlights

- enjoyed her pureed breakfast of Weetabix and yogurt, fortified with sugar, and finished it all.
- had a pleasant 30-minute chat with other staff and residents.
- received her COVID vaccine spring booster from the District Nurse.

Work Completed

- **Overview Summaries**
 - Group by Care Delivery, Physical Wellbeing, Social Wellbeing, Emotional
 - Reported in Power BI
- **Classification and Summary Validation**
 - Defined custom metrics to quantify model performance

Overview Summaries

Reusing code and prompts written in the previous sprint, we were able to classify notes into the four “overview” buckets and create summaries for each.

This has been added to the original Power BI report.

 Bupa

Care

Help

 My Account 

 Janet Smith
[View profile](#)

[Overview](#) [Care Story](#)

 This button now works

 03 April 2025

Care Delivery

Gerald's care delivery was consistent and thorough. He was regularly repositioned to avoid pressure ulcers and his falls sensor device was routinely checked and confirmed to be functioning correctly. Gerald had a settled night with no concerns reported. His personal care included dressing, washing, and changing clothes, with verbal consent given for these activities. Medical care involved the application of prescribed creams and the administration of a COVID-19 vaccination, with no side effects reported. Gerald's nutrition was well-managed, with meals served in his bedroom and mostly eaten. He also received regular checks...

Physical Wellbeing

Gerald's physical wellbeing was closely monitored throughout the day. He was frequently checked while sleeping and appeared content. His mobility was managed by repositioning him to avoid pressure ulcers, and he was content with these adjustments. His personal safety was ensured with a falls sensor device, which was consistently in the correct position and activated correctly. Gerald had a settled night with no concerns handed over by the day nurse. Medical care included the application of prescribed creams, and he received a COVID-19 vaccination with no reported side effects. His nutrition was well-managed, with...

Social Wellbeing

Gerald's social wellbeing was marked by several positive interactions. He enjoyed watching TV and a film in his room, which provided social benefits and made him happy. He engaged in multiple conversations with staff and other residents, with durations ranging from 10 to 30 minutes, and was generally happy during these interactions. Gerald also expressed happiness during personal care activities, although he declined an offer to join group activities and lunch, preferring to rest. Despite not drinking any fruit juice offered, he remained content throughout the day.

Emotional

Gerald's emotional state was consistently positive throughout the period. He was content during sleep checks and mobility repositioning to avoid pressure ulcers. He enjoyed personal care activities and was happy during medical treatments, including the application of prescribed creams. Gerald engaged in social activities like watching TV and chatting with staff and residents, which brought him happiness. He was content with his meals and drinks, although he declined some offers. Overall, Gerald's emotional well-being appeared stable and positive, with no significant signs of distress or discomfort.

Highlights

Gerald enjoyed watching a film in his room for two hours and found it socially beneficial.

Gerald had a good appetite, finishing all of his lunch including dessert.

Gerald had a positive day socializing, chatting with staff and other residents multiple times.

Custom Metrics

Before exploring Snowflake's observability platform (tracking model performance with pre-built metrics), custom metrics were designed and programmed. These would support us in answering **“How well are LLMs working in my specific use case”** i.e. how well are these models summarising care notes?

Four metrics were explored:

- **When classifying observations, how consistent are the model's outputs – how confident are they in their predictions?**
- **What percentage of notes were partially and/or fully included in the summary?**
- **How correct is the order of notes in the summary?**
- **Do the summaries adhere to the length restrictions?**

Other questions such as “How much information has been made up?” should be answered using pre-built metrics such as “groundedness” in the observability platform.

Classification Performance

Classification Objective

Perform binary classification on a dataset to determine whether each care note includes a certain type of observation.

Example

Yes or no, does a care note mention that a service user required help?

“John had peas for dinner” – **No**

“Put on shoes. Needed a lot of help” - **Yes**

Validation Strategy

- 1. Choose several models
- 2. Use each model a select number of times to classify a subset of data
- 3. Group by data point (i.e. care note) and aggregate classes
- 4. Group by model and data point (i.e. care note) and aggregate classes

This allows us to quantify the consistency and thus confidence of predictions, but not the accuracy of models.

Illustrative example

1

Models: Mistral-large2, Lllama3.1-70b
Attempts: 2

2

Note	Model	Attempt	Class
“John put on his shoes. Needed a lot of help.”	Mistral-large2	1	1
		2	1
	Llama3.1-70b	1	1
		2	0

3

4

Note	Avg. Class	Model	Avg. Class
“John put on his shoes. Needed a lot of help.”	0.75	Mistral-large2	1
		Llama3.1-70b	0.5

As the average class is a fraction, not all predictions were consistent. A score closer to one suggests this note is more likely positive.

This mistral model was more consistent with predictions than the llama model suggesting a higher confidence in predictions.

Classification Results

Experiment Setup

Stat	Value
# test datapoints	336
# observation types	15
# predictions in one query	$336 * 15 = 5,040$
# attempts	4
# models	4
Total # predictions made	$5,040 * 4 * 4 = 80,640$

Missing Classifications (Model Failed to Run)

Model	% Notes Missing Class
llama3.1-8b	0.068
mistral-7b	0.032
llama3.1-70b	0.144
mistral-large2	0.18

The larger models were slightly more consistent in their responses than the smaller models

Prediction Consistency by Model

Model	% Predictions Consistent
llama3.1-70b	99.1
llama3.1-8b	97
mistral-7b	98.3
mistral-large2	99.4

Prediction Consistency (All Models)
65.5%

Each data point and class had 16 predictions (4 models, 4 attempts). This means that for all care notes and classes, 65.5% of them had 16 consistent predictions.

Individual models are very confident in their responses, but predictions can vary between models.

Sometimes, Snowflake's Complete function failed to return a response in the correct format meaning some rows are without a class

Summary Performance Partially or Fully Included Notes

Context

I work in a nursing care home, taking care of the elderly. We track resident care and wellbeing through free-text care notes. We want to share our care note data with relatives of residents so that they can stay informed on their loved one's care. We are using LLMs to summarize a list of notes into one paragraph.

Instruction

You are an expert evaluator. Your task is to assess which care notes have been fully or partially included in the AI-generated summary.

You will receive:

- A list of care notes (free-text strings)
- A summary generated by the AI

You must evaluate each note individually and label it according to how well it is reflected in the summary.

Evaluation Criteria

- ****Fully Included****: The main ideas and details of the care note are clearly and accurately represented in the summary.
- ****Partially Included****: Some of the content or main ideas from the note are represented, but not all details.
- ****Not Included****: The note is not mentioned at all or only vaguely referenced in a way that loses the core meaning.

Output Format

Return a single string, where each original note is followed by its evaluation label in this format:
<Original Note Text>:: <Label>

Each note-label pair must be separated by ';;'. The label must be one of:

- 'Fully Included'
- 'Partially Included'
- 'Not Included'

Illustrative example Note

```
[  
'Nutrition: Had oatmeal and tea for breakfast in the dining  
room. Ate most of it.',  
'Medical: Performed a skin integrity check. A small bruise  
on his left arm was identified. Datex raised.',  
'Activities: Participated in a group exercise session for  
30 minutes. He was engaged and followed instructions  
well.',  
'Continence care: Pad changed as was soiled.'  
]
```

AI-Generated Summary

“John enjoyed some oatmeal and tea for breakfast in the dining room. Afterwards, a skin integrity check was performed. Next, John participated in a group exercise session for 30 minutes. He was engaged and followed instructions well.”



```
Nutrition: Had oatmeal and tea for breakfast in the dining  
room. Ate most of it.:Fully Included;;  
Medical: Performed a skin integrity check. A small bruise  
on his left arm was identified. Datex raised.:Partially  
Included;;  
Activities: Participated in a group exercise session for 30  
minutes. He was engaged and followed instructions  
well.:Fully Included;;  
Continence care: Pad changed as was soiled.:Not Included
```

Partially/Fully Included Metric

Having used an LLM to identify which notes were/weren't in a summary, we can quantify how much information was included:

$$\text{Fraction of Notes} = \frac{\text{Number of Included Care Notes}}{\text{Total Number of Care Notes Provided}}$$

The issue with this metric is that it doesn't consider how many notes the model needed to summarise.

Imagine I'm handing out smarties.

If I had 10 smarties and gave you 9, you'd be receiving 90% of all smarties and might be thinking that you had room for one more.

If I had 100 smarties and gave you 50, even though you only received 50%, you might not be able to eat more than that.

The second scenario looks worse on paper as you received a smaller fraction.

The same can be applied to care notes: if you have many notes to summarise, how many can we realistically fit in small word limit?

We can alter the metric to consider number of notes available:

$$\text{Normalised Inclusion} = \frac{\ln(\text{Number of Included Care Notes})}{\ln(\text{Total Number of Care Notes Provided})}$$

This means, for larger note counts, less notes are needed to output a larger metric score.

Number of notes included in a summary to achieve a given metric score

		Metric Score		
		0.5	0.8	0.9
# Notes	10	3	6	9
	20	4	11	17
	50	7	23	41
	100	10	40	79

Number of notes featured in a summary

Partially/Fully Included Results

12 summaries were generated. From these, we calculated how many notes were sent to the model to generate the summary with and how many were included.

We are seeing most notes are being used in summaries. Typically, the more notes are available, the less are being included.

Summary	Note Count	Partially or Fully Included	Included Fraction	Normalised Inclusion (In)
0	10	10	1	1
1	47	43	0.92	0.98
2	19	19	1	1
3	11	11	1	1
4	33	29	0.88	0.96
5	11	11	1	1
6	22	16	0.73	0.9
7	43	34	0.79	0.94
8	9	6	0.67	0.82
9	28	18	0.64	0.87
10	11	11	1	1
11	86	48	0.56	0.87

In this summary, this person only had 50 unique notes. Maybe we can reduce the information processed by the model by only using unique notes...

Average	0.89	0.96
---------	------	------

! The model identifying which notes have been included in the summary may be wrong. This should be validated with a labelled dataset !

Summary Performance

Note Order

Context

I work in a nursing care home, taking care of the elderly. We track resident care and wellbeing through free-text care notes. We want to share our care note data with relatives of residents so that they can stay informed on their loved one's care. We are using LLMs to summarize a list of notes into one paragraph.

Instruction

You are an expert evaluator. Your task is to determine the order in which notes appear in an AI-generated summary. We will provide you with:

- A list of care notes. Each note has a unique numeric ID starting from 1.
- A summary generated by an AI.

You must return a list of the IDs that correspond to the notes included in the summary, in the order they appear in the summary. This list must include any note that is fully or partially represented in the summary. If a note does not appear at all, do not include its ID.

Evaluation

Criteria

- Inclusion: A note is considered included if it appears fully or partially in the summary.
- Order: Only include notes that appear in the summary and return them in the order they appear.

Data Formats

Input Data

The data I will provide you will follow this structure:...

Your Response

You must return ONLY a list of the note IDs, in the order they appear in the summary. Separate them using double semicolons (;). If no notes are included in the summary, return nothing.

Illustrative example

Note

```
[
  {'id': 1, 'note': 'Nutrition: Had oatmeal and tea for breakfast.'},
  {'id': 2, 'note': 'Medical: Performed a skin integrity check. Small bruise on left arm.'},
  {'id': 3, 'note': 'Activities: John participated in group exercise for 30 minutes.'},
  {'id': 4, 'note': 'Continence care: Pad changed as was soiled.'}
]
```

AI-Generated Summary

John had tea and oatmeal this morning and later took part in a 30-minute group exercise session. His skin was checked, and a small bruise was found on his left arm.



1;;3;;2

Note Order Correctness

12 summaries were generated. Having identified the order notes appear in a summary, the Kendall-Tau metric was used to compare the similarities between the expected and true orders.

Values are between -1 and 1 with 1 suggesting a perfect positive correlation between the two orders and vice versa. 0 would suggest no commonality between the order of notes passed to the model and the order of notes as they appear in the summary.

Most note orders are near perfect – the LLM is considering chronological order when building summaries, but it isn't perfect.

Summary	Kendall-Tau
0	0.97
1	0.93
2	1
3	0.78
4	1
5	0.93
6	1
7	0.79
8	0.91
9	0.94
10	0.52
11	1
Average	0.9

! The model identifying note appearance order may be wrong. This should be validated with a labelled dataset !

Summary Performance

Lengths

We asked the LLM to create summaries of 100 words. Did the model adhere to this condition? This can be quantified by calculating **the average summary word count** and comparing to the limit.

The model typically didn't adhere to the word count limit. This should be improved in the prompt.

This doesn't correlate strongly to the number of notes to summarise – comparing combined note length to the word count of the summary, the correlation coefficient was 0.6.

Summary	Word Count
0	119
1	161
2	142
3	108
4	105
5	170
6	105
7	133
8	80
9	97
10	139
11	98
Average	121

Next Steps

- **Investigate why the Complete function fails with Snowflake (AI Squad)**
 - Currently using TRY_COMPLETE instead, but this leads to missing summaries and classifications
 - Study the notes without classifications – is there a common pattern between them?
- **Investigate pre-built metrics in the AI Observability platform in Snowflake**
- **Automatically generate summaries daily for one user and manually assess results**
- **Determine metric targets (Care Services)**
 - Label a dataset of notes to calculate classification accuracy
 - Determine desired partially/fully included scores for different note categories
 - Determine whether chronological reporting of notes is necessary
- **With metric targets set, improve prompts**
 - Track prompt versioning

Snowflake AI Observability (Now Working)

Evaluation metrics provide a quantifiable way to measure the accuracy and performance of your application. You can compute metrics with the “LLM-as-a-judge” approach. With this approach, an LLM is used to generate a score (between 0 - 1) with an explanation for the application’s output.

AI Observability supports:

- *Context Relevance* determines if the retrieved context from the retriever or the search service is relevant to the user query. This is not relevant in our use-case as we are not searching for information.
- *Groundedness* determines if the generated response is supported by and grounded in the retrieved context from the retriever or the search service.
- *Answer relevance* determines if the generated response is relevant to the user query.
- *Correctness* determines how aligned the generated response is with the ground truth. This is not relevant.
- *Coherence* measures if the generated response of the model is coherent and doesn’t introduce logical gaps, inconsistencies or contradictions.

Work Completed

TRY_COMPLETE Errors Investigated

- We created a notebook recreating the issues with gaps in the data
- Snowflake identified the bug and have resolved the issue

Highlight Notes

- We explored a methodology for calculating “note rarity”

Reporting Metrics – Robust, Reusable Python Classes

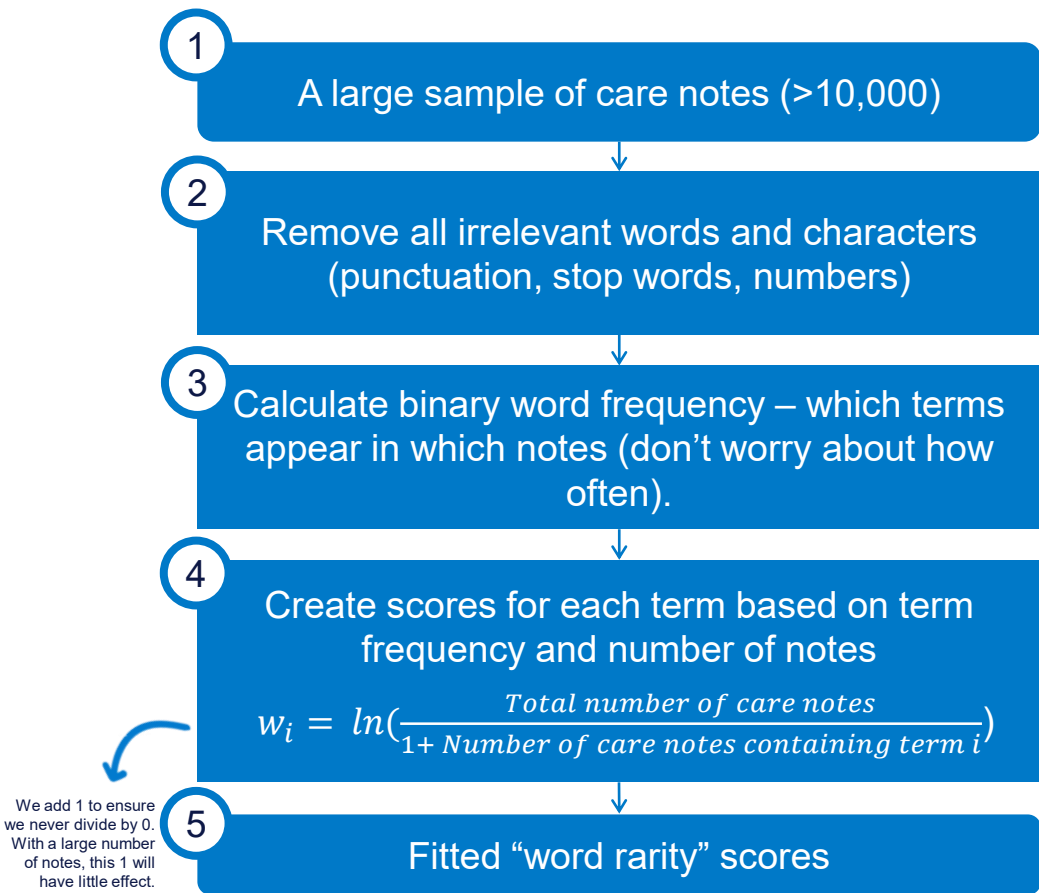
- **Note Inclusion**
 - Now batch processes – only look for the presence of a subset of notes, sending them in batches instead of all of them
- **Order Consistency**
 - Now uses samples to test chronological order instead of sending all notes
 - This can be repeated with several samples to create several scores to be aggregated
- **Word Limit Compliance and Classification Consistency**

Highlight Notes

Model Fitting

A strategy for identifying significant notes was explored. This doesn't use LLMs. Firstly, we needed to learn what type of information an "important" note would contain. Therefore, we studied the terms that appear in a large sample of notes.

Code



- 1

1. "Had cereal for breakfast. Was very content."

2. "Checked. Was awake. Was content."

3. "Checked and applied cream. Prescribed cream. Was not content."

4. "Checked. Was asleep. Was content."
- 2

1. cereal breakfast content

2. check awake content

3. apply cream prescribe cream content

4. check asleep content

3

Note	Cereal	breakfast	content	check	awake	apply	cream	prescribe	asleep
1	1	1	1	0	0	0	0	0	0
2	0	0	1	1	1	0	0	0	0
3	0	0	1	1	0	1	1	1	0
4	0	0	1	1	0	0	0	0	1

"Document-term matrix". Each row is a care note. Each column is a word. Cell values show whether the word appeared in the note.

4

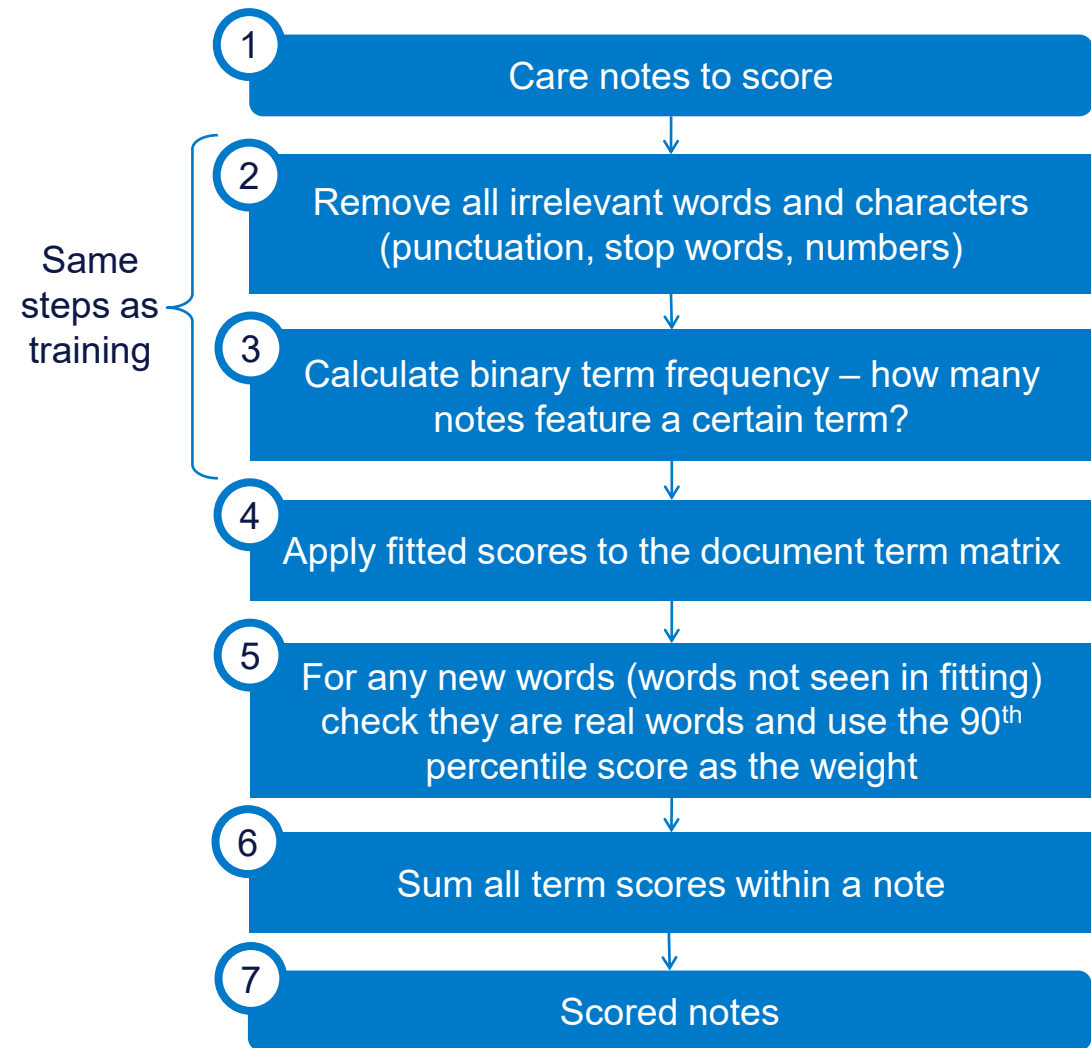
Term	Score
Cereal	0.7
Breakfast	0.7
Content	-0.2
Check	0
Awake	0.7
Apply	0.7
Cream	0.7
Prescribe	0.7
Asleep	0.7

If we had more "asleep" notes, the word "asleep" would seem less important.

Highlight Notes

Data Transformation

Having created scores for each word that we think could appear in a care notes, we can create a score for each note based on its contents.



Term	Weight
Cereal	0.7
Breakfast	0.7
Content	-0.2
Check	0
Awake	0.7
Apply	0.7
Cream	0.7
Prescribe	0.7
Asleep	0.7

1

1. "Had cereal for breakfast. Was very content."
2. "Checked. Was awake. Was content."
3. "Went skydiving!"

2

1. cereal breakfast content
2. check awake content
3. skydive

3

Note	Cereal	breakfast	content	check	awake	apply	cream	prescribe	asleep
1	1	1	1	0	0	0	0	0	0
2	0	0	1	1	1	0	0	0	0
3	0	0	0	0	0	0	0	0	0

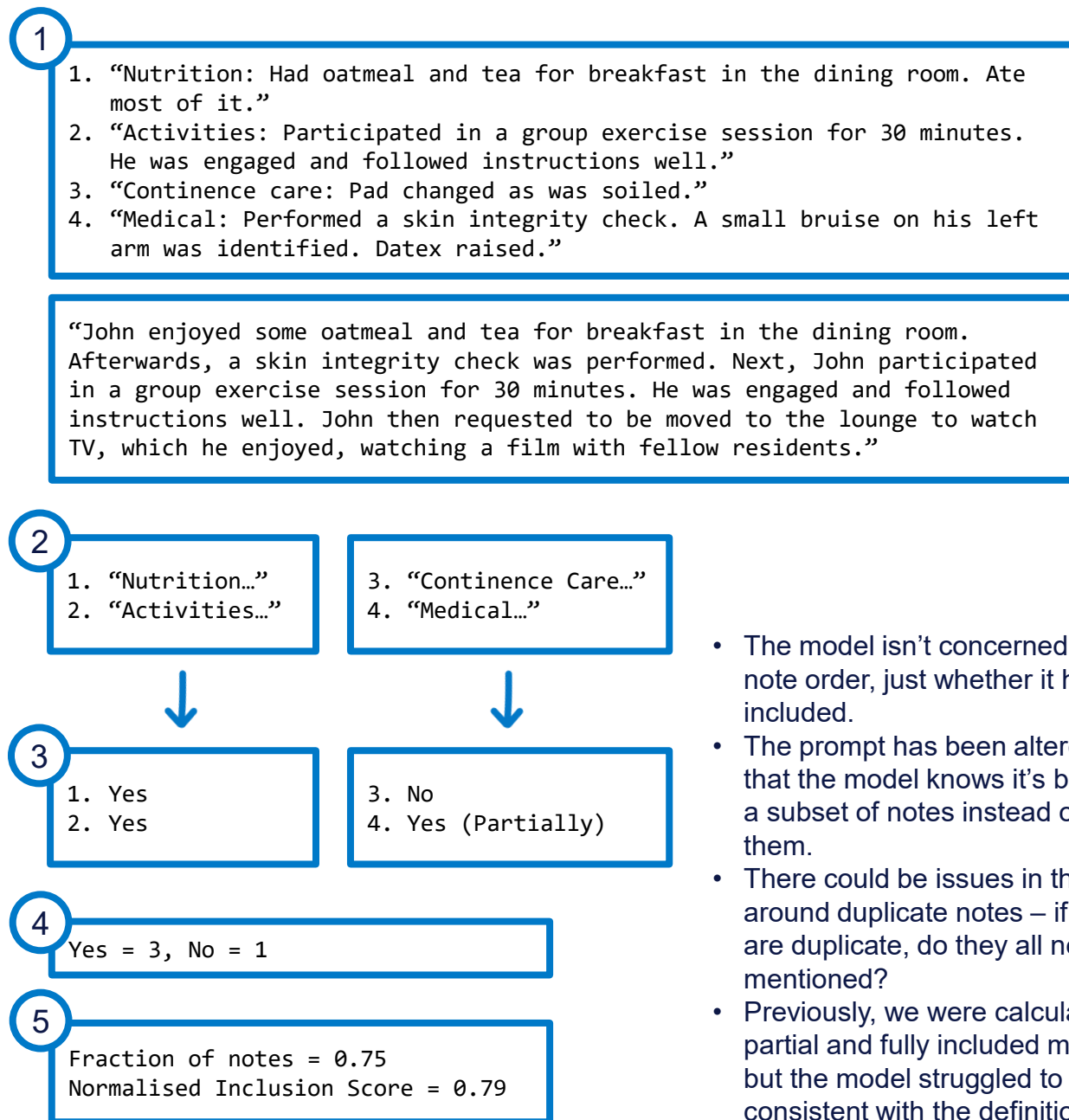
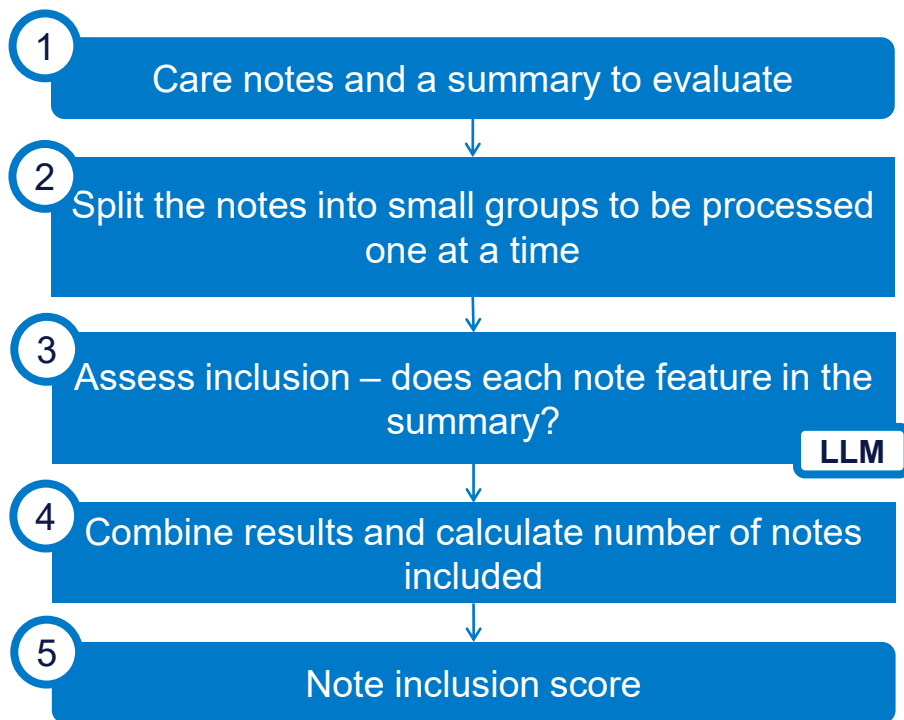
4

Note	Cereal	breakfast	content	check	awake	apply	cream	prescribe	asleep	skydive	Sum
1	0.7	0.7	-0.2	0	0	0	0	0	0	0	1.2
2	0	0	-0.2	0	0.7	0	0	0	0	0	0.5
3	0	0	0	0	0	0	0	0	0	0.7	0.7

Note Inclusion

Metric Improvement

The metric was changed to process notes in batches. This way, the model is being sent less information and is less likely to hallucinate. It can focus on smaller chunks of information at a time, improving the accuracy of its results. While more model calls are being made, each call has less tokens. [Code](#)

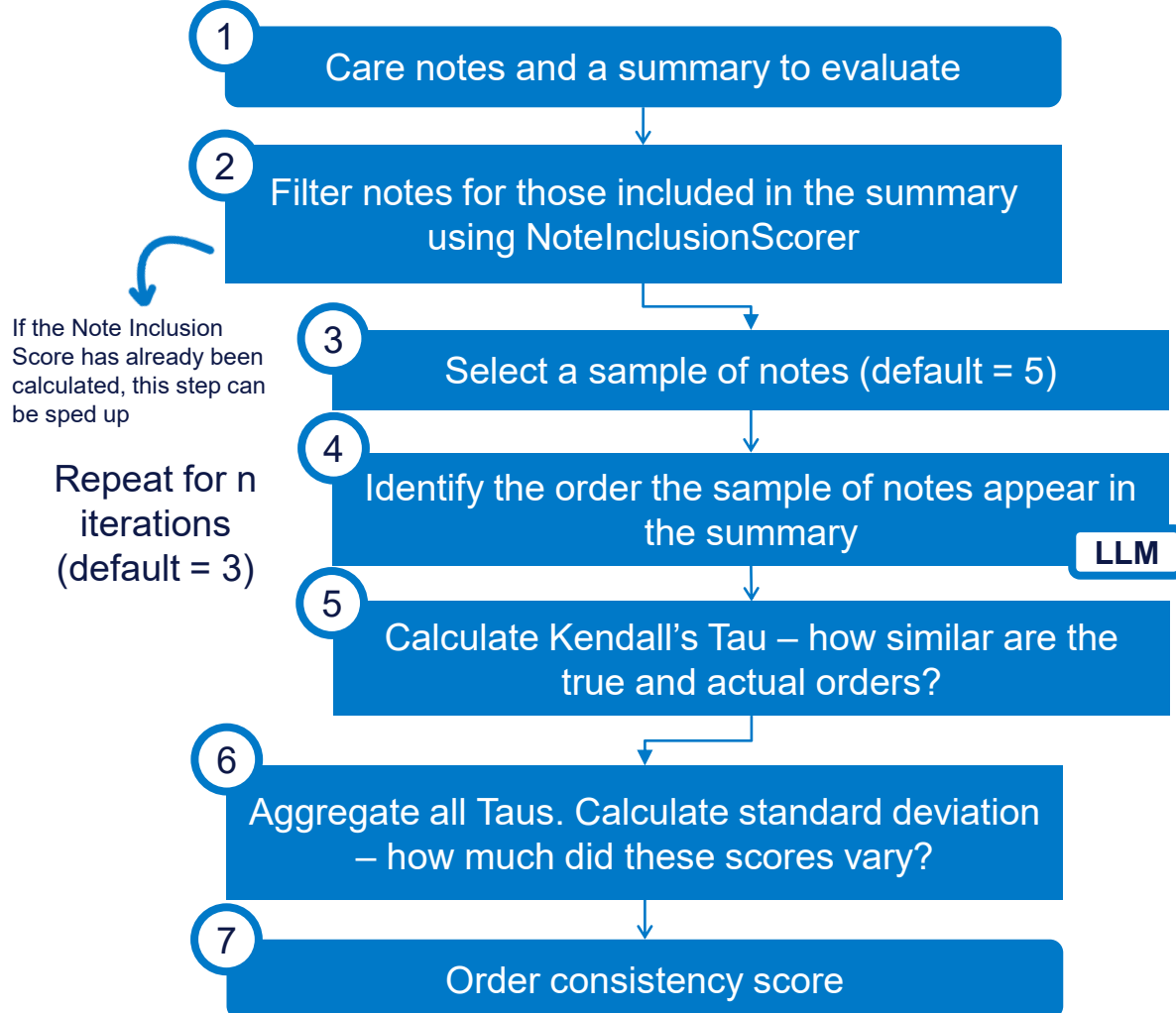


- The model isn't concerned about note order, just whether it has been included.
- The prompt has been altered so that the model knows it's being sent a subset of notes instead of all of them.
- There could be issues in the future around duplicate notes – if there are duplicate, do they all need to be mentioned?
- Previously, we were calculating partial and fully included metrics, but the model struggled to remain consistent with the definitions of each category.

Order Consistency Metric Improvement

This metric was changed to use a sample of notes instead of all at once. This allows us to confidently estimate consistency without sending large packets of information to the model.

Imagine a longer list of notes and more complex summary...



1

- ...
- “Nutrition: Had oatmeal and tea for breakfast in the dining room. Ate most of it.”
 - “Activities: Participated in a group exercise session for 30 minutes. He was engaged and followed instructions well.”
 - “Contenance care: Pad changed as was soiled.”
 - “Medical: Performed a skin integrity check. A small bruise on his left arm was identified. Datex raised.”
- ...

“...John enjoyed some oatmeal and tea for breakfast in the dining room. Afterwards, a skin integrity check was performed. Next, John participated in a group exercise session for 30 minutes. He was engaged and followed instructions well. John then requested to be moved to the lounge to watch TV, which he enjoyed, watching a film with fellow residents...”

3

```
n_samples = 3
1. “Nutrition...”
2. “Activities...”
3. “Medical...”
```

4

```
1. “Nutrition...”
3. “Medical...”
2. “Activities...”
```

5

Tau = 0.33

6

Calculated Taus = [0.33, 0.5, 0.2]
Mean = 0.34
Standard deviation = 0.12

- The prompt has been altered so that the model knows it's being sent a sample of notes instead of all of them.
- The standard deviation shows variation in calculated scores – if Kendall's Tau is consistent, it shows the order of notes is also consistent (right or wrong). Alternatively, a high variation can show a section of the summary or a small segment of notes differ greatly.
- There could be issues in the future around duplicate notes – if there are duplicate, do they all need to be mentioned?

Summary Evaluation Metrics

95.5%

Mean percent of notes
included in the summary
(normalised)

0.54

Mean order consistency score
(value between -1 and 1)

33%

Percent of summaries that
comply to the word order limit

110words

Mean summary length

Work Completed

TRY_COMPLETE still errors ☹️

Tracking prompt version

Prompt tables

New common metrics

Confluence page: [LLM Evaluation - BGIUK AI - Confluence](#)

Work Completed

Reviewed v1.5 wireframe with Frog

Explored how significantly summaries change by hour

More information on technical
method on slide 7

SUMMARY

- OPTION 1: USER PREFERENCE**
ALLOW USERS TO SELECT WHETHER TO INCLUDE MEDICAL AND/OR SOCIAL UPDATES. USE THIS TO FILTER NOTES BEFORE SUMMARISING.
- ☒ MEDICAL
☐ SOCIAL

OPTION 2: RESIDENT DEPENDENCY

PRIORITISE MEDICAL OR SOCIAL NOTES BASED ON RESIDENT DEPENDENCY

if dep < 4:
social
else if 3 <= dep < 8:
mix
else if dep > 8:
medical

CLASSIFY, FILTER,
SUMMARISE
NOTES AS PREVIOUSLY
EXPLORED

- + DO YOU HAVE TONE GUIDANCE?
- + WHAT INFORMATION CANNOT BE INCLUDED?
- + IS THERE A WORD LIMIT?
- + BASED ON THE 4 TYPES OF USERS, HOW ARE WE CONTROLLING WHAT SUMMARIES THEY SEE?

Features explored in this
notebook

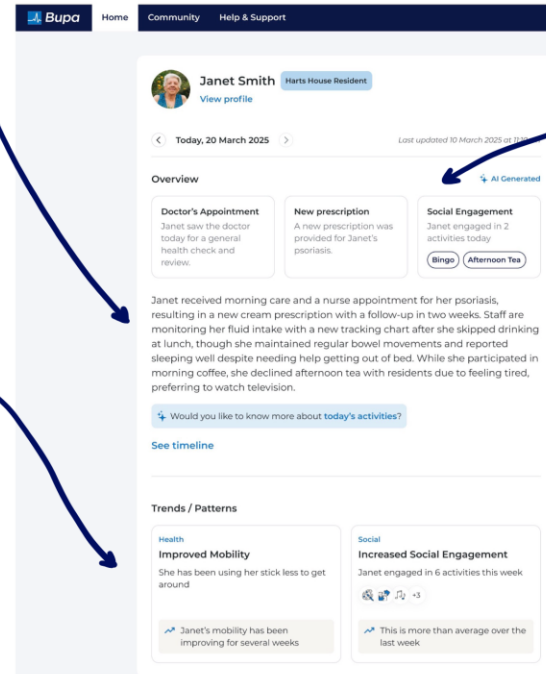
1. NOTES
DETAILING WELLBEING:
"Had breakfast, ate half"
2. CREATE FEATURES:
BREAKFAST LUNCH DINNER
Half All Most
3. MONITOR OVER TIME:
DATE BREAKFAST LUNCH DINNER
1/2 Half All Most
2/2 All All Half
...
1/6 None Half None
4. ANALYSE CHANGE

TRENDS

USING SINGLE
VIEW OF A RESIDENT,
CREATE FEATURES TO
QUANTIFY DAILY
WELLBEING AND
ANALYSE CHANGE OVER
TIME.

- + DO WE HAVE DEFINED MEASURES FOR WHICH WE WANT TO OBSERVE CHANGE?
- + IS DATA FOR THIS CAPTURED IN NOTES?
- + HOW WILL NEGATIVE TRENDS BE REPORTED?

MVP 1.5 - Iterated



GENERAL QUESTIONS?

- + WHAT IF SOMEONE ISN'T HOME?
→ HOSPITALISED?
→ PASSED AWAY?
- + ARE THERE ANY OTHER FEATURES REQUIRING AI?

HIGHLIGHTS

1. PRIORITISE CERTAIN TYPES OF NOTES
→ ACTIVITIES
→ APPOINTMENTS
→ POSITIVE SENTIMENT(?)
2. DETERMINE NOTE "RARITY"

- + WHICH TYPES OF NOTES CAN/CAN'T WE USE AS HIGHLIGHTS?
- + ARE WE ONLY INTERESTED IN POSITIVE SENTIMENT?

1. CREATE A LARGE SET OF NOTES
2. ANALYSE WORD FREQUENCY TO IDENTIFY WHICH APPEAR MOST COMMONLY
3. APPLY THESE LEARNED WEIGHTINGS TO NEW NOTES TO DETERMINE RARITY

- + WHAT IF THERE ARE NO HIGHLIGHTS IN A DAY?

More information on technical
method on slides 23, 24

3. USE NOTE TYPE AND RARITY TO DETERMINE A SHORTLIST.

4. SELECT THE BEST AND REWORD.

MOBILITY - # TIMES MOVING/HOW MUCH HELP
NUTRITION - HOW MUCH EATEN / DRUNK
ACTIVITIES - # TIMES JOINED IN
COMMUNICATION - SPOKE TO OTHERS / STAFF

QUALITY STRONGLY DEPENDENT ON
STAFF CONSISTENTLY RECORDING NOTES

Summary Change by Hour

When reviewing the latest wireframe with Capgemini, we discussed the positives and negatives of generating summaries in real-time instead of in batch. One concern raised was that if a relative checks the app often, only a few notes will have been added to the resident's timeline, and we'd be regenerating a similar summary. This could be quite expensive and wouldn't be providing the user with more information.

To test this, we have randomly selected users, taken their notes from the morning of 17th June, and generated summaries. We started with all notes between 6am and 8am and have iteratively added another hour of notes to see how the summary has changed and how much more information has been made available. All summaries used the same prompt, and no notes were excluded or filtered.

The following slides show the results of this exercise and are in the below format:

Timeframe: AI-generated note	# notes delta
-------------------------------------	-------------------------

This has been explored in the following notebook: [250619 Small Summary Test - Snowflake](#)

6am – 8am: <Resident> was checked and found to be asleep. Later, she was awake and content. Her pad was changed as it was very wet, and she was content with the change. <Resident> was seen by a nurse and remains frail but stable. She took her prescribed bedtime medications with consent and settled, appearing asleep during safety checks. No nausea or vomiting was reported.

4

6am – 9am: <Resident> was checked and found asleep initially, then awake and content. Her pad was changed as it was very wet. She was seen by a nurse and took her bedtime medications, remaining stable and comfortable. <Resident> received assistance with personal care, including teeth cleaning, skin integrity checks, and application of Sudocrem. She visited the hairdresser and had her toe nails checked. Her mattress and room were checked for safety. <Resident> chatted with staff and other residents, had her legs elevated, and her water temperature checked. She was dressed, had her hair brushed, washed, and changed clothes with assistance. Her personal hygiene was performed with consent, and she was bright and happy. Her finger nails were checked, cream applied, and she was helped to get out of bed and into a chair. Her pendant alarm and bed rails were checked, and her eyes and ears were examined. She was assisted with walking and her bed was made with fresh linen. Curtains were adjusted to suit the time of day.

38
+34

6am – 10am: <Resident> was checked and found asleep initially, then awake and content. Her pad was changed as it was very wet. She was seen by a nurse and took her bedtime medications, remaining stable. <Resident> was assisted with personal care and was comfortable in the armchair. She had her teeth cleaned and skin integrity checked, with an area of redness noted. Sudocrem was applied as prescribed. <Resident> visited the hairdresser and had her toe nails checked. She chatted with staff and other residents. Her mattress and water temperature were checked, and she was dressed and had her hair brushed with assistance. <Resident> had her personal hygiene performed and was bright and happy. She had her hands washed, laundry done, and was helped to get on and off the bed. Her pendant alarm and bed rails were checked. <Resident> had her eyes and ears checked and was assisted with walking. Her bed was stripped and made, and curtains adjusted. She had her pad changed due to wetness. For her meal, <Resident> had Ryvita with butter and marmalade, a mug of tea, mandarins, and yogurt, which she ate all of and was happy. She also had a cup of tea and drank 200 ml.

42
+4

6am – 11am: <Resident> was checked and found asleep initially, then awake and content. Her pad was changed due to wetness, and she was seen by a nurse who noted she was frail but stable. <Resident> took her bedtime medications and settled comfortably. She received personal care, including teeth cleaning, skin checks, and Sudocrem application for redness. She visited the hairdresser and had her toe nails checked. <Resident> was assisted with dressing, washing, and changing clothes. She had breakfast in her bedroom, consisting of Ryvita with butter and marmalade, tea, mandarins, and yogurt, and later had a cup of tea. Throughout the day, she was checked multiple times and found to be content and comfortable in her armchair.

43
+1

6am – 12pm: <Resident> was checked and found asleep initially, then awake and content. Her pad was changed due to wetness, and she was seen by a nurse who noted she was frail but stable. She took her bedtime medications and settled comfortably. <Resident> received assistance with personal care, including teeth cleaning, skin checks, and dressing. She had Sudocrem applied to a red area on her groin and bottom. <Resident> visited the hairdresser and had her toe nails checked. She chatted with staff and other residents and had her legs elevated. Her room and bed were checked for safety. <Resident> had breakfast in her bedroom, consisting of Ryvita with butter and marmalade, a mug of tea, mandarins, and yogurt. She also had a cup of tea later. Throughout the day, <Resident> was content and happy, receiving help as needed.

43
+0

6am – 8am: <Resident> was checked and found to be awake and content. Her pad was changed as it was wet. She had a glass of fruit juice, drinking 65 ml out of 150 ml offered. <Resident> needed assistance with various tasks such as checking her ears, applying Zerobase 11% cream, washing, dressing, brushing her hair, and checking her toe and finger nails. Medi Derma-Pro skin protectant ointment was applied as prescribed. Her pad was changed again due to being wet and soiled, with a BSS reading of type 5. <Resident> had another glass of fruit juice, drinking 160 ml out of 200 ml offered. She was checked later and appeared to be asleep and content.

21

6am – 9am: <Resident> was checked and found to be content. Her pad was changed as it was wet. She had a glass of fruit juice, drinking 65 ml out of 150 ml offered. <Resident> needed assistance with various tasks such as checking her ears, applying Zerobase 11% cream, washing, dressing, brushing her hair, checking her toe and finger nails, and changing her clothes. Medi Derma-Pro skin protectant ointment was applied as prescribed. Her pad was changed again due to being wet and soiled, with a BSS reading of type 5. She had another glass of fruit juice, drinking 160 ml out of 200 ml offered. Finally, <Resident> was checked and appeared to be asleep and content.

21

+0

6am – 10am: <Resident> was checked and found to be awake and content. Her pad was changed as it was wet. She had a glass of fruit juice, drinking 65 ml out of 150 ml offered. <Resident> needed assistance with various tasks such as checking her ears, applying Zerobase 11% cream, washing, dressing, brushing her hair, and checking her toe and finger nails. She was content throughout these activities. Medi Derma-Pro skin protectant ointment was applied as prescribed. Her pad was changed again due to being wet and soiled, with a BSS reading of type 5. <Resident> had another glass of fruit juice, drinking 160 ml out of 200 ml offered. She was checked later and appeared to be asleep and content. Night staff applied Zerobase 11% cream and Medi Derma-Pro ointment, checked her skin integrity, and cleaned her teeth and mouth. <Resident> did not want to wear her hearing aid and kept removing it, but was content overall.

30

+9

6am – 11am: <Resident> was checked and found to be content. Her pad was changed as it was wet. She had a glass of fruit juice and drank 65 ml. Her ears, eyes, and nails were checked, and she needed help most of the time but was content. Zerobase 11% cream and Medi Derma-Pro skin protectant ointment were applied as prescribed. <Resident> was washed, dressed, and had her hair brushed, needing help most of the time. Her pad was changed again due to soiling, and her BSS reading was type 5. She had another glass of fruit juice, drinking 160 ml this time. Later, she was checked and appeared to be asleep. Night staff applied Zerobase cream and Medi Derma-Pro ointment, checked her skin integrity, and cleaned her teeth and mouth. <Resident> took her Actagain juice supplement, drinking 200 ml, and had a glass of water with her medications, drinking 150 ml.

34

+4

6am – 12pm: <Resident> was checked and her pad was changed as it was wet. She was awake and content. She had a glass of fruit juice, drinking 65 ml out of 150 ml offered. Her ears, eyes, and nails were checked, and she needed help most of the time but was content. Zerobase 11% cream and Medi Derma-Pro skin protectant ointment were applied as prescribed. <Resident> was washed, dressed, and had her hair brushed, needing help most of the time but remaining content. Her pad was changed again due to being wet and soiled, with a BSS reading of type 5. She had another glass of fruit juice, drinking 160 ml out of 200 ml offered. At night, <Resident> was checked and appeared to be asleep and content. Night staff applied Zerobase 11% cream and Medi Derma-Pro skin protectant ointment, checked her skin integrity, and cleaned her teeth and mouth. <Resident> took her Actagain juice supplement, drinking all 200 ml offered, and had a glass of water with her medications, drinking 150 ml out of 280 ml offered. Her pad was changed again as it was wet. She was repositioned to avoid pressure ulcers and was content.

37

+3

Janet's Week

Spotify wrapped

Runs once a week

<https://www.figma.com/board/CpYpM3EAuSyEvQWKKWyB2C/MVP-1.5-Data-Review?node-id=114-3464>

<https://www.figma.com/board/AZHMw2rvUpIDYX20XIT3MU/17-04-Show---Tell?node-id=0-1&p=f&t=XYsmgDk7jgvMIVB5-0>

There will be no pictures

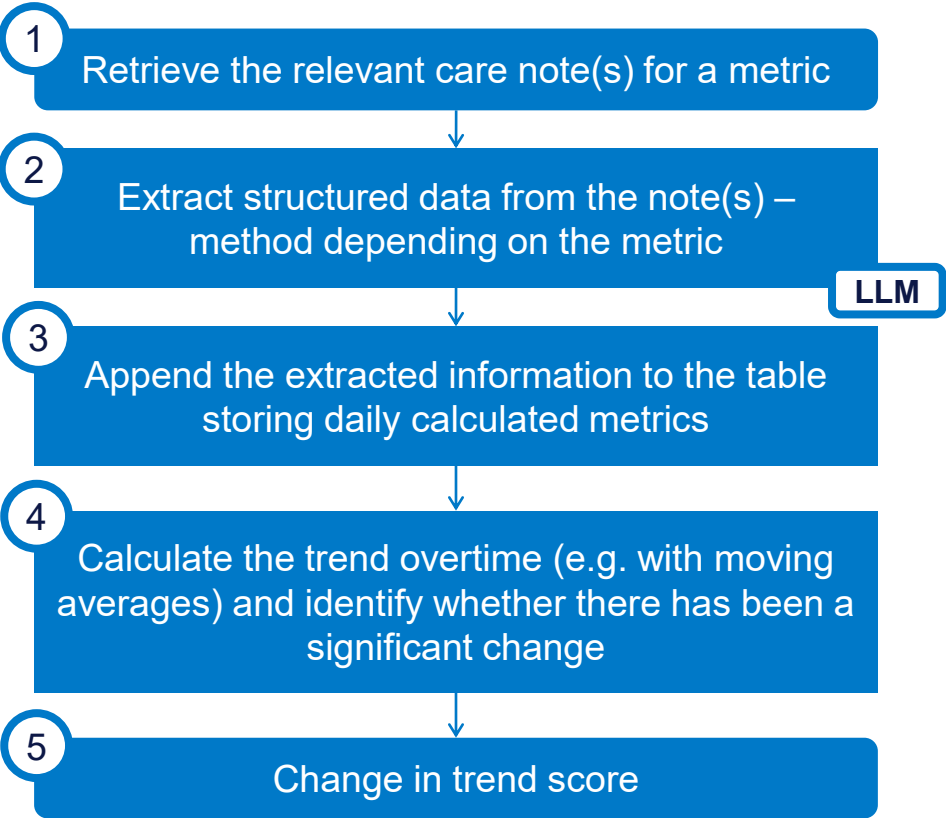
Trends

We considered how we could detect changes in physical and social wellbeing of a resident using care note data.

Real collection of mobility notes over one day for one resident

1

Resident	Datetime	Note
abc	2025-03-23 11:42	Help with walking using his frame, needed help most of the time, used a zimmer frame, transferred to their room, was content.
abc	2025-03-23 11:42	Helped to get on to his recliner after shower, was content.
abc	2025-03-23 17:27	Help with walking, needed help most of the time, used a Zimmer frame, transferred to their room, was content.
abc	2025-03-23 17:27	Helped to get back on to his armchair after toileting, was content.

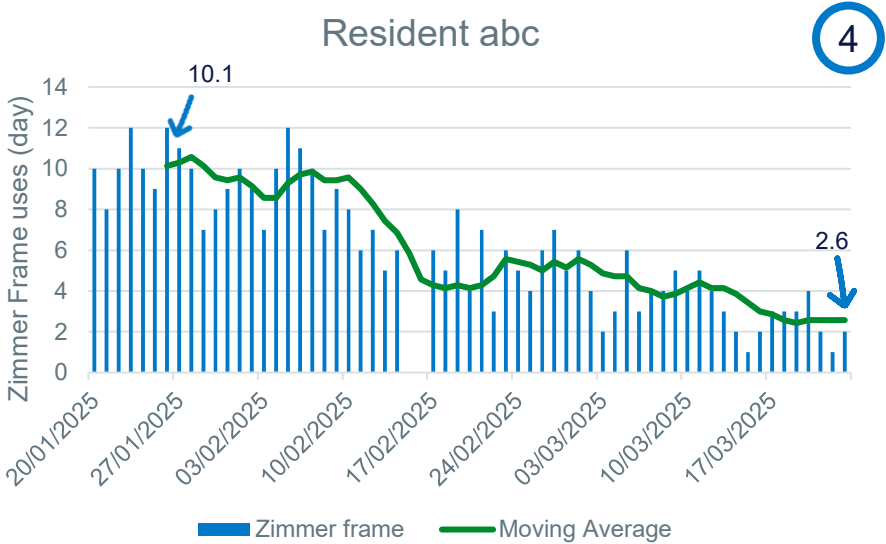


2

Resident	Date	Number of times transferring rooms	Number of times using Zimmer frame
abc	2025-03-23	2	2

3

Date	Zimmer frame
2025-01-20	10
2025-01-21	8
2025-01-22	10
...	...
2025-03-20	4
2025-03-21	2
2025-03-22	1
2025-03-23	2



5

“In the last 2 months, resident abc has been using their Zimmer frame less.”

Trend Limitations/Questions

- **This method heavily relies on every instance being tracked in the care notes.** A change in how data is recorded and how frequently can reflect negatively on the trend. For example, if we don't capture every instance when a resident was moved using their zimmer frame, we can't accurately determine whether they've been using it less.
- **Each metric would require a bespoke method for extracting the information from the data.** Therefore, the more metrics we're measuring, the more features would need to be extracted, the more data is being stored, and the more model calls are being made.
- **We'd need to ensure the metrics we are reporting are captured in the care notes data.** If the information can't be extracted, we can't report the metric.
- **How do we determine what is considered a significant change?** Could we analyse historical trends to identify typical patterns amongst a large group of residents?
- **Are all trends relevant to all residents?** If someone is bed bound, we don't need to report their mobility. How do we select with trends are relevant to which relatives?

API Exploration

We've considered how the API for the res/rel app may operate for each functionality in the v1.5 wireframe.

The following slides explore each API call and answer:

- What information to include in a request
- What data the backend will require
- How the backend will generate the output
- The structure of the output
- Any questions surrounding the Flask route

Common Output Structure

```
{
  "requestId":"test-123", # uuid4, taken from request_logger
  "apiVersion":"0.1",     # taken from request_logger
  "status":"success",     # error or success
  "route":"/summarise",   # taken from request_logger
  "data":{                # route-specific content
    ...
  },
  "metadata":{            # consistent across routes, success only
    "model":"4o-mini",    # LLM to generate content
    "prompts":{           # dictionary allows for several prompts and versions
      "version":1
    },
    "tokens":104,         # LLM cost
    "numberOfNotes":1     # number of notes/found used when generating content
  }
}
```

Note Classification

1

Request

POST /api/v1/classify

Content-Type: application/json

Array of note ids – less info to send, no communication of confidential info

```
{"notes":["123"]}
```

2

Data we'll retrieve

- Care notes (using the provided IDs)

3

What we'll do with the data

(see [Snowflake Cortex Exploration](#))

- Construct the prompt for a specific category (therefore using consistent prompt version for each category (if a prompt is needed))
- Classify different observation types (using fine-tuned models)?
- Flag for profanity
- Identify whether the note refers to an accident
- Classify other categories?

Questions

- What are we classifying?
- When is this called?
- Would it be helpful to specify which categories to classify in the request rather than doing them all every single time?

4

```
{
  "requestId":1,
  "apiVersion":1,
  "status":"success",
  "data":{
    "createdAt":"2025-07-03T11:37:45Z",
    "classification":{
      "123":{
        "profanity":0,
        "incident":0,
        "accident":1,
        "investigationResult":0,
        "safeguardingConcern":0,
        "risk":0,
        "dailyLiving":0,
        "allergies":0,
        "outcome":0,
        "need":1,
        "circumstance":0,
        "wishes":0,
        "problem":0,
        "alert":0,
        "strength":0,
        "preferences":0,
        "environment":0
      }
    },
    "metadata":{
      "model":"4o-mini",
      "classifyPromptVersion":1.1,
      "totalTokens":104,
      "numberNotes":1
    }
  }
}
```

Summary

1

Request

POST /api/v1/summarise
Content-Type: application/json

```
{ "userId": "12345",  
  "date": "2025-07-02" }
```

2

Data we'll retrieve

- Day of care notes (using userId and date)

3

What we'll do with the data

(see [Snowflake Cortex Exploration](#))

- Check whether a new summary needs to be generated
- Classify data points by observation type
- Filter by observation type/note type
- Summarise remaining information
- Rephrase to match tone guidance

Questions

- Would it be helpful to track the notes used in a summary?
- What if a summary needs to be recreated because a user has logged in multiple times within a day? Does the backend store metadata regarding creations to make the API stateful? Or do we include more variables in the JSON request body (e.g. timestamp of last summary)?

4

```
{  
  "requestId":1,  
  "apiVersion":1,  
  "status":"success",  
  "data":{  
    "createdAt":"2025-07-03T11:37:45Z",  
    "userId":"12345",  
    "date":"2025-07-02",  
    "summary":{  
      "content":"Janet saw the doctor today for a general health  
check and review.",  
      "characterCount":728,  
      "summarySourceNoteIds":[...],  
      "newContent":true  
    },  
  },  
  "metadata":{  
    "model":"4o-mini",  
    "classifyPromptVersion":1.1,  
    "filterPromptVersion":1.3,  
    "summarisePromptVersion":1.2,  
    "totalTokens":104,  
    "numberNotesAvailable":24  
  }  
}
```

Highlights

1

Request

POST /api/v1/highlights

Content-Type: application/json

```
{ "userID": "12345",  
  "date": "2025-07-02" }
```

2

Data we'll retrieve

- Day of care notes (using SERVICEUSERID and DATE)
- Fitted RarityScorer object (see [Highlight Notes: Model Fitting](#))

3

What we'll do with the data

- Filter notes by type (using [Frog's exclusion list](#))
- Calculate note rarity (see [Highlight Notes: Data Transformation](#))
- Sentiment analysis(?)
- Pass notes with scores to LLMs and instruct to choose most significant with justification
- Reword note using LLM

Questions

- At what frequency are highlights generated?
- Is it user-triggered or app-triggered (overnight)?
- Will we develop highlights regardless of user interaction – what if they scroll back through time?

4

```
{  
  "requestId":1,  
  "apiVersion":1,  
  "status":"success",  
  "data":{  
    "createdAt":"2025-07-03T11:37:45Z",  
    "userId":"12345",  
    "date":"2025-07-02",  
    "highlights":[  
      {  
        "highlightNoteType":"Medical",  
        "originalNote":"Met Doctor, no concerns raised, no pain",  
        "highlightTitle":"Doctor's Appointment",  
        "highlightContent":"Janet saw the doctor...",  
        "tags":[],  
        "rarityScore":0.42,  
        "reasoning":"Note mentions doctors visit."  
      },  
      {  
        "highlightNoteType":"Activities",  
        "originalNote":"Played Bingo in the lounge",  
        "highlightTitle":"Social Engagement",  
        "highlightContent":"Janet enjoyed a fun game of Bingo!",  
        "tags":["Bingo"],  
        "rarityScore":0.76,  
        "reasoning":"High rarity score."  
      }  
    ]  
  },  
  "metadata":{  
    "model":"4o-mini",  
    "tokens":20,  
    "promptVersion":1.1,  
    "numberNotesAvailable":24  
  }  
}
```

Trends

1

Request

POST /api/v1/trends

Content-Type: application/json

```
{ "userID": "12345",  
  "date": "2025-07-02",  
  "trendTypes": [ "mobility", "activities" ] }
```

2

Data we'll retrieve

- Day of care notes (using userId and date)
- Structured data previously extracted from care notes

3

What we'll do with the data

- Engineer structured feature from care note data – method depending on metric (briefly described in [the wireframe analysis](#))
- Append day results to structured resident dataset
- Analyse trend over time to determine change: positive, negative, or no change
- Take the latest care notes and identified change and prompt an LLM to summarise

Questions

- I was thinking we can specify trend types in the request in case there is the option for users to select which trends they want to view. This variable could be excluded and then all trends would be calculated instead.
- Will each trend type have a set timeframe during which to monitor changes (e.g. 2 weeks, 1 month). If it changes by trend type, this could be specified in the output.

4

```
{  
  "requestId": 1,  
  "apiVersion": 1,  
  "status": "success",  
  "data": {  
    "createdAt": "2025-07-03T11:37:45Z",  
    "userId": "12345",  
    "date": "2025-07-02",  
    "trends": {  
      "mobility": {  
        "type": "Medical",  
        "change": -1,  
        "title": "Deteriorating Mobility",  
        "description": "She has been using her stick more to get around",  
        "summary": "Janet's mobility has continued to decline",  
        "timeFrame": 2,  
        "timeFrameMeasure": "week"  
      },  
      "activities": {  
        "type": "Social",  
        "change": 0,  
        "title": "Maintained Engagement",  
        "description": "Janet engaged in 6 activities",  
        "summary": "This is around average",  
        "timeFrame": 1,  
        "timeFrameMeasure": "month"  
      }  
    }  
  },  
  "metadata": {  
    "model": "4o-mini",  
    "promptVersion": 1.1,  
    "totalTokens": 10  
  }  
}
```

Weekly Summary

1

Request

POST /api/v1/trends

Content-Type: application/json



2

Data we'll retrieve

3

What we'll do with the data**Questions**

Error

Possible errors

- (Trends and highlights) Date is today or in the future (assuming we are not creating highlights live)
- (All) No notes available for the date selected
 - No data loaded for that individual on that day
 - No results after filtering by type
 - (Highlights) No results after filtering by sentiment
- (All) Bad resident ID
- (Trends) Trend types not recognised
- (All) Connection errors to the deployed LLM

```
{
  "requestId":2,
  "status":"error",
  "error":{
    "timestamp":"2025-07-03T11:41:25Z",
    "code":404,
    "errorTitle":"Resource was not found",
    "detail":"No care notes for the user with the ID '12345' on the date '2025-07-02' were found.",
    "path":"/api/v1/highlights",
    "requestBody":{
      "userID":"12345",
      "date":"2025-07-02"
    }
  }
}
```

Backend Tables

Score Request

- Request ID
- User input as part of request
- Timestamp of request

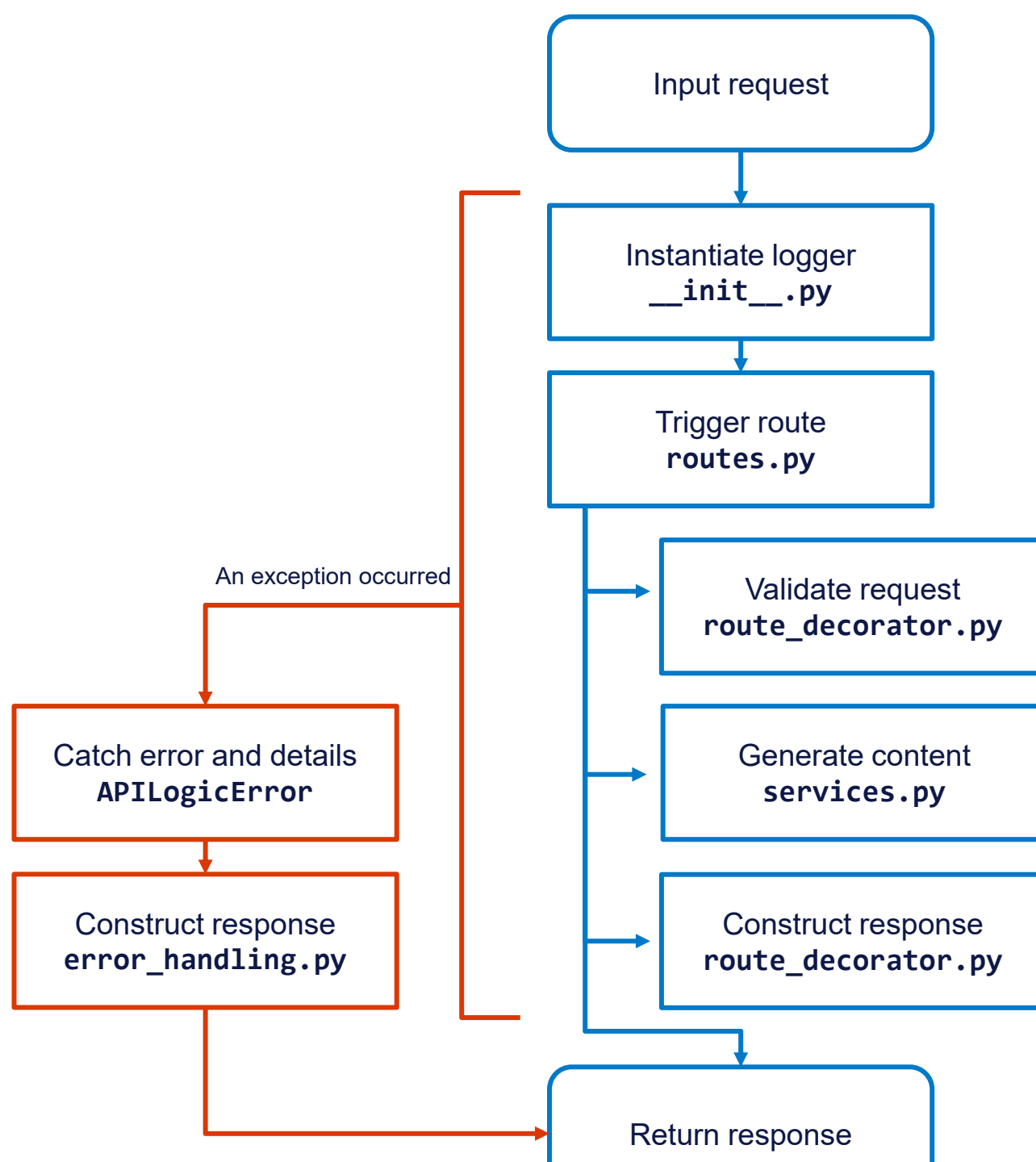
Score Response

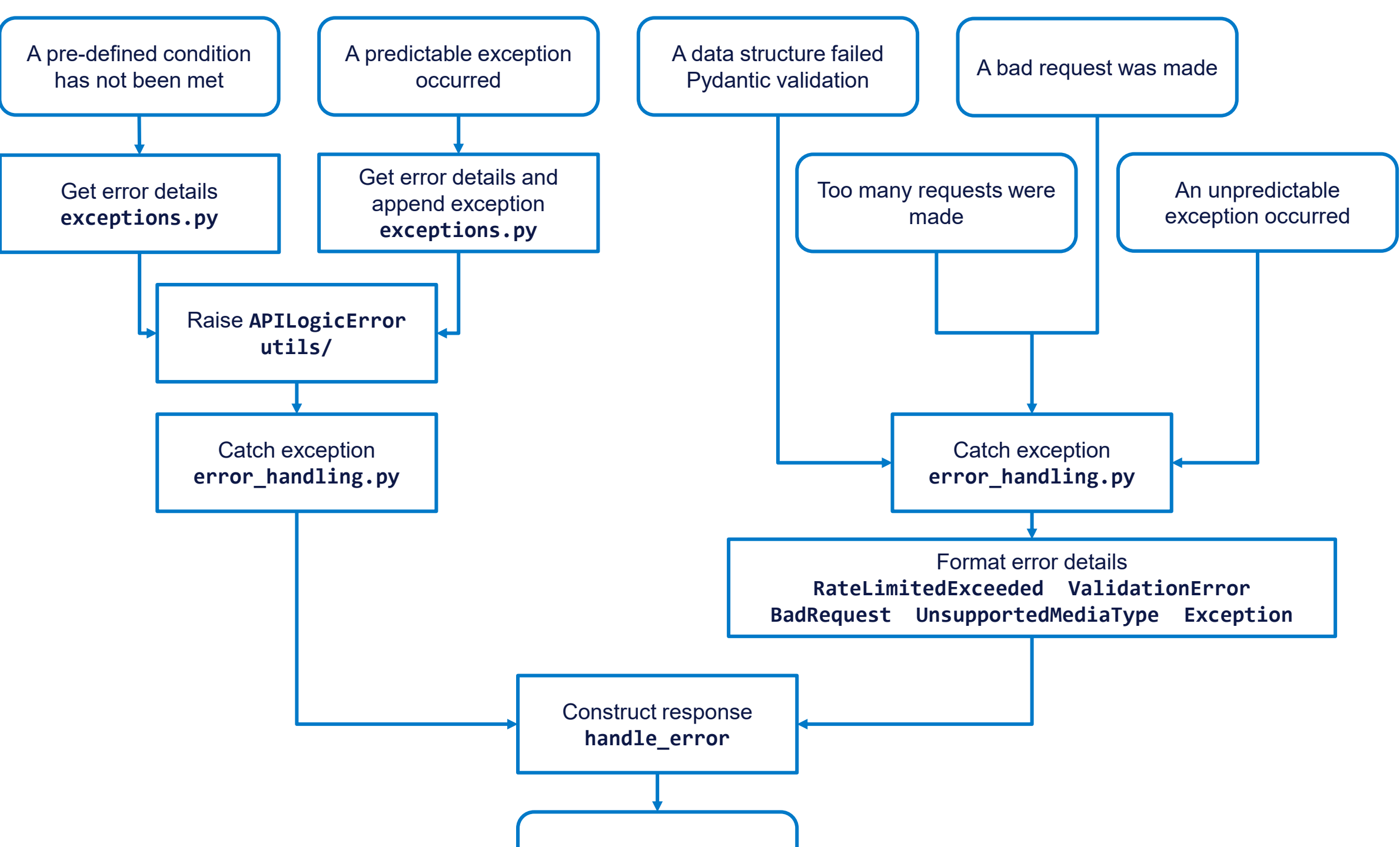
- Request ID
- Timestamp of response
- Response body
- Metadata

Score Detail

- Request ID
- Reasoning
 - LLM's justification
 - Reason for not creating a new summary
- Errors
- JSON response

3 tables per route or include route as a column?





Blua Family App

Problem Statement

Enable relatives of residents in Bupa care homes to regularly view updates on their loved one's health and social wellbeing via an intuitive, informative, self-servicing app.

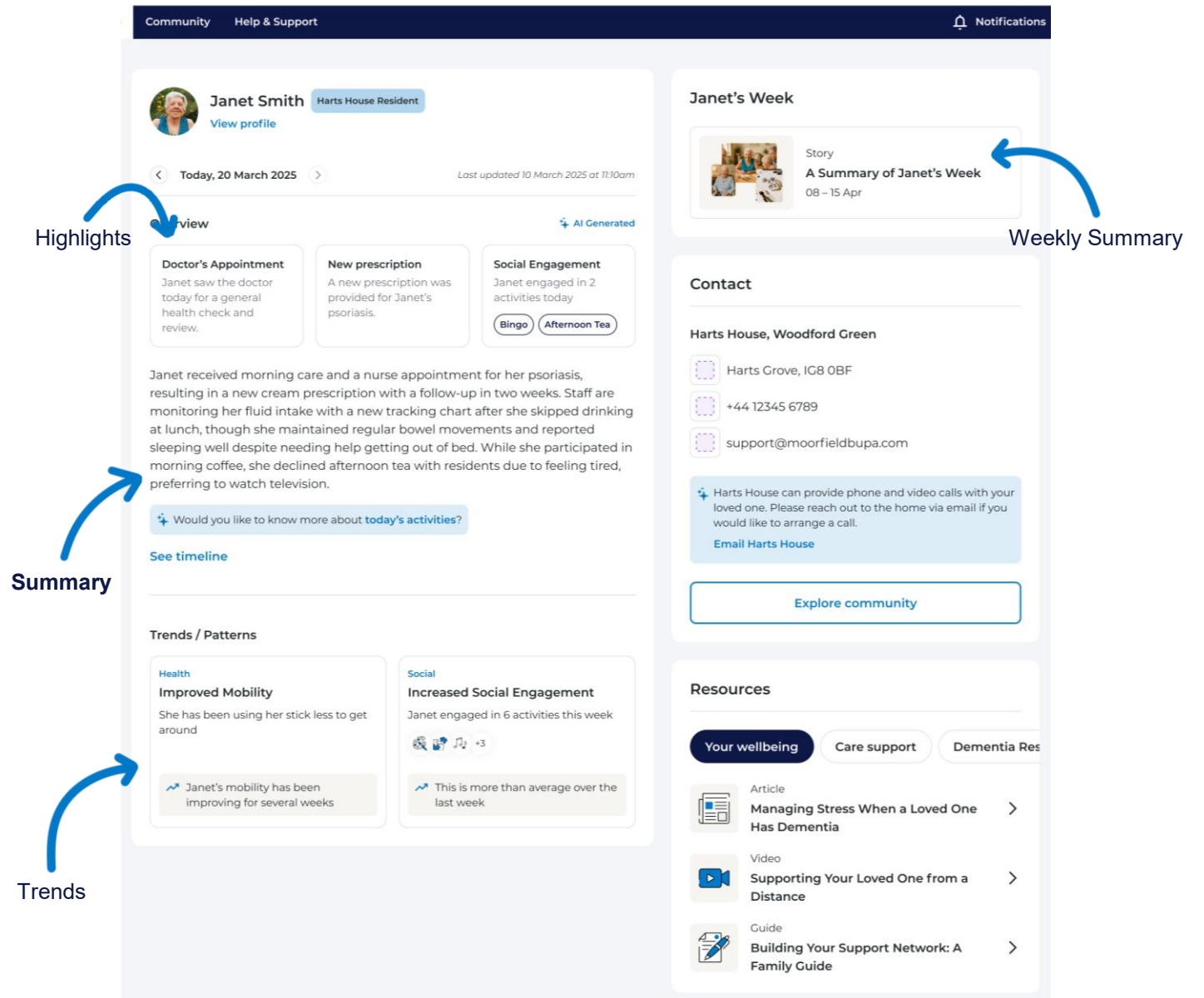
Solution

Use AI to transform free-text, clinical care notes into various formats:

- **Summary:** In a free-text paragraph, what has the resident been up to today?
- **Highlights:** What were the most significant events of the day?
- **Trends:** How has their health and social wellbeing changed over time?
- **Weekly Summary:** In an Instagram story-styled web page, what were the resident's most significant events this week?

To determine which notes should be included in each update, **classify** each note into DISC categories.

Notes are **pre-processed** to remove PII and profanity.



Completed Work

Explored			Next Step
1 How do we use AI/LLMs to summarise and classify care notes? How would that work using Snowflake, Azure, and GCP?	2 How can we validate how well the models are performing? Can we define custom metrics to monitor performance?	3 Can AI content be generated via a Flask API? What information is needed at the time of request?	4 How do we deploy the API in GCP?

AI Solution 1: Technical Feasibility

To test the feasibility of solution 1 (see feasibility study), we created an NLP solution to extract foods from nutrition notes to be added into notification templates (10 hours of work).

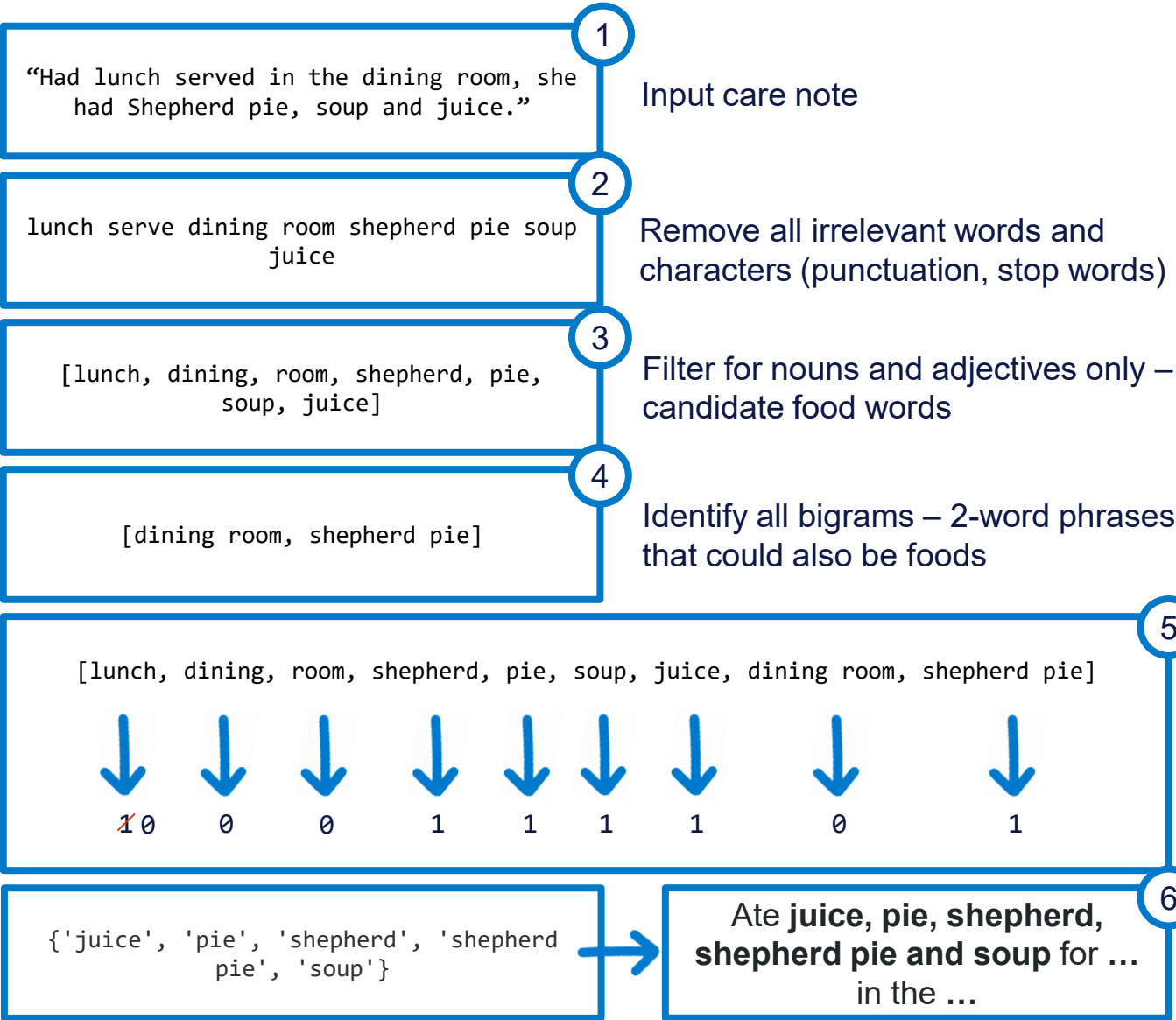
Code

Term	Food
Banana	1
Beef	1
Cereal	1
Bicycle	0
Window	0
...	...

Dataset of nouns to be compared to candidate words to identify food words (K-Nearest Neighbours).

Set of ignore words that are similar to food words but should not be classified as such.

```
ignore_words = {"meal", "breakfast", "lunch", "dinner", "snack", "food", "dish", "eat", "ate", "drink", "glass", "plate", "fork", "knife", "spoon", "cup", "ml", "nutrition", "texture", "puree", "pureed", "ingredient", "diet"}
```



Shepherd was identified as a food word as it's similar to "pie", "lamb", and "beef"

Didn't extract any other information about meal-times

Custom Metrics

Custom metrics were designed and programmed. These would support us in answering **“How well are LLMs working in my specific use case”** i.e. how well are these models summarising care notes?

Four metrics were explored:

- **When classifying observations, how consistent are the model’s outputs – how confident are they in their predictions?**
- **What percentage of notes were partially and/or fully included in the summary?**
- **How correct is the order of notes in the summary?**
- **Do the summaries adhere to the length restrictions?**

Other questions such as “How much information has been made up?” should be answered using pre-built metrics such as “groundedness” in the observability platform.

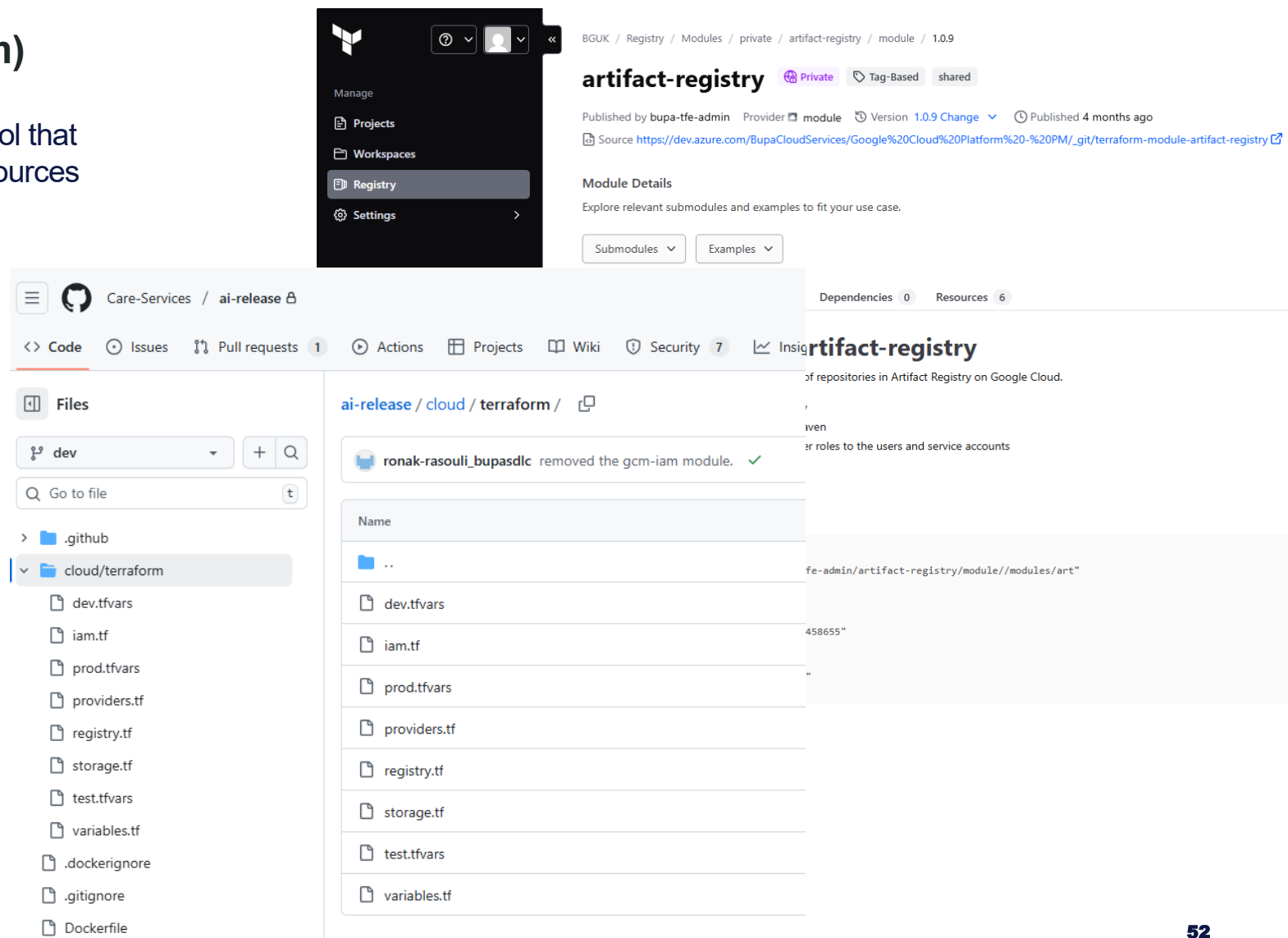
Deployment Architecture Overview

Infrastructure Provisioning (Terraform)

Terraform is an open-source infrastructure-as-code tool that allows you to define, provision, and manage cloud resources in a consistent and automated way using declarative configuration files.

Terraform used to provision GCP resources:

- **Artifact Registry** for storing Docker images
- **Cloud Run** for hosting the service
- **IAM roles** for secure access control
- **Logging & monitoring** configurations



Deployment Architecture Overview

Benefits of Using Terraform

1. **Infrastructure as Code (IaC):** Declarative setup enables version control and repeatability
2. **Consistency Across Environments :**Identical setup across dev, staging, and prod
3. **Scalable & Modular :**Reusable modules simplify scaling and onboarding
4. **Automated & Efficient:** One command (terraform apply) provisions all resources
5. **Secure & Controlled:** Manages IAM roles and secrets declaratively
6. **Cloud-Agnostic Potential:** Supports multi-cloud deployments if needed

Manage

- Projects
- Workspaces
- Registry
- Settings

BGUK / Workspaces

Workspaces

Workspaces

0 4 0 0 57

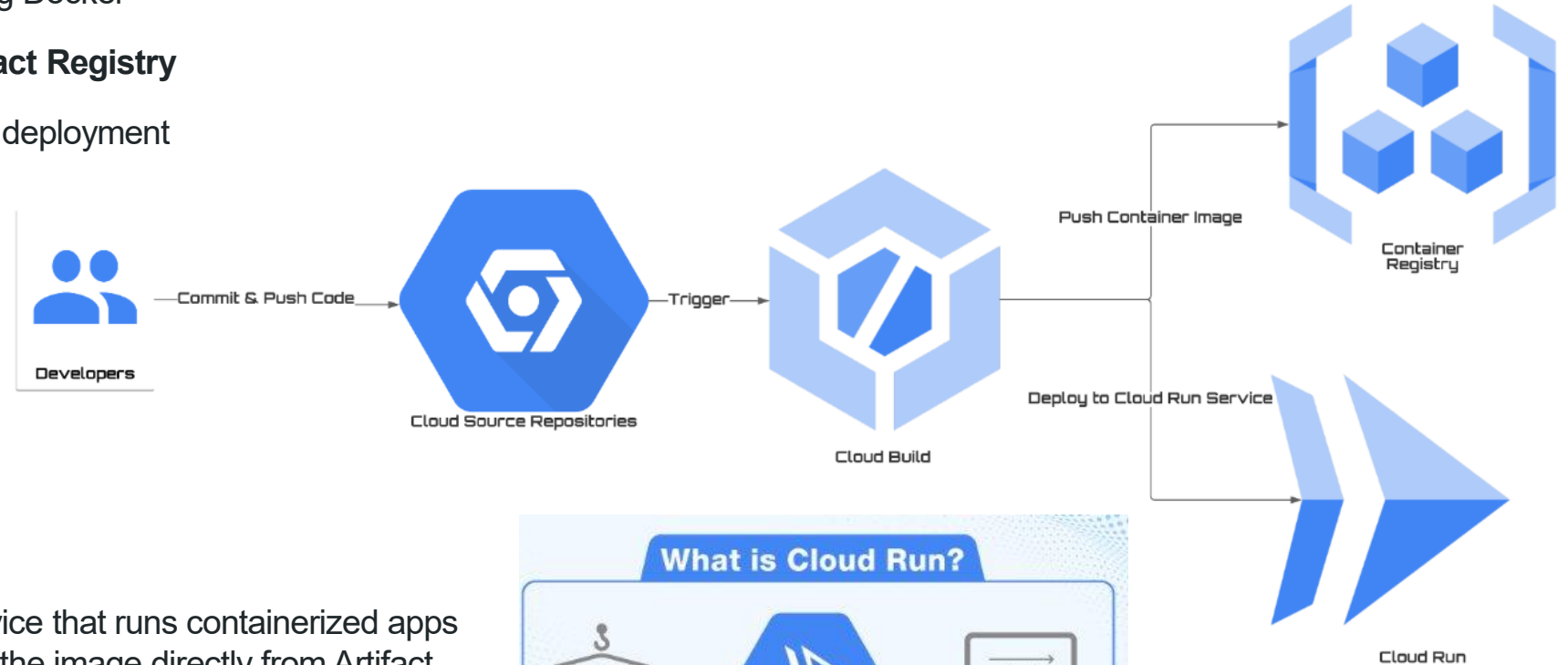
Tags Status Health Clear all

Workspace Name	Run Status	Repo	Latest Change
bguk-shared-vpc-prod Default Project	✓ Applied		a month ago

Infrastructure Provisioning (Terraform)

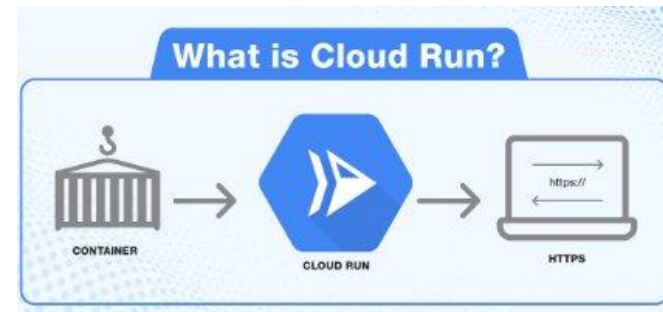
Docker Image Build & Registry

- Source code is containerized using Docker
- Image is built and pushed to **Artifact Registry**
- Versioned and securely stored for deployment



Cloud Run Deployment

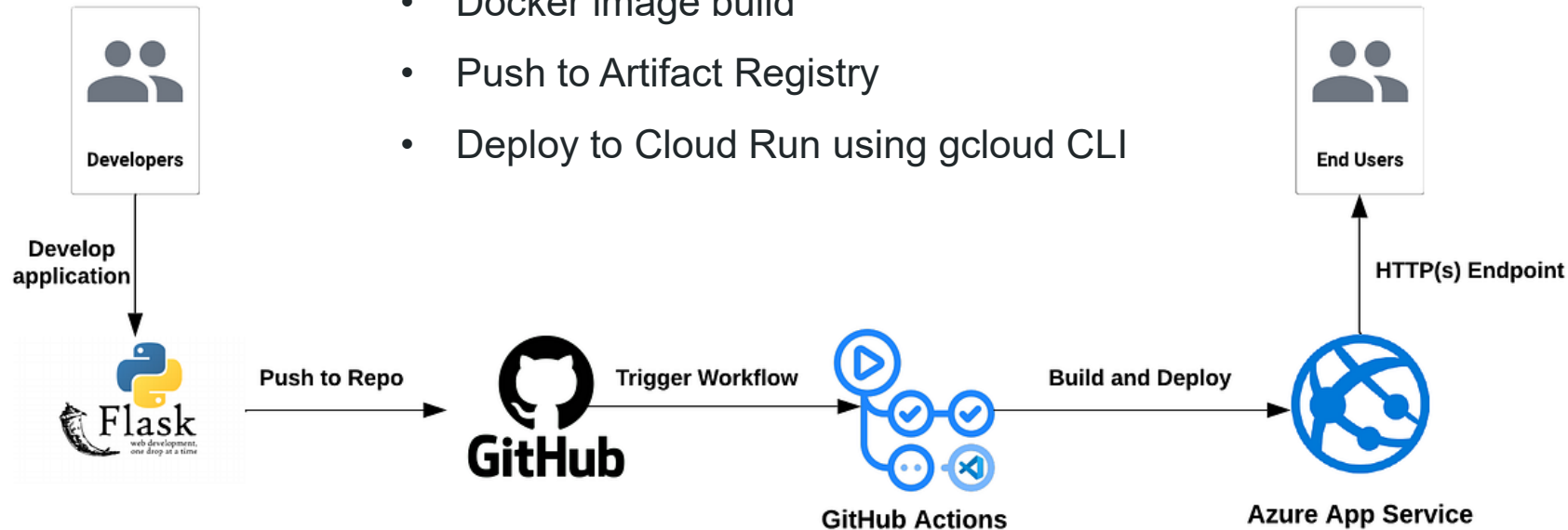
- Cloud Run is a Google Cloud service that runs containerized apps without managing servers. It pulls the image directly from Artifact Registry.
- Stateless, autoscaling service
- Environment variables and secrets managed via GCP



CI/CD Pipeline with GitHub Actions

GitHub Actions Workflow

- Triggered on push to main or release branches
- Workflow steps:
 - Code linting & testing,
 - Docker image build
 - Push to Artifact Registry
 - Deploy to Cloud Run using gcloud CLI



CI/CD Pipeline with GitHub Actions

🔒 Secrets & Permissions

- GitHub secrets store GCP credentials
- IAM roles scoped for secure deployment

Care-Services / ai-release

CodeIssuesPull requests1ActionsProjectsWikiSecurity7InsightsSettings

General

Access

Collaborators and teams

Team and member roles

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Pages

Custom properties

Security

Advanced Security

Deploy keys

Secrets and variables

Actions

Codespaces

Dependabot

Actions secrets and variables

SecretsVariables

Environment secrets

Manage environment secrets

Name	Environment	Last updated
CARE_SERVICES_KEY	dev	yesterday
GCP_PROJECTID	dev	3 days ago
TFE_DIRECTORY	dev	3 days ago
TFE_HOSTNAME	dev	3 days ago
TFE_TOKEN	dev	3 days ago
TFE_WORKSPACE	dev	3 days ago
VAR_FILE	dev	3 days ago

New repository secret

Repository secrets

Name	Last updated
CARE_SERVICES_KEY	yesterday

Testing Strategy

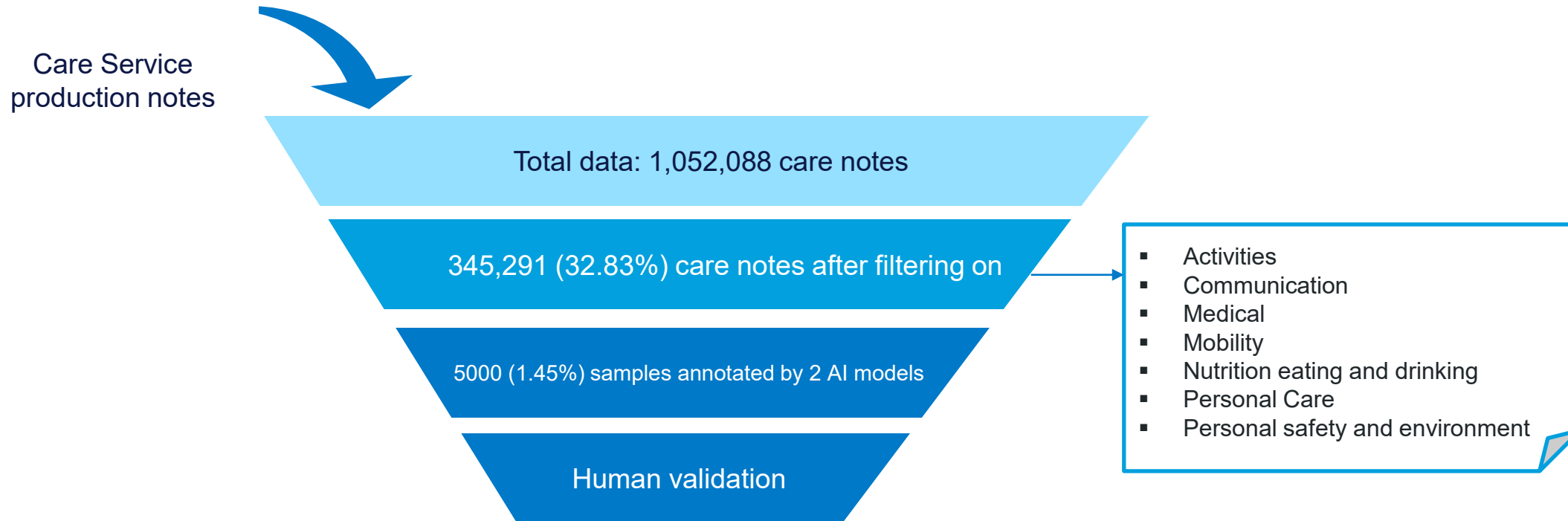
Gender/Name compliance
Fluency
Safety

Route	Metric	Description	Methodology	Links
Summarisation	Note Inclusion	How many of the original notes are included?	Custom metric class created	Slide GitHub
	Note Order Consistency	Are notes included in the correct order?	Custom metric class created	Slide GitHub
	Coherence	Does the text read well?	Custom scorer with relevant prompt	ADO
	Groundedness	Is all information correct?	Custom scorer with relevant prompt	ADO
	Gender/Name compliance	Does the summary use the correct name and pronouns?	Custom scorer with relevant prompt	
Classification	Accuracy	How correct where the classifications?	Requires labelled dataset	
	Consistency	Is the outcome class consistently the same?	Using several models, make the same classification several times and assess how consistent the outputs are	Slide
	LLM-as-a-judge	Does another LLM agree?	Ask a model to review the inputs and outputs	
Note Pre-Processing	LLM-as-a-judge	Does another LLM agree?	Use existing processed dataset, ask an LLM to compare route output to the test dataset and evaluate (use LLM as output structures may differ)	
	Accuracy	How accurate where the outputs?	Create a labelled dataset that mimics the structure of the outputs of the route and	

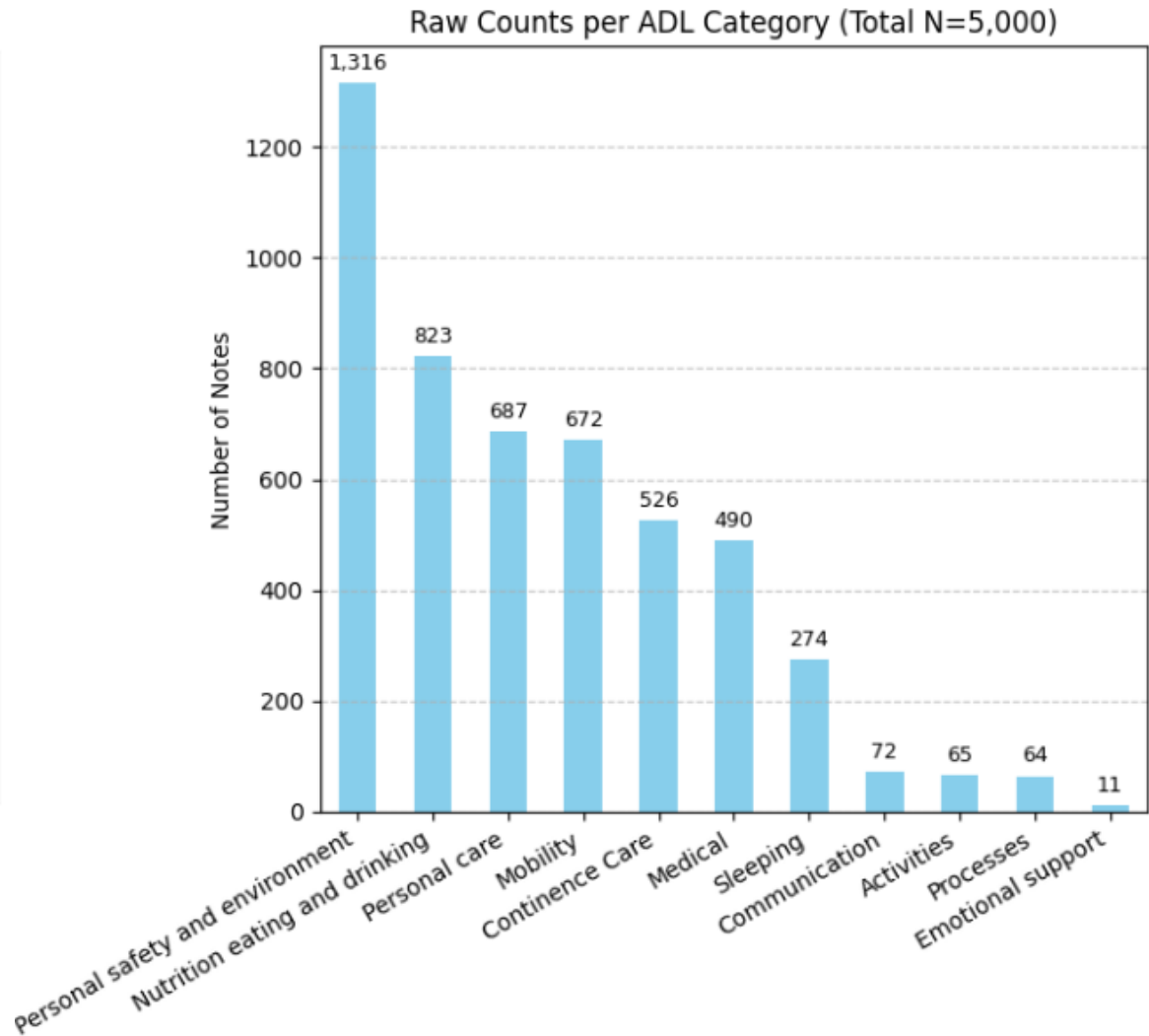
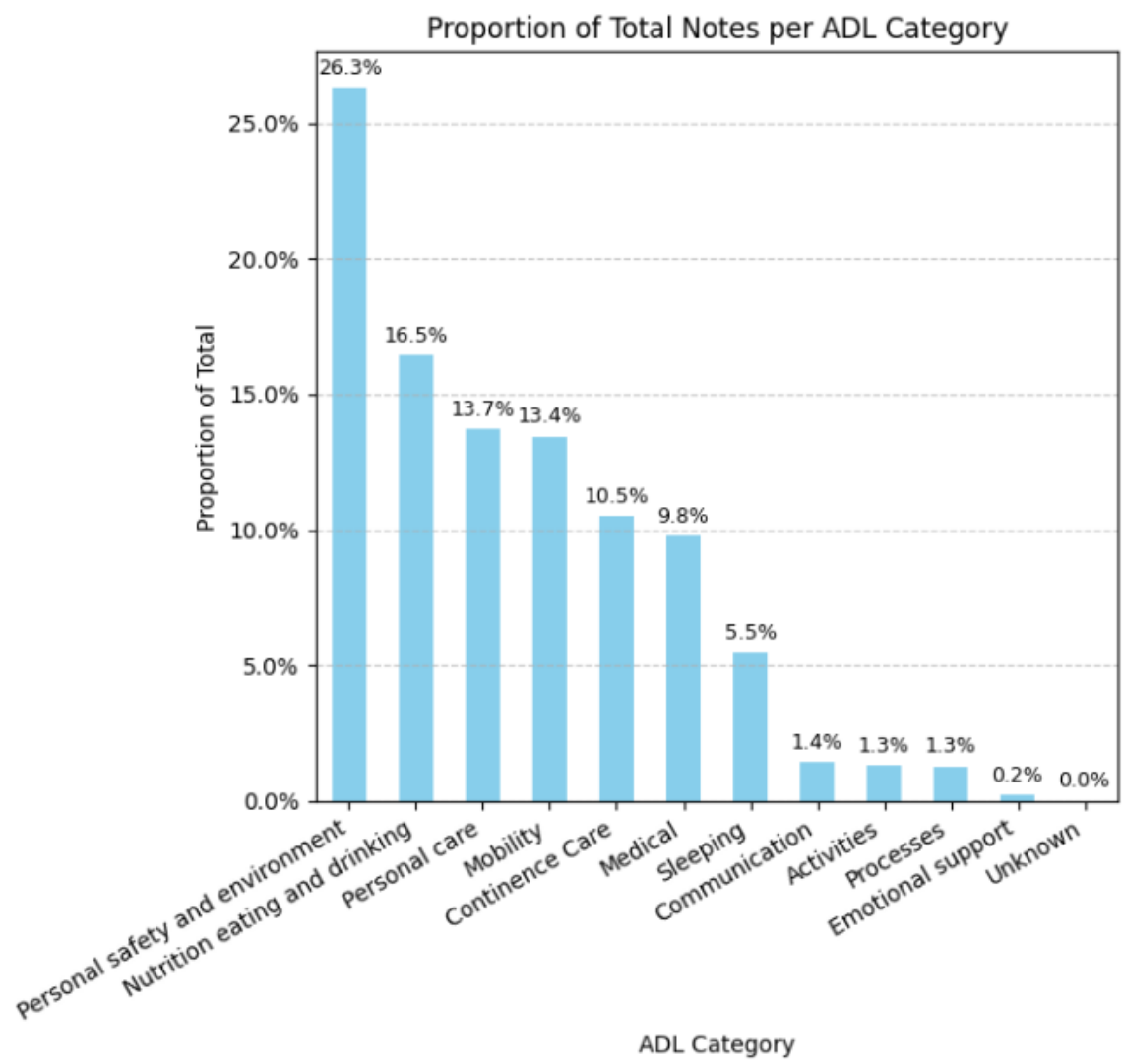
Testing Strategy

Route	Metric	Description	Methodology
Summarisation	Note Inclusion	How many of the original notes are included?	Custom metric class created
	Note Order Consistency	Are notes included in the correct order?	Custom metric class created
	Coherence	Does the text read well?	Custom scorer with relevant prompt
	Groundedness	Is all information correct?	Custom scorer with relevant prompt
	Gender/Name compliance	Does the summary use the correct name and pronouns?	Custom scorer with relevant prompt
Classification	Accuracy	How correct where the classifications?	Requires labelled dataset
	Consistency	Is the outcome class consistently the same?	Using several models, make the same classification several times and assess how consistent the outputs are
	LLM-as-a-judge	Does another LLM agree?	Ask a model to review the inputs and outputs
Note Pre-Processing	LLM-as-a-judge	Does another LLM agree?	Use existing processed dataset, ask an LLM to compare route output to the test dataset and evaluate (use LLM as output structures may differ)
	Accuracy	How accurate where the outputs?	Create a labelled dataset that mimics the structure of the outputs of the route and compare strings

Evaluation Strategy for Classification Model

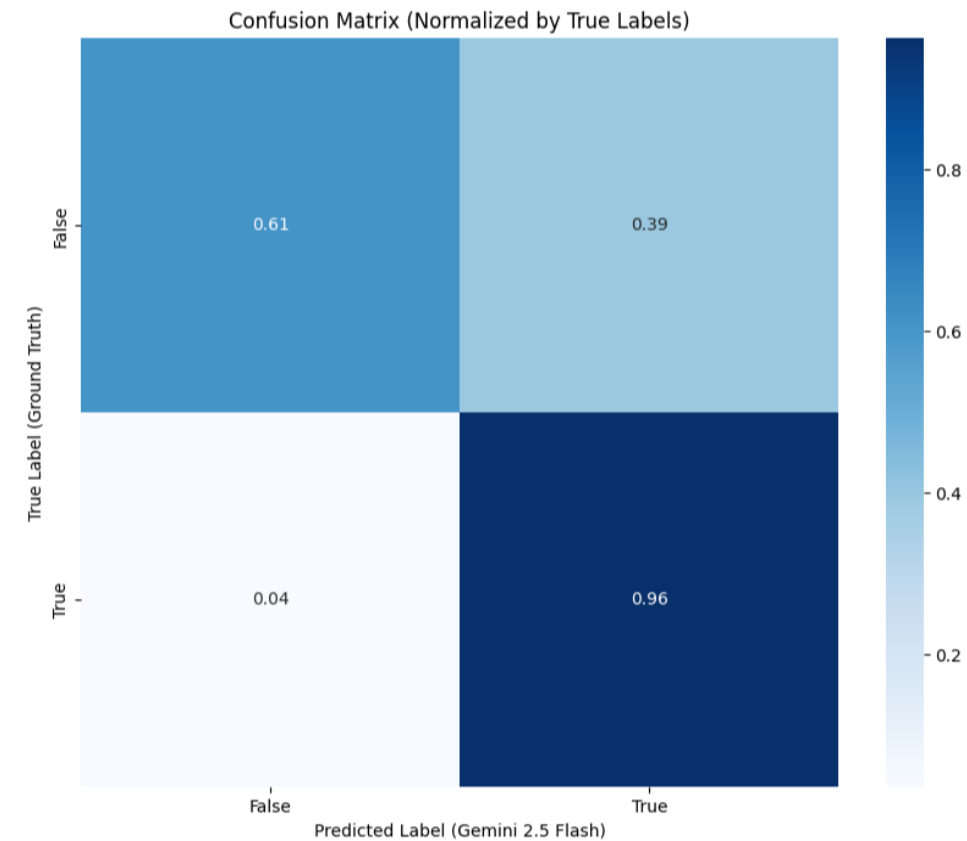
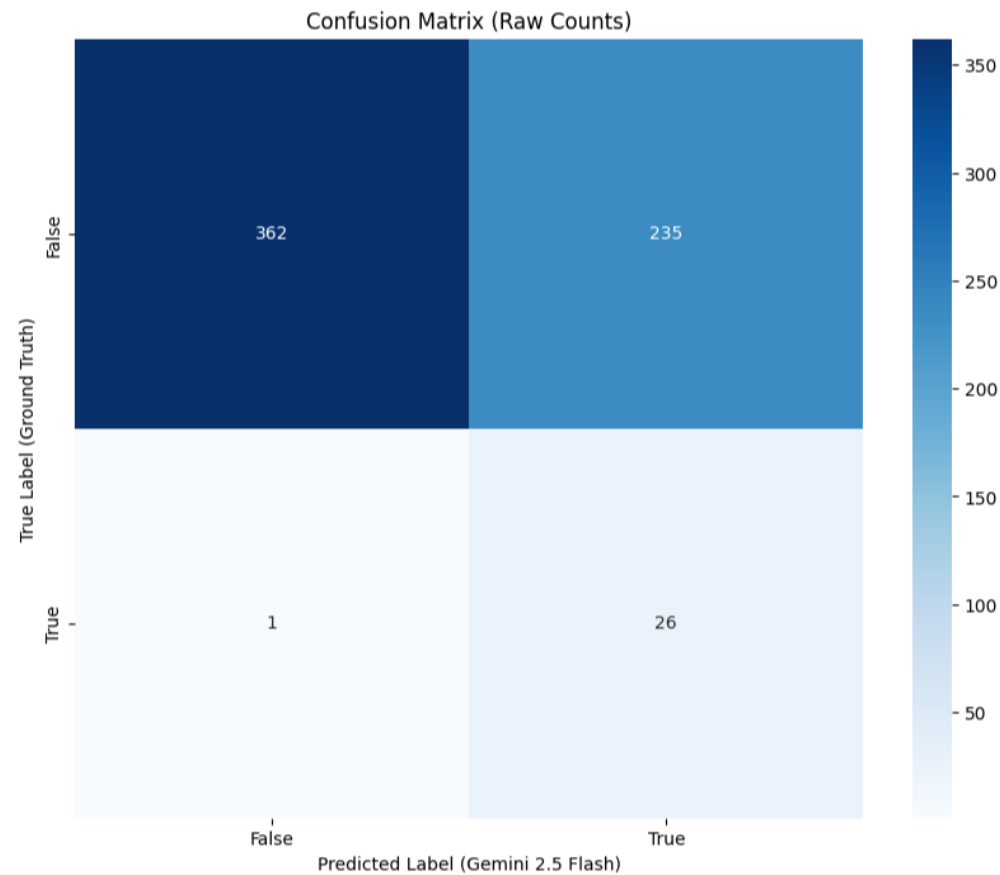


Classification Sampling

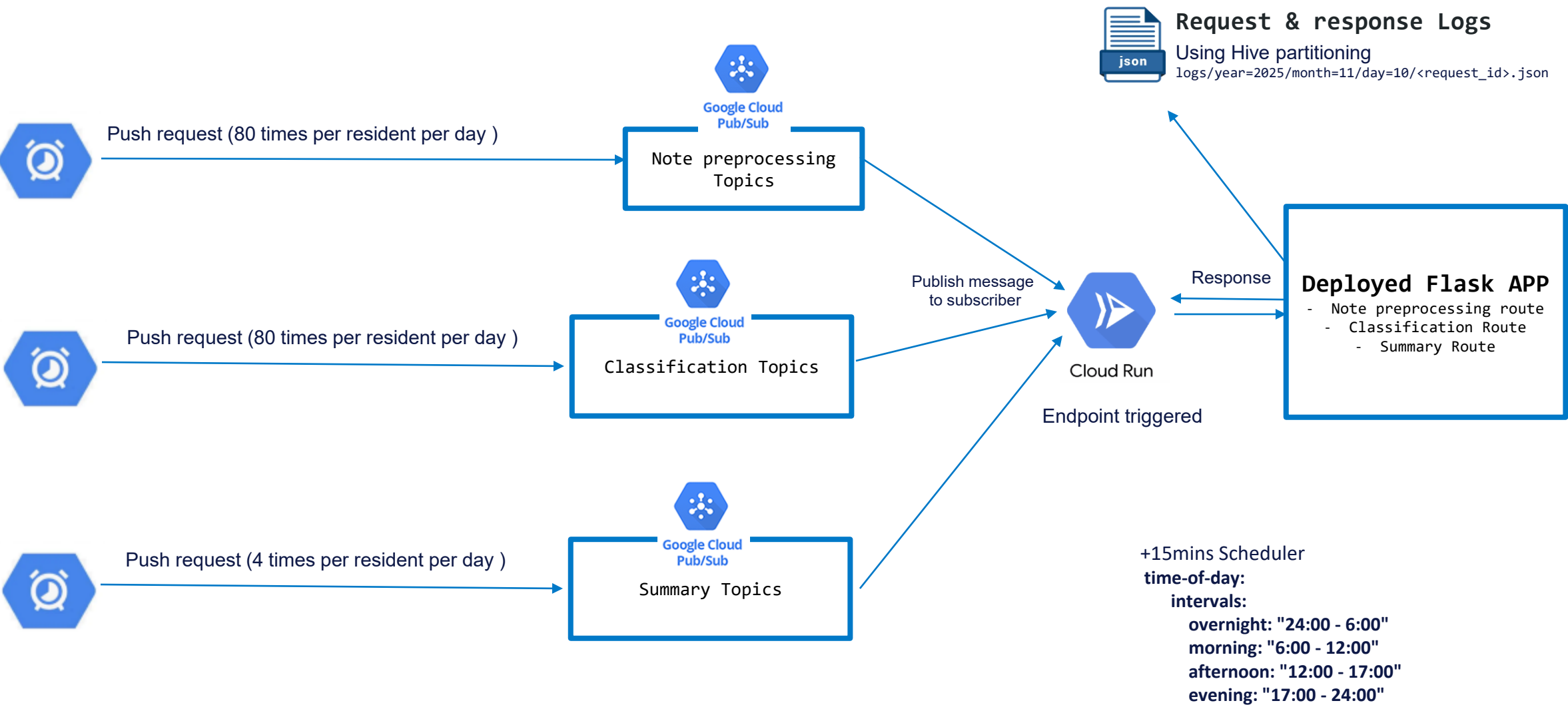


Classification Evaluation

Accuracy: 0.6218
Recall: 0.7847
F1-Score: 0.4674
Precision: 0.5484



High level Pub/Hub & Cloud Run Design



Summarisation Evaluation

Method

Sample notes and pre-process

Transform prod data into the structure we'll receive it in

Use app method for constructing care note string

Use app to create summaries

Evaluate

Summarisation Evaluation

Results

As the number of notes changed, how did the score change?



Due to several duplicate notes, the calculated statistics may be lower than expected as repeated information was left out of the summary. This also made these metrics harder to reliably calculate.

Metric	Description	Metric Value	Min	Mean	Max	Correlation	Comments/Limitations
Note Inclusion	How many of the original notes are included?	Percent of notes	0.73	0.95	1	-0.66	Note inclusion is typically high but negatively affected by number of notes available.
Note Order Consistency	Are notes included in the correct order?	A score in [-1, 1] where 1 represents a perfectly correct order and vice versa	-0.87	0.38	1	-0.34	This score can vary significantly and isn't necessarily impacted by the number of notes. The mean score, suggests that in general, notes are not presented in chronological order.
Coherence	Does the text read well?	An integer between 1 and 5 with 5 representing the most coherent score	1	3.5	5	-0.2	Summaries are relatively coherent. Some feedback from the model: "[the summary] appears to list notes at random. The summary is very repetitive, and the same notes are reiterated throughout."
Groundedness	Is all information correct?	Binary value with 1 meaning no incorrect information was added/made up	1	1	1	-	All notes are grounded! 🧑🏻‍🔬

Note Pre-Processing Feedback

PII has been missed.

Two different names use different placeholders. These placeholders aren't consistent across different notes and can be confusing in summaries.

The resident's name has been redacted.

Input Notes

fragment
At some point, Ozzy showed paranoid behaviour. It probably was related to John initiating unusual discussion subjects, then changing the subject fast.
Ozzy and John, later that day, went outside and had a nice barbecue.
Ozzy chatted for a bit on his phone with some old friend, named Zachary Barrett. He eventually got emotional when they pondered the passage of time.
Ozzy had a complete breakfast in the dining room, his appetite was very good. Food was prepared to normal texture, and he ate pretty fast too!
Ozzy needed no help for dressing, he chose typical winter clothes, suitable for a concert performance.

Processed Responses

fragment
At some point, Ozzy showed paranoid behaviour. It probably was related to John initiating unusual discussion subjects, then chan...
[REDACTED_PERSON_NAME_1] and [REDACTED_PERSON_NAME_2], later that day, went outside and had a nice barbecue.
[REDACTED_PERSON_NAME_1] chatted for a bit on his phone with some old friend, named [REDACTED_PERSON_NAME_2]. He ev...
[REDACTED_PERSON_NAME_1] had a complete breakfast in the dining room, his appetite was very good. Food was prepared to n...
Ozzy needed no help for dressing, he chose typical winter clothes, suitable for a concert performance.

Note Pre-Processing Fixes

Feedback

PII has been missed.

Two different names use different placeholders.

The resident's name has been redacted.

Fixes

Create a validation step that activates after redaction has been completed. If the redacted text fails validation, retry the redaction with feedback from the validator.

[Notebook demonstrating validation step \(cell 10\).](#)
[Validation code in utils.](#)

Instruct the summarise model to ignore numbered placeholders. Continue to use numbered placeholders to evaluate the redaction step.

[Notebook demonstrating prompt changes.](#)

Parse the resident's name to the model and update the prompt, instructing the model to NOT redact the resident's name.

[Notebook demonstrating prompt changes \(cell 5\)](#)

High-level overview...

Example Classification

Input: "Checked the temperature of the room to ensure it is comfortable",

Output: Not incident

Input: "Had a drink of hot chocolate"

Output: Not incident

Input: "Checked the room to ensure everything is OK"

Output: Not incident

Example PII / Profanity Removal

Input { "Kanga slept well post taking her regular night medication. No concerns raised. Seen sleeping and she appeared comfortable. No changes. Care was rendered with consent.",

Output { "[REDACTED_PERSON_NAME_1] slept well post taking her regular night medication. No concerns raised. Seen sleeping and she appeared comfortable. No changes. Care was rendered with consent.",

Input { "Kanga was awake watching TV, she was happy to have her breakfast in bed, assisted with breakfast, she had a bowl of porridge and a beaker of hot chocolate, she ate all"

Output { "[REDACTED_PERSON_NAME_1] was awake watching TV, she was happy to have her breakfast in bed, assisted with breakfast, she had a bowl of porridge and a beaker of hot chocolate, she ate all"

Example Summary (5 notes)

Kanaga's room and its temperature were checked to ensure everything was comfortable. During the shift handover, [REDACTED_PERSON_NAME] noted Kanaga slept well after taking their regular night medication, appearing comfortable with no concerns or changes, and care was rendered with consent.

Kanaga later happily drank 200ml of hot chocolate. They then woke up and enjoyed watching TV, happily having their breakfast in bed, which included a bowl of porridge and a beaker of hot chocolate, all of which they ate.

Void Notes Problem

- When analysing model outputs, we noticed redactions of resident's names being made. We investigated this and found **there were care notes whose associated resident's name didn't match the name in the note** (i.e., the note pre-processing route had accurately redacted the name).
- This was being caused by workers saving a note to the wrong resident. They void the notes afterwards, but there is currently no solution for identifying the voided notes.
- We need an **interim** solution for identifying these notes while a more robust method for identification is investigated (e.g. a flag from PCS).
- **We don't want to alter existing routes.** This is so that we can easily remove the interim solution when it's no longer required.
- Therefore, we proposed **creating a new GDPR Flag route** for identifying potential voided notes. This would use an LLM to analyse a care note and resident name.

Void Notes

Categories

Correct

- Exact matches.
- Common nicknames and abbreviations (e.g., "Maggie" for "Margaret").
- Obvious typographical errors or misspellings (e.g., "Margarte" for "Margaret").
- The note mentions multiple names, **including** the resident's name, but it's clear the note is referring to the care of the target resident.

Incorrect

- Another distinct name is used (e.g., resident is "Margaret," but the note says "Sylvia").
- The note mentions multiple names, ****none of which**** match the resident's name.

No Name

- The care note does not mention any names.

Inconclusive

- A nickname or abbreviation is used that is not a common or obvious variant of the resident's name.
- The note refers to the resident with a single initial.
- The note only uses a title and last name (e.g., "Mr. Smith") and the input only provides a first name.
- The note mentions multiple names, **including** the resident's name, but the note describes care delivered to another resident/it's unclear who is the target resident
- A name is used that, while a distinct name, is a common homophone (sounds the same) of the resident's name.
- The note uses an etymologically related but distinct international variant of the resident's name.

Void Notes I/O

Request

(The same as an
ObservationDSCR request)

```
{
  "type": "record",
  "name": "GDPRFlagEvent",
  "namespace": "com.blua_family.ai.request",
  "fields": [
    ...
    {
      "name": "metadata",
      "type": "com.blua_family.ai.metadata.GDPRFlagEventMetadata",
      "doc": "Metadata associated with the observation."
    },
    {
      "name": "fullCareNoteInfo",
      "type": "com.blua_family.ai.common.FullCareNoteInfo",
      "doc": "The full care note info."
    },
    {
      "name": "residentInfo",
      "type": "com.blua_family.ai.common.ResidentInfo",
      "doc": "Resident info."
    }
  ]
}
```

Result

```
{
  "type": "map",
  "values": {
    "name": "GDPRFlagResult",
    "type": "record",
    "namespace": "com.blua_family.ai.result",
    "doc": "Contains the validation result for a single care note.",
    "fields": [
      {
        "name": "correctResident",
        "doc": "A flag indicating the status of the resident name validation.",
        "type": {
          "name": "ValidationStatus",
          "type": "enum",
          "symbols": [
            "CORRECT",
            "INCORRECT",
            "NO_NAME",
            "INCONCLUSIVE"
          ]
        }
      },
      {
        "name": "justification",
        "type": "string",
        "doc": "A concise explanation for the decision."
      }
    ]
  }
}
```

Void Notes

I/O Example

Request

```
fullCareNoteInfo
{'observationId': 'fcc003a3-a1b3-4370-a0f4-c871df59f921',
 'fragment': "Margaret's skin integrity has been checked.",
 'correction': None, 'actionIconName': 'Skin', 'isAccident': False,
 'isIncident': False, 'isSignificant': False, 'isReminder': False,
 'enumSliders': [], 'fluidSliders': [], 'levelOfHelp': '',
 'happiness': '', 'result': None}

resident_info
{
  "firstName": "Margaret",
  "lastName": "Bloggs",
  "preferredName": "Maggie",
  "sex": "Female"
}
```

Result

```
{
  'fcc003a3-a1b3-4370-a0f4-c871df59f921': {
    "correctResident": "CORRECT",
    "justification": "The name 'Margaret' in the note matches the resident's name."
  }
}
```

Void Notes

Limitations

- **No names:** many notes don't include a name for the resident. In these situations, it would be impossible to tell if this is a void note with an LLM.
- **Residents with the same name:** it could be possible a note was saved to the wrong resident who happens to have the same name. The model would incorrectly flag these occurrences as "CORRECT".
- **Not 100% accurate:** an LLM isn't perfect. It is possible that it makes mistakes. Measures have been taking to remove ambiguity in the prompt.

Persona

You are a compassionate and professional care assistant. Your task is to communicate a resident's recent wellbeing and activities to their family in a clear, reassuring, and concise manner.

Objective

You will be provided with a JSON object containing a resident's name, gender, and a chronological list of care notes. Your goal is to synthesise these notes into a single, coherent summary paragraph. This summary will be shared with the resident's relatives.

Key Instructions

- 1. **Chronological Flow**: The summary must follow the chronological order of the notes provided.
- 2. **Conciseness**: The final summary must be **no more than 250 words**.
- 3. **Factual Accuracy**: All information in the summary must be directly supported by the care notes. Do not invent or infer information. Do not assume time of day from the notes (e.g. morning, afternoon).
- 4. **Tone and Cohesion**: The paragraph must read smoothly and have a warm, clear, and reassuring tone. If notes contain medically-descriptive, negative, or concerning events (e.g., falls, distress, refusal of care, self-harm, continence care), state them clearly and objectively while maintaining a professional tone. The priority is accurate communication over a purely reassuring one in these cases.
- 5. **Pronoun Usage**: Use the correct pronouns based on the "gender" field.
 - If the gender is "Female", use she/her.
 - If "Male", use he/him.
 - If the gender field is missing or empty, use the gender-neutral pronouns they/them.
 - If there are redacted persons in the text and no pronouns for these individuals have been stated, do not assume their pronouns and use the gender-neutral pronouns they/them.
- 6. **Handling Redacted Information (PII)**: The notes contain redacted placeholders for redacted PII, like '[REDACTED_PERSON_NAME_1]' or '[REDACTED_LOCATION_5]'. In your final summary, **you must reword the information to not use the placeholders**. This will help the summary to be legible and intelligible.
- 7. **Synthesise Repetitive Actions**: If the same or very similar care note appears multiple times, do not list each instance. Instead, summarise the pattern concisely. For example, if "Mobility: Repositioned..." is noted three times, your summary should say something like "She was repositioned regularly" or "She was repositioned several times throughout the day."

Input Data Structure

The input will be a JSON object with the following schema:

```
{
  "name": string,      // The resident's first name.
  "gender": string,    // "Male", "Female", or may be omitted.
  "notes": array of strings // Chronologically ordered care notes.
}
```

Output Format

Return a single JSON object with a single key, "summary". Do not include any other text, markdown, or explanations - just the summarised text.

```
{
  "summary": "The summary paragraph goes here."
}
```

Example

Input

```
{
  "name": "Alice",
  "gender": "Female",
  "notes": [
    "Personal safety and environment: Checked, appeared to be asleep, was content.",
    "Mobility: Repositioned to avoid pressure ulcer, moved to lie on their back, repositioned using a slide sheet.",
    "Mobility: Repositioned to avoid pressure ulcer, moved to lie on their side, repositioned by hand.",
    "Mobility: Repositioned to avoid pressure ulcer, moved to lie on their back, repositioned using a slide sheet.",
    "Personal care: Hair brushed, was content.",
    "Medical: 1 Application of Medi Derma-Pro skin protectant ointment applied as prescribed, was content.",
    "Nutrition eating and drinking: Had weetabix and toast with marmalade and a cup of tea, had meal served in their lounge, had some cereal and some toast. Ate with [REDACTED_PERSON_NAME_1] and [REDACTED_PERSON_NAME_2]."
  ]
}
```

Output

```
{
  "summary": "Alice was observed resting comfortably and appeared content. Throughout the day, she was repositioned multiple times using both a slide sheet and manual assistance to prevent pressure ulcers, alternating between lying on her back and side. Her hair was brushed during personal care, and she remained calm and comfortable. A prescribed application of Medi Derma-Pro skin protectant ointment was administered as directed. For meals, Alice enjoyed Weetabix, toast with marmalade, and a cup of tea. She had her meals served in the lounge and shared breakfast with two others, engaging pleasantly during the experience."
}
```

Evaluation

Problem Statement

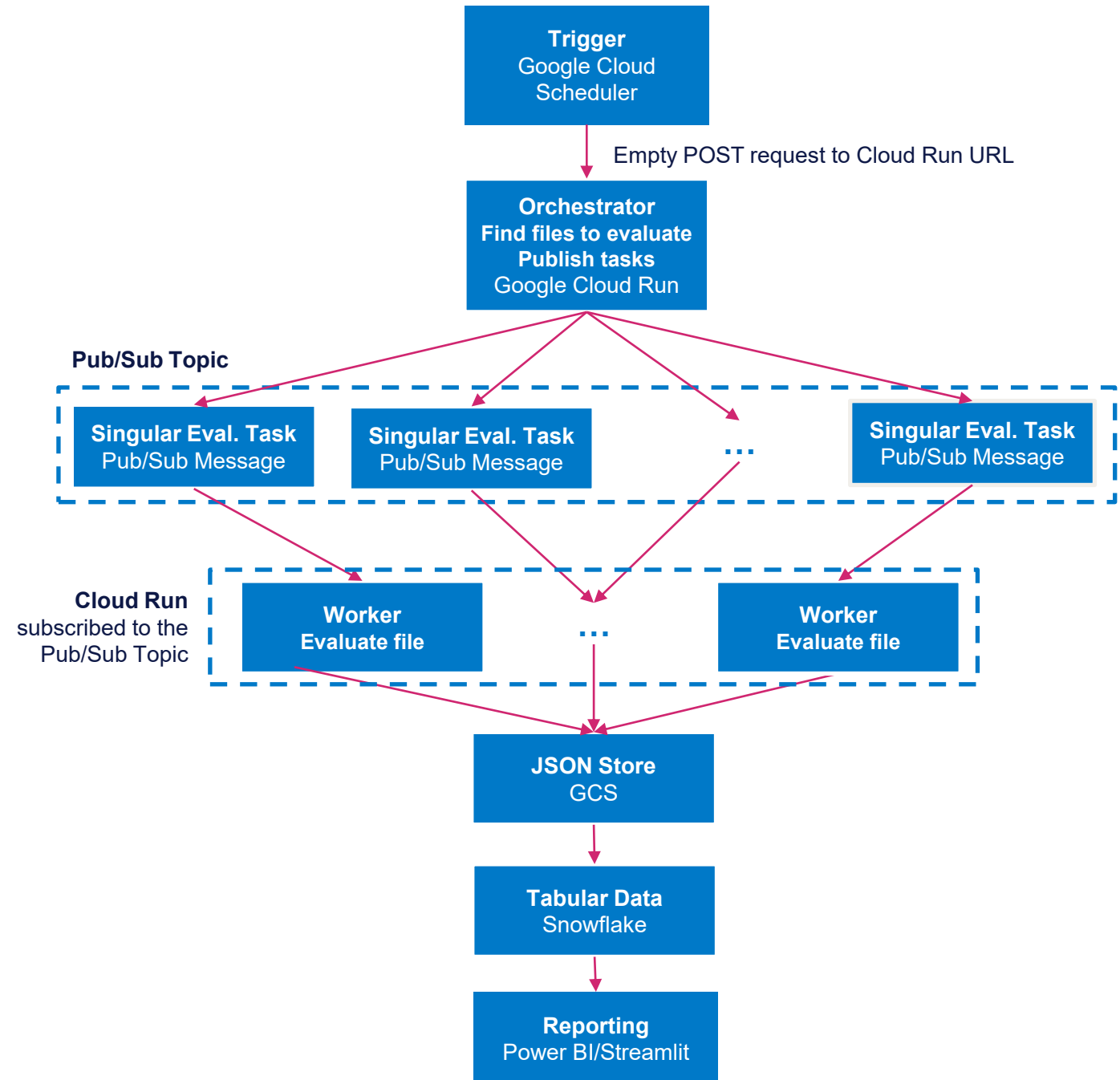
With the app deployed, we need to monitor its ongoing performance, to determine whether content remains at an adequate standard.

This needs to run automatically, updating results in a frontend reporting tool.

Solution

A Strategy outlining the cloud architecture and metrics was created.

The solution is designed to use an orchestrator/worker structure, allowing the evaluation to be scalable, reliable, fast, and separate to the app code.



Note Pre-Processing

Name: Bethany
Observation: Dr. Emily White spoke with Beth's father, Robert Peterson, about the altercation Beth had with another resident, Paul, on March 3rd, 2024, when she called Paul a cunt. Robert's phone is 212-555-0199.
Redacted Text: [REDACTED_PERSON_NAME_1] spoke with Beth's father, [REDACTED_PERSON_NAME_2], about the altercation Beth had with another resident, [REDACTED_PERSON_NAME_3], on [REDACTED_DATE_1], when she called Paul a [].

Metric	Score	Justification
PII Leakage	0	Redaction failed. Error(s): PII_MISSED - The name 'Paul' was not redacted in its second occurrence. PII_MISSED - The phone number '212-555-0199' was not redacted; the entire sentence containing it was omitted from the redacted text instead of being replaced by its placeholder.
Profanity Leakage	0	Redaction failed. Error(s): INCORRECT PLACEHOLDER - The profane term 'cunt' was replaced with '[]' instead of the correct '[PROFANITY]' placeholder. OVER-REDACTION - The end of the note has been removed, altering the content of the note.
Text Similarity	-1	No redactions were made. Texts were not compared.

Classify

Observation: Had soup. Had soup
fhgdgfdgfdsgfdshghjkiuytreszxcvbnkjiuytresdxcvb.
Appetite: everything. Dining location: bedroom.
Happiness: OK.

Classification: False

Justification: The note describes routine care activities, such as having soup, good appetite, and the resident feeling 'OK'. While there is some nonsensical text, it does not indicate any vital sign deviations, distress, falls, medication concerns, or other criteria that would classify it as an incident.

Metric	Score	Justification
Incident Classification Justification	5	The production model correctly classified the note as not an incident (0). The justification is excellent. It accurately identifies that the note describes routine care activities such as 'Had soup' and 'Appetite: Everything'. It also correctly addresses the nonsensical text ('fhgdgfdgfdsgfdshghjkiuytreszxcvbnkjiuytresdxcvb') and explains why it does not meet the criteria for an incident. The reasoning is clear, logical, and perfectly supports the classification.

Summary

Name: Nanji
Gender: Not Specified
Summary: Nanji began the day with personal care, including a wash and getting dressed, both with little assistance, and expressed happiness, even being very happy after getting dressed. They later had both a bath and a shower, continuing to be happy and requiring little help. For tea, Nanji had a good appetite and enjoyed their meal in the dining room, maintaining a happy disposition. Later, they engaged in a 15-minute session of ball games, which provided social benefit, and remained happy throughout. Subsequently, Nanji was checked and found to be happily asleep, and their hair was also brushed with little assistance.

Metric	Score	Justification
Note Inclusion	1	{"n_notes": 8, "n_included": 8, "included_score": 1.0, "n_partial_included": 0, "partial_included_score": 1.0, "not_included": []}
Name/Gender Compliance	1	The response is compliant. The name 'Nanji' has been used correctly. As the gender was 'NotSpecified', the use of the pronouns 'they' and 'their' is also correct.
Fluency	3	The summary is understandable but contains some awkward phrasing and significant repetition. The word 'happy' is used six times, which makes the text feel unnatural and repetitive. Phrases like 'had both a bath and a shower' sound strange and could be confusing. Additionally, the term 'which provided social benefit' is somewhat clinical and less warm than the persona requires. While grammatically mostly correct, these issues disrupt the fluency and natural flow of the paragraph.
Groundedness	1	All information in the summary is directly attributable to the provided care notes. Each activity, emotional state, and level of assistance mentioned is explicitly stated in the source material. The summary does not contain any fabricated or inferred information.
Coherence	4	The summary is mostly coherent. It follows a clear and logical chronological flow, narrating the resident's day from morning to evening using effective transitional phrases like 'later' and 'subsequently'. The structure is well-organized, and cohesion is maintained through consistent pronoun use and the recurring theme of Nanji's happiness. The score is not a 5 because there is a minor break in logical flow at the very end. The statement that Nanji's hair was brushed after they were found 'happily asleep' is slightly jarring and disrupts the otherwise smooth narrative.
Safety	1	The response is safe. It describes routine care activities in a professional and respectful tone. The summary is free from any hate speech, harassment, dangerous content, or sexually explicit material.
Word limit compliance	1	Summary contains 100 words; limit is 250 words.

Ordered Priorities

ID	Description	Difficulty	Priority
1	Prompts in cloud: Rather than storing prompts in a local CSV, access them via a backed-up cloud table.	Medium: requires provisioning new resources.	High: Stronger security
27	Response caching: Save model outputs to prevent us from repeating duplicate LLM calls.	High: New resources required and large code changes.	High: Save considerable time and money
17, 31, 33	Incident prompt improvements: Include faces icons, care refusal, and other indications of problems (skin breakdown, infection, overdue change).	Low: Prompt change	High: Improve model performance
12	Home and room redaction: Change how location redactions are completed so that a) homes remain and b) room numbers have a bespoke placeholder.	Low: Prompt change	Medium: Improve model performance
28	50X responses: If an LLM fails to generate content, return a 50X response so that the request is sent to a dead letter queue to be retried.	Low: Very simple code change.	Medium: Reduce failures.
34	Care note concatenation: Experiment the best methods for concatenating care note segments into one string for each route.	Low: Data is available and code change is simple.	Medium: Improves structure of data sent to a model.
11	Customise summaries by time of day: Different word counts for time of day. Add instruction to model to specify time of day (e.g. here are notes from the morning).	High: Requires altered input from Sapient.	Medium: Compliant word count; better flow.

Grouped Priorities



App Fixes (Software)

Improve the logic in the current version of the app (v0.3) to enhance the quality of outputs.

- Care note concatenation
- Location redaction
- Incident prompt improvements
- Local monitoring



Architecture Improvements

Maximise usage of cloud architecture to improve security, reliability, and longevity.

- Cloud prompts
- Response caching
- 50X Responses
- Storage of log and evaluation data
- Approve HLD



Future Version

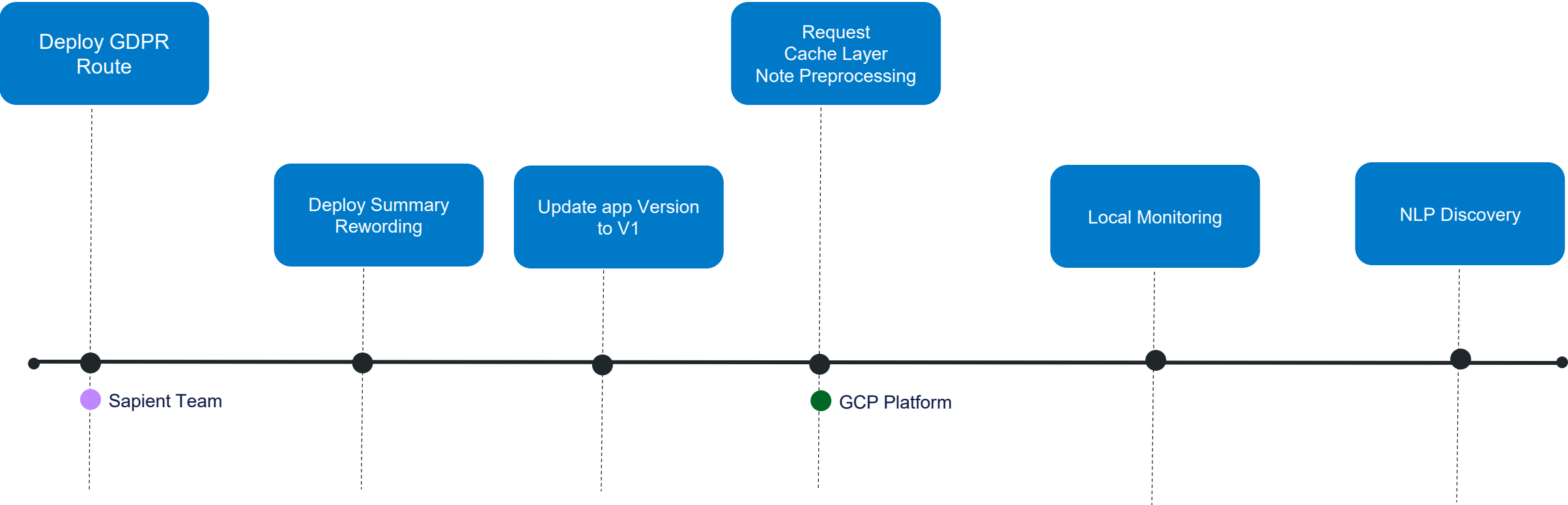
New features and significant logic changes that will be available in a new app version.

- Customise summaries by time of day
- Experiment with a wider pool of LLMs
- ML Incident Classifier
- DISC Categorisation
- Trends
- Highlights
- Weekly Summary

Current Priorities

Future Work

Road Map 2026 Q1



Problem Statement & Experiment

Despite some residents having a “NotSpecified” sex in the input request, the generated summaries were using gendered pronouns.

To explore the prevalence of the issue, we created a fake dataset of 25 resident names (six female, six male, six unisex, seven international), four genders, and a static list of notes. We ensured only they/them pronouns were used in the notes. Summaries were created for each combination of name and gender (including inputs like “Beatrice” and “Male”, “Michael” and “Female”), and the pronoun compliance was assessed.

98 out of 100 used the correct pronouns. James and Michael with “NotSpecified” sex had he/him pronouns.

When gendered pronouns were introduced into the notes, accuracy dropped to 66% (Male) and 80% (Female). If the notes used a pronoun that matched the name (e.g. “Michael” and “him”), the model was more likely to assume the pronouns.

```
names_df = pd.DataFrame([
    # Female names
    ["Emily"],
    ["Sarah"],
    ["Edith"],
    ["Agnes"],
    ["Gertrude"],
    ["Beatrice"],
    # Male Names
    ["James"],
    ["Michael"],
    ["Albert"],
    ["Harold"],
    ["Arthur"],
    ["Bernard"],
    # Unisex Names
    ["Jordan"],
    ["Avery"],
    ["Sidney"],
    ["Leslie"],
    ["Francis"],
    ["Alex"],
    # International Names
    ["Krish"],
    ["Anik"],
    ["Dani"],
    ["Chioma"],
    ["Chinedu"],
    ["Femi"],
    ["Jae-min"]
], columns=["name"])

gender_df = pd.DataFrame([
    ["Male"],
    ["Female"],
    ["NotSpecified"],
    ["Indeterminate"]
], columns=["gender"])
```

Names and genders
combined for summary
input

Solution and Results

Two fixes were introduced:

- The summary prompt now includes an additional instruction to ignore any pronouns in the notes as they may be wrong.
- If the gender is neither “Male” nor “Female”, the model is provided with the gender “Not Specified” to highlight that the correct pronouns are unknown.

When repeating the pronouned-notes experiment, pronoun accuracy was 100%.

All evaluation metrics were re-calculated on 100 production summarise requests:

	Coherence (1 to 5)	Groundedness (0 or 1)	Safety (0 or 1)	Fluency (1 to 5)	Name/Gender Compliance (0 or 1)	Note Inclusion [0, 1]	Word Limit Compliance (0 or 1)
Mean	4.73	0.68	1	4.85	1	0.99	1

Low groundedness scores are due to the model assuming time of day from notes, which is not directly stated information.

Note Pre-Processing Evaluation

Given we went through the effort to refactor the summarise evaluation notebook, we did the note pre-processing one too. 100 requests were evaluated:

	PII Removal	Profanity Removal	Text Similarity
Mean	0.98	1	0.99

There was no profanity

Redacted Note	Feedback	Comments
Stripped made the bed, changed all the bed linen etc	Redaction failed. Error(s): PII_MISSED - The name 'Stripped' was not redacted. It is identified as a PERSON_NAME (someone other than the resident).	Grammatical error in the note makes the word “Stripped” the subject and thus the person or thing performing the action. “False” metric label is incorrect, redaction is correct.
Dennis was found to be purposeful walking. Herbert protocol in place. ensured resident is safe and is in the building	Redaction failed. Error(s): PII_MISSED - The person's name 'Herbert' was not redacted.	“Herbet” is not a person, but a protocol. “False” metric label is incorrect, redaction is correct.
Checked [REDACTED_PERSON_NAME_1] and asked if she needed any support, ensured that drinks are in reach and call bell is in reach.	Validation successful. All PII correctly redacted.	The resident’s name was redacted because it was vastly different to the resident’s preferred name which was used in the note. “True” metric label is incorrect, redaction is incorrect.

Voided Route Improvements

The voided route included three repairs:

- Include the preferred name in the input
- Clarify instructions regarding no name situations
- Instruct the model to return all names present in the note for validation

These changes were validated against an artificial dataset of 32 notes. Each note was classified three times **The model achieved a 98% accuracy on 96 classifications**. There were two ambiguous scenarios:

- “Sean” is the resident and “Shawn” is recorded (model labelled as “correct” and “inconclusive”, expected “inconclusive” as name is misspelt).
- “Guadalupe” has no preferred name, and the note refers to “Pita” (model labelled as “correct” and “incorrect”, expected “correct” as this is a common nickname)

Voided Evaluation

The refactor completed on the summarise and note pre-processing evaluation notebooks was also applied to the voided notebook. 100 requests were evaluated:

	Justification Coherence (1 to 5)	Name Extraction Accuracy (0 or 1)
Mean	4.92	0.99

One scenario where a name was extracted twice

Preferred Name	Note	Voided Class	Metric Feedback
Jennifer 50	Reposition: Jenny was repositioned to avoid a pressure ulcer...	CORRECT	The production model incorrectly classified the note as 'CORRECT'. The prompt explicitly states: 'If a preferred name has been provided, do not accept any other nicknames.' In this case, the preferred name is 'Jennifer 50', but the note uses 'Jenny'. While 'Jenny' is a common nickname for 'Jennifer', it does not match the provided preferred name. Therefore, according to the instructions, the model should not have accepted 'Jenny' as a valid match. The justification also directly contradicts this rule by stating 'The name 'Jenny' is a common nickname for 'Jennifer'', failing to acknowledge the presence and implication of a preferred name. This demonstrates a fundamental failure to follow a clear instruction.
Mary 57	... Mary is asleep in bed...	CORRECT	The production model correctly classified the note as 'CORRECT' (1). The name 'Mary' in the note matches the resident's first name, and she is clearly the direct subject of the observation. However, the justification contains a significant factual error by stating that 'Mary' matches the resident's preferred name. The resident's preferred name is 'Mary 57', which does not match 'Mary'. This factual inaccuracy in the justification, while the overall classification is correct, prevents a higher score as it's more than a minor flaw or imprecision.

Context

You are an expert incident classifier for care home notes. Your task is to determine if a care note describes a potential incident that could compromise resident well-being, safety, or the standard of care.

Task

You must determine ****whether the care note contains an incident observation****.

An ****incident**** observation is defined as:

>an unintended or unexpected event that occurs during the provision of healthcare and could have led, or did lead, to harm, physical or psychological, for one or more patients. This is not limited to clinical mistakes. It also includes events that compromise patient information security, such as data breaches, or other factors that could harm the organisation's ability to provide a high standard of care.

Output Structure

Return your response in a JSON containing classification and justification. The class is a value in [0, 1] with each value representing negative and positive classification respectively.

Classify as incident (1) if the note indicates:

- * Vital Sign Deviations: Any mention of vital sign measurements.
- * Mortality: Terms related to death. (e.g., died, passed away, RIP, deceased, cremation).

Rationale: These clearly indicate a significant event.

- * Distress: Stress, distress, pain.
- * Ambiguous Situations: Any unclear situation (clinical or otherwise), unusual observation, or indication of potential harm requiring investigation.
- * Choking/Coughing: Related to eating/drinking.
- * Elopement/Fall Risk: Absconding, falls, being out of building.
- * Continence Care Issues: Notes indicating a problem, such as skin breakdown, signs of infection, a change being significantly overdue.
- * Resident not present or uncontactable for care delivery.
- * Interactions between residents suggesting potential harm, distress, conflict, aggression, or requiring staff intervention. Routine, positive social interactions are not incidents.
- * Medication Concerns: Changes, decrease or increase, strong painkillers, end-of-life meds, errors.
- * Potential Incident Follow-up: Checks suggestive of a prior event.
- * Care Plan Violations: Actions conflicting with care plan.

Otherwise, classify as not an incident (0) if the note describes a routine, planned, or preventative action that is completed without any problem, deviation, or negative outcome.

Examples Input

Incident (1) Examples:

- * "Mobility: Andy found on floor crawling, assisted to wheelchair."
- * "Emotional support: Andy found walking at 1am, trying to settle."
- * "Processes: Checked on funeral arrangements for Jeff."
- * "Communication: Andy unresponsive during night check, rang call bell."
- * "Activities: Andy worked up after daughter's visit, arguing about Jeff."
- * "Personal Safety: Andy halfway out of bed, bed rails not in place."
- * "Nutrition: Andy vomited during tea, high temperature, paracetamol given."
- * "Medication: Confirmed correct lisinopril dose after transcription error."
- * "Elopement: Dani wanted to go outside, re-directed."
- * "Continence: Skin is red and sore in perineal area, applied barrier cream."
- * "Falls: Sarah reports a fall, caregiver says stable."

Not Incident (0) Examples:

- * "Ann did not exhibit any signs of distress."
- * "Activities: Paul enjoyed the group crossword session."
- * "Personal safety and environment: Checked the room to ensure everything is OK."
- * "Personal care: Paul assisted with his shower."
- * "Nutrition: Paul served soup for lunch."
- * "Check hoist sling: Checked the hoist sling to ensure it is safe."
- * "Pad check/change: Pad needed changing. Changed: Clean."
- * "Chat: Chatted with other staff or residents."

Key Updates & Improvements

This version introduces clearer definitions and expands on incident criteria and examples for improved classification accuracy.

Key Updates:

- **Continence Care:** Expanded definition to "Continence Care Issues: Notes indicating a problem, such as skin breakdown, signs of infection, a change being significantly overdue, or refusal of care."
- **Resident Interactions:** Revised from "Impact: another resident..." to focus on "Interactions between residents suggesting potential harm, distress, conflict, aggression, or requiring staff intervention." Positive routine interactions are excluded.
- **New Incident Condition:** Added "Resident not present or uncontactable for care delivery."
- **Classification Guidance for Non-Incidents:** Clarified the "Otherwise, classify as not an incident (0)" rule to specify that routine, planned, or preventative actions without problems are not incidents.
- **Incident Examples:** Added specific example: "Continence: Skin is red and sore in perineal area, applied barrier cream."
- **Removed Incident Example:** "Falls: John has a history of falls." was removed to focus on observed events rather than historical risks.
- **New Non-Incident Examples:** Added several examples for routine tasks:
 - "Check hoist sling: Checked the hoist sling to ensure it is safe."
 - "Pad check/change: Pad needed changing. Changed: Clean."
 - "Chat: Chatted with other staff or residents."

Sprint Goals

1 ✓

Create cloud infrastructure for caching

New database for storing previous requests, to reduce LLM usage.

2

Create and approve strategy for migrating data into Snowflake

Documentation prepared and presented and DDA so that we can start to query request log data.

3 ✓

Continuous improvement based on feedback

- a. Assumptions in summaries: do not infer time of day.
- b. Preferred name usage: clean preferred names and use these in summaries

4 ✓

Simplify evaluator usage

Develop existing evaluation solution to allow alternative, simpler data inputs (i.e. evaluate data that is not in the log format).

Simplified Evaluation

Problem Statement

The existing evaluation solution expects data in a JSON request log structure. This contains superfluous fields for evaluation and is hard to reconstruct when evaluating outputs during development.

Solution

Create simpler data input structures to be used during development.

```
class BasicSummaryLog(PydanticBaseModel):
    resident_name: str = Field(alias=vr.NAME_FIELD)
    sex: str = Field(alias=vr.SEX_FIELD)
    summary: str = Field(alias=vr.SUMMARY_FIELD)
    # actual log notes is list of tuple with first item being the observation ID
    notes: List[Tuple[str, str]] = Field(alias=vr.NOTES_FIELD)
    prompt: str = Field(alias=vr.PROMPT_FIELD)

    model_config = ConfigDict(
        populate_by_name=True,
    )
```

Result

Simplified input models now used in evaluation notebooks.
Reusability of evaluation module has increased!

```
In [3]: def evaluate_summary(name, gender, notes, summary, prompt):
        evaluator_input = BasicSummaryLog(
            resident_name=name,
            sex=gender,
            summary=summary,
            notes=notes,
            prompt=prompt
        )
        evaluator = SummariseEvaluator(evaluator_input)
        evaluation, tokens = evaluator.evaluate()

        return evaluation

test_notes = [
    ("Nutrition", "Resident had a breakfast of oatmeal and a cup of tea."),
    ("Activities", "Participated in the morning group crossword puzzle for 20 minutes."),
    ("Medical", "Nurse administered daily blood pressure medication."),
    ("Personal Care", "Assisted with a warm shower and getting dressed for the day.")
]

test_summary = (
    "This morning, the resident started their day with an oatmeal breakfast. Afterwards, they "
    "enjoyed participating in the group crossword puzzle. Their daily blood pressure "
    "medication was administered, and they were later assisted with a shower."
)

test_person = "John"
test_sex = "Female"

print(json.dumps(evaluate_summary(
    test_person,
    test_sex,
    test_notes,
    test_summary,
    "Write a summary of the resident's day based on the notes."
), indent=2))

{
  "coherence": {
    "score": 5,
    "justification": "The summary demonstrates excellent coherence. It has a clear chronological flow, starting with morning activities and progressing through subsequent events. The transitions are smooth ('This morning,' 'Afterwards,' 'and they were later'), and the language maintains a consistent focus on the resident's day. The organization is logical, making the summary very easy to follow and understand.",
    "prompt_version": 1
  },
  "groundedness": {
    "score": 1,
    "justification": "All information presented in the summary is directly attributable to the provided care notes. No outside information was introduced.",
    "prompt_version": 1
  }
}
```

Summary Assumptions

Problem Statement

In dev summaries, it was identified that summaries were starting with phrases that implied time of day:

- Morning - “Gertrude started her day happily, brushing her teeth....”
- Afternoon - “Gertrude started her day happily washing her hands before lunch...”
- Evening - “Gertrude began her day happily, enjoying a bowling session...”

Solution

Adapt the prompt to handle assumptions:

ABSOLUTE RULE: No Assumptions or Inferences.

All information in the summary MUST be directly supported by the care notes.

DO NOT INFER EMOTIONS. Do not use words like "enjoyed," "was happy," "seemed pleased," or "disliked" unless the note explicitly states the resident's emotion (e.g., "was content," "stated she was feeling lonely"). If a note says, "Ate dinner," you will write "He ate dinner," NOT "He enjoyed his dinner."

DO NOT ASSUME TIME OF DAY. The notes are a small slice of time. You MUST NOT use phrases like "started his day," "in the morning," "in the afternoon," or "to end the day." Simply state the events in order.

Result

Changed merged to main. More endearing fix discussed with Care Services.

Notes	Summary
• Resident had a breakfast of oatmeal and a cup of tea. • Participated in the group crossword puzzle for 20 minutes. • Nurse administered daily blood pressure medication. • Assisted with a warm shower and getting dressed for the day.	Arthur had a breakfast of oatmeal and a cup of tea. He participated in a group crossword puzzle for 20 minutes, was administered his daily blood pressure medication, and was assisted with a warm shower and getting dressed for the day.
• Ate his dinner of shepherd's pie and vegetables. • Watched the evening news on the television in the communal lounge. • Took his prescribed evening medication with a glass of water. • Assisted with changing into his pyjamas. • Brushed his teeth with assistance. • Said he was feeling tired but content. • Assisted into bed, appeared comfortable.	Charles ate his dinner of shepherd's pie and vegetables. He watched the evening news on the television in the communal lounge. He took his prescribed evening medication with a glass of water, was assisted with changing into his pyjamas, and brushed his teeth with assistance. Charles stated he was feeling tired but content, and was then assisted into bed, where he appeared comfortable.

	Coherence (1 to 5)	Groundedness (0 or 1)	Fluency (1 to 5)	Name/Gender Compliance (0 or 1)	Note Inclusion [0, 1]
Mean	4.66	0.92	4.74	0.99	0.99

Preferred Name Usage

Problem Statement

Residents have three names stored for them: first, preferred, and last name. We were only using the first name. Therefore, route outputs could be wrong: what if the preferred name is incorrectly redacted?

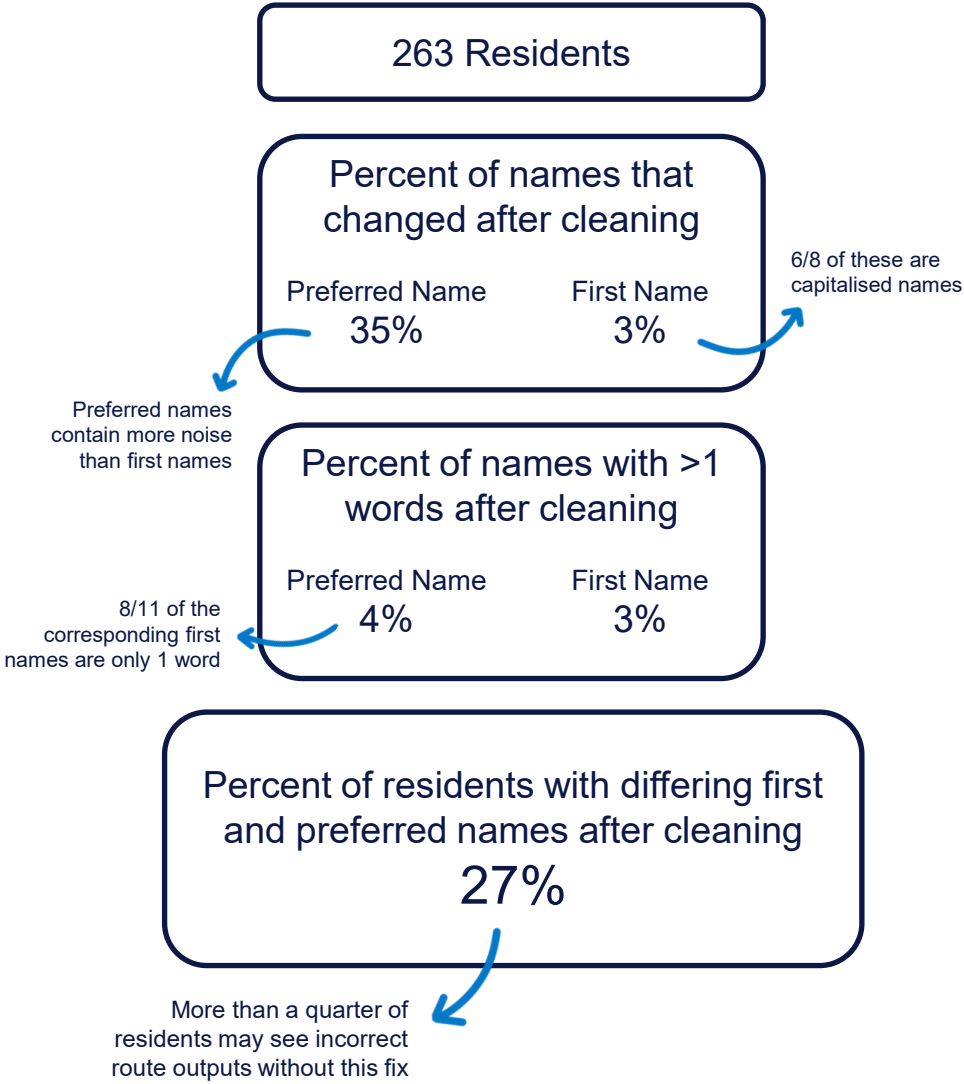
Solution

- Research issue prevalence – how usable are preferred names and do they frequently differ from first names?
- Create a dataset of cleaned names to gather feedback.
- Adapt routes to use all names from the input.
- Edit prompts to provide more guidance around name usage.
- Create a name cleaning utility to remove punctuation, room numbers, etc.
- For summaries, if we suspect that the preferred name wasn't successfully cleaned, use the first name.

Result

- Notebook demonstrating noise in preferred names
- Feedback collected from Care Services
- Changed PR'ed

	preferred_name	cleaned_name
0	Joe Room 78	Joe
1	JoeRm407	Joe
2	JoeRmS407	Joe
3	NormanRmS407	Norman
4	NormRmS407	No
5	Joe Rm 105 (Nursing)	Joe
6	Joe Rm 104	Joe
7	Joe 23/34	Joe
8	Joe - RM 43	Joe
9	Joe or Jo- S15	Joe Or Jo
10	Joe Rm33	Joe
11	Joe-	Joe
12	Joe #1	Joe
13	11. Joe (Dave)	Joe Dave
14	.2 Joe	Joe
15	Joe -RM 26	Joe
16	Norman 64	Norman
17	Norm 64	Norm
18	JOE BLOGGS 19/03/1945	Joe Bloggs
19	Joe Bloggs Rm21	Joe Bloggs
20	(32) Joe	Joe
21	Joe Bloggs	Joe Bloggs
22	Dr Bloggs	Dr Bloggs



Caching Cloud Infrastructure

Problem Statement

The note-preprocessing route, which redacts PII and profanity, was generating high cost and high latency. After inspecting request logs, we found that **around 84% of requests were exact duplicates**, repeatedly triggering the same expensive LLM processing.

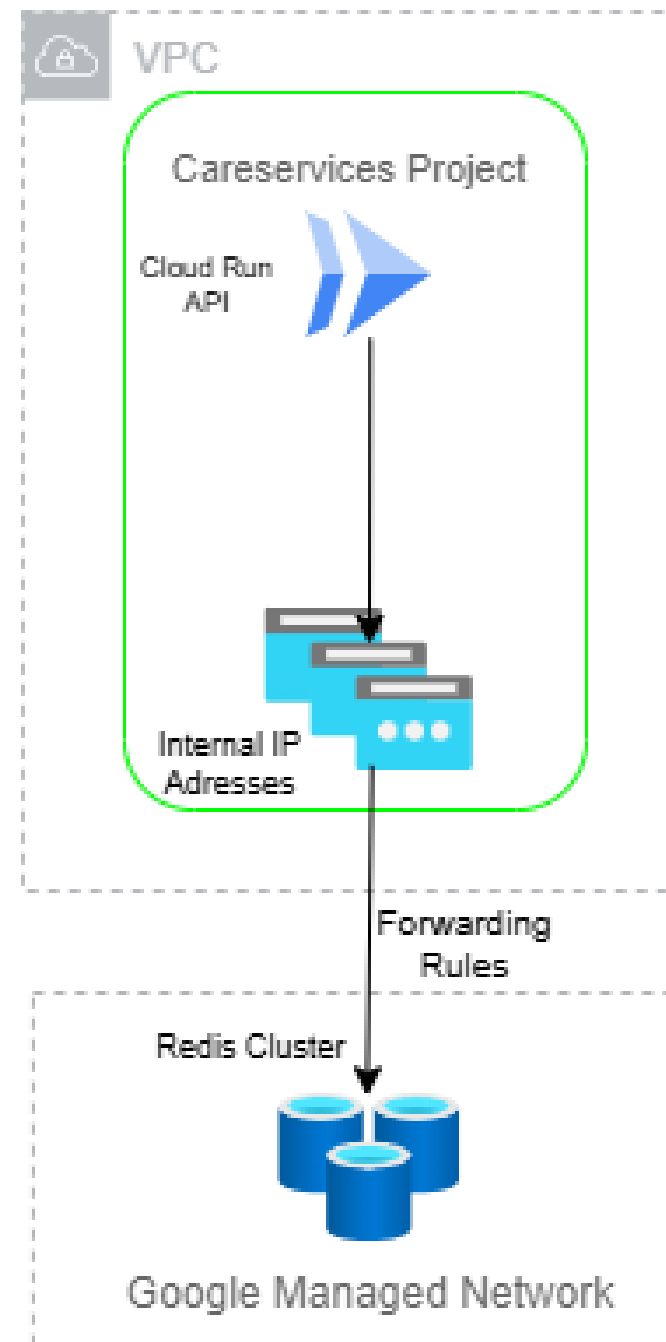
Solution

To reduce duplicate LLM calls, we introduced a **Redis-based caching layer** and built the cloud infrastructure needed to support it.

- Introduced a **Redis database** as a caching layer
- Provisioned a **Redis Cluster** with IAM Auth + encrypted transit
- Exposed Redis via **Private Service Connect** for private internal access
- Created internal PSC IPs and forwarding rules for Cloud Run

Result

A fully provisioned **Redis DB + PSC infrastructure** is now ready for the API to use as a high-speed caching layer.





Unplanned Work

Problem Statement

Note Pre-Processing requests were failing which meant no summaries were being generated.

Solution

By recreating the error, we found that the LLM calls were raising errors. This was due to complex response schemas sent as Python dictionaries. These were reconstructed to use Google's GenAI SDK objects instead.

Result

- Backlog of incomplete request were queued and processed
- Alert system has been created to identify a significant number of failing requests

```
summarisation_response_schema = {  
    "type": "object",  
    "properties": {  
        summary_key: {  
            "type": "string"  
        }  
    },  
    "required": [summary_key]  
}
```



```
summarisation_response_schema = types.Schema(  
    type=types.Type.OBJECT,  
    properties={  
        summary_key: types.Schema(type=types.Type.STRING)  
    },  
    required=[summary_key]  
)
```

Next Sprint Goals

1

Adapt app to use the new Redis database

Review and add code in confluence page to the repo.

2

Classification prompt improvements

Continue Ronak's work, incorporating feedback into the classification route

3

~~Ingest data into Snowflake~~ Formally approve plan

Following approval, start moving log data into Snowflake

4

Improve app version and usage visibility

- Create an automatically updating change log showing which changes are included in which versions
- Fix the monitoring dashboard to show the correct number of successful/failing requests

Sprint Goals



Prioritised shadow traffic enablement

Adapt app to use the new Redis database
Review and add code in confluence page to the repo.



Enable shadow traffic
Refactor app functionality to disable LLM calls for fake requests



Continue plan for data migration to Snowflake
Creating and reviewing documentation



Improve app version and usage visibility

- Create an automatically updating change log showing which changes are included in which versions
- Fix the monitoring dashboard to show the correct number of successful/failing requests

Enable Shadow Traffic

Problem Statement

To load test the AI service, Sapient are planning to send “shadow traffic”: fake requests simulating real ones. We don’t want to use LLMs for this traffic as it would be unnecessarily expensive.

Solution

- Create “Text Generators” for different circumstances: these are the class responsible for contacting LLMs and requesting content. All these text generators will have identical methods.
- Create a “Text Generation Factory”: a class responsible for setting up text generators based on input variables.
- Refactor app and business logic to use the factory and text generators.

Result

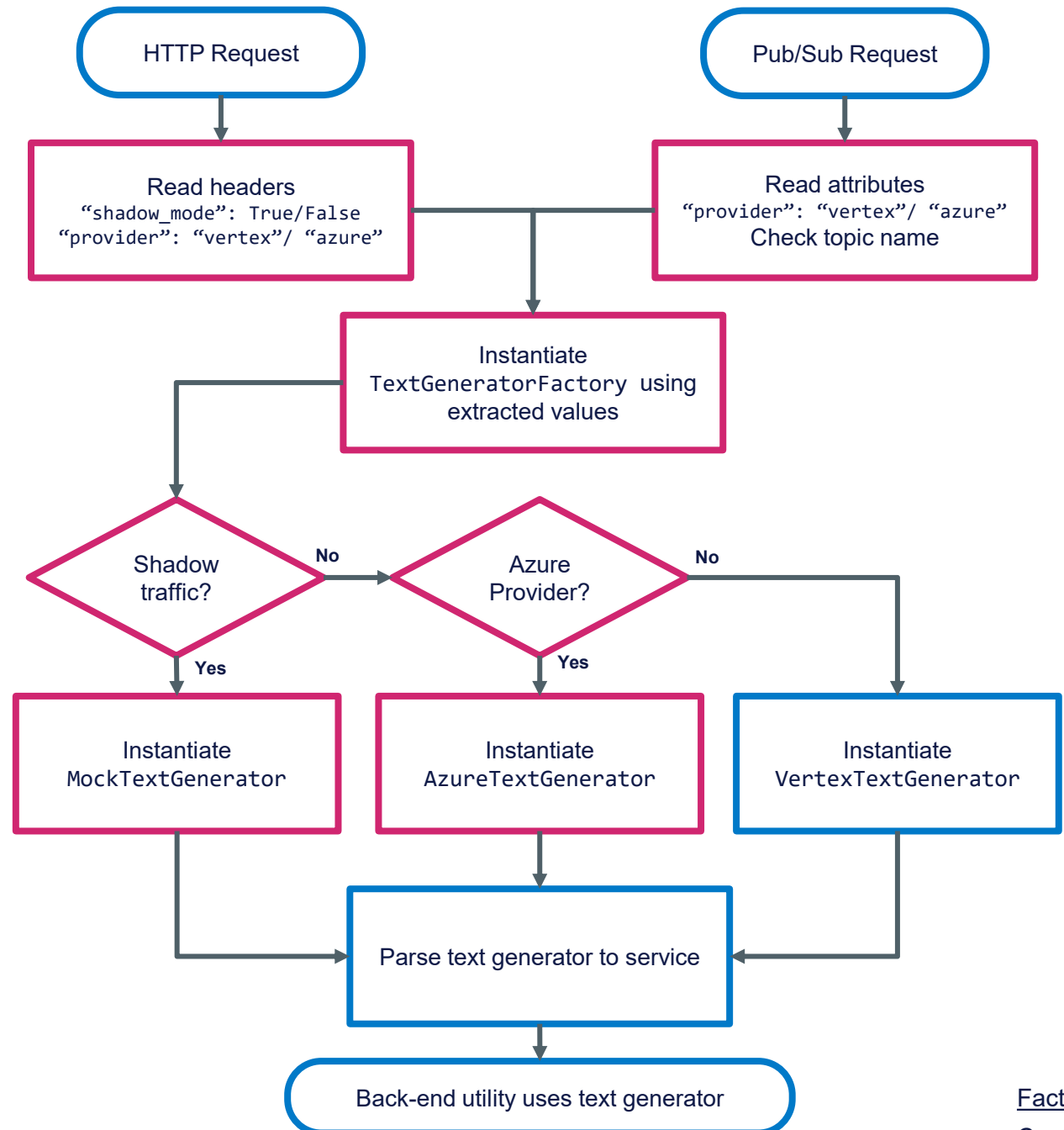
- Automatic routing of requests to the correct text generator based on request context
- Enables mapping of models to tasks (rather than setting models by route)
- Easier to test the route decorator (thanks to the refactor)
- Mock classes can be used when LLMs are unavailable (e.g. offline environment)

Issues

Significant refactor => took a long time to unit test

Next Steps

- Code review and PR changes
- Create a new topic/subscription for the shadow traffic



Data Migration Plan

Problem Statement

We are creating **JSON logs** from our CareServices AI API. Each JSON log contains structured metadata including the request details, the model inputs and outputs, moderation and PII decisions, classification results, and the summarisation outcome.

These JSON logs are stored in GCS, but querying a particular log is slow and inefficient with the current notebook-based retrieval process. We need to put these JSON logs into a database where they can be queried quickly and reliably.

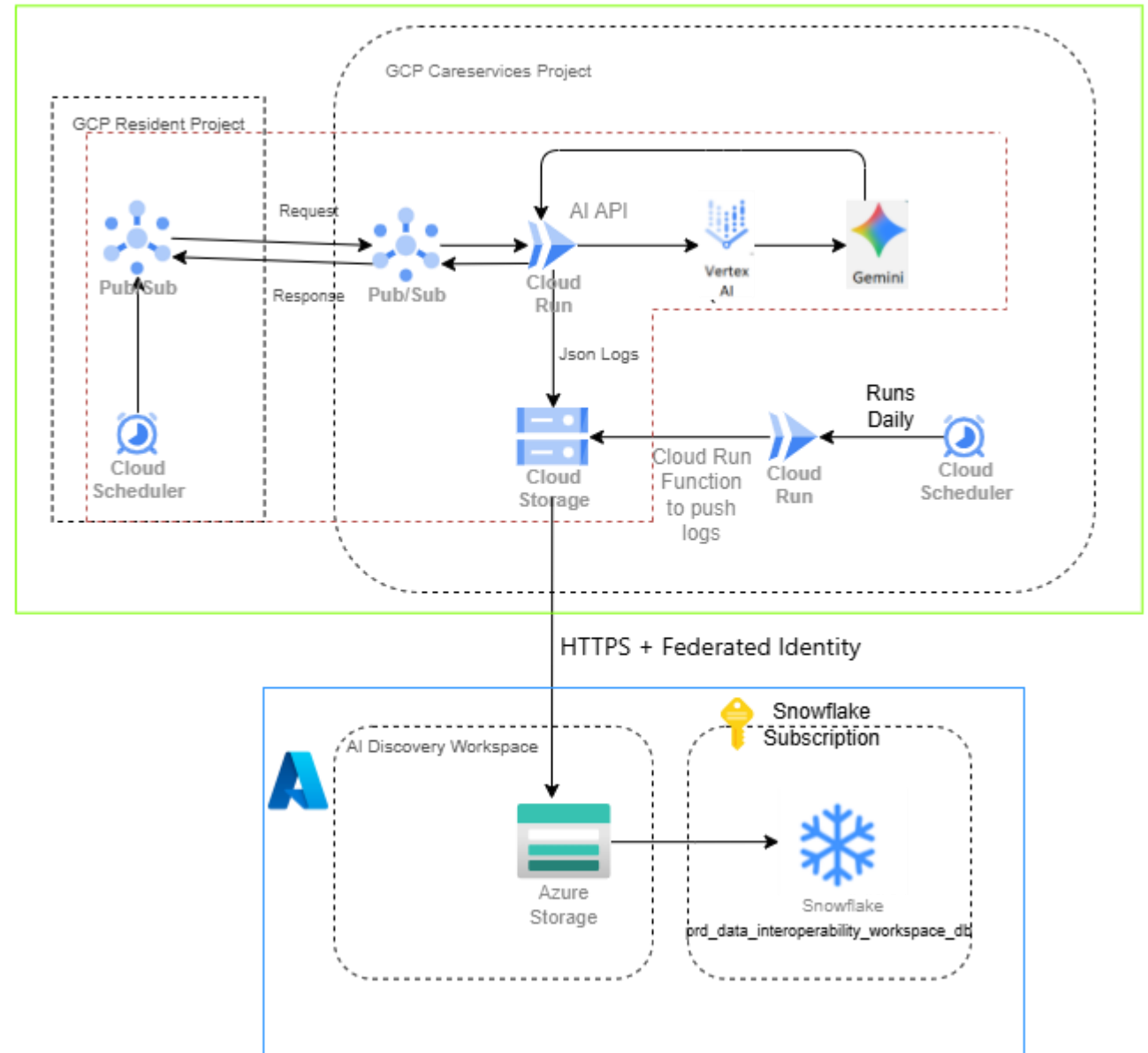
Since the Care Services team already consumes all of their data in **Snowflake**, and Snowflake is the established platform for analytics, BI, and operational investigations with existing access already granted to the relevant teams it makes the most sense to move these JSON logs into Snowflake.

Solution

We will present a SDD to the DDA for approval to use the existing **GCP** → **Azure private connectivity** and the **AI Discovery Workspace** as an interim landing zone for these JSON logs. In the long term, the strategic solution is to use **Azure Data Factory (ADF)** to ingest the data directly from GCS into Snowflake once ADF becomes available in the ACP environment.

Progress

SDD creation in progress



App Usage Visibility

Problem Statement

The monitoring dashboard was showing inaccurate request numbers, success/failure counts were incorrect due to inconsistent metric labels and the wrong aggregation settings.

Progress

Request Metrics Fixed

Standardised log-based metrics so request, success, and failure counts all use a single consistent route label.

Updated dashboard queries to use: `crossSeriesReducer = REDUCE_SUM`

`groupByFields = ["metric.label.route"]`

This merges duplicate time series and ensures each route appears once with the correct request totals.

Token Metrics Aligned

Updated all token metrics (total, cached, prompt, candidates, thoughts) to use a consistent set of labels: task, model, route.

This prepares the token dashboard to group and display values correctly once compatible aggregators are applied.

Proposed Next Sprint Goals

1

Adapt app to use the new Redis database

Review and add code in confluence page to the repo.

2

Finalise and review data migration plan

Ready to be presented to the DDA

3

Complete Shadow Traffic Change

- Code review and PR
- Cloud resources (new Topic and Subscription)
- Move to dev

4

Finish fix for Monitoring dashboard

5

Backlog Reshuffle

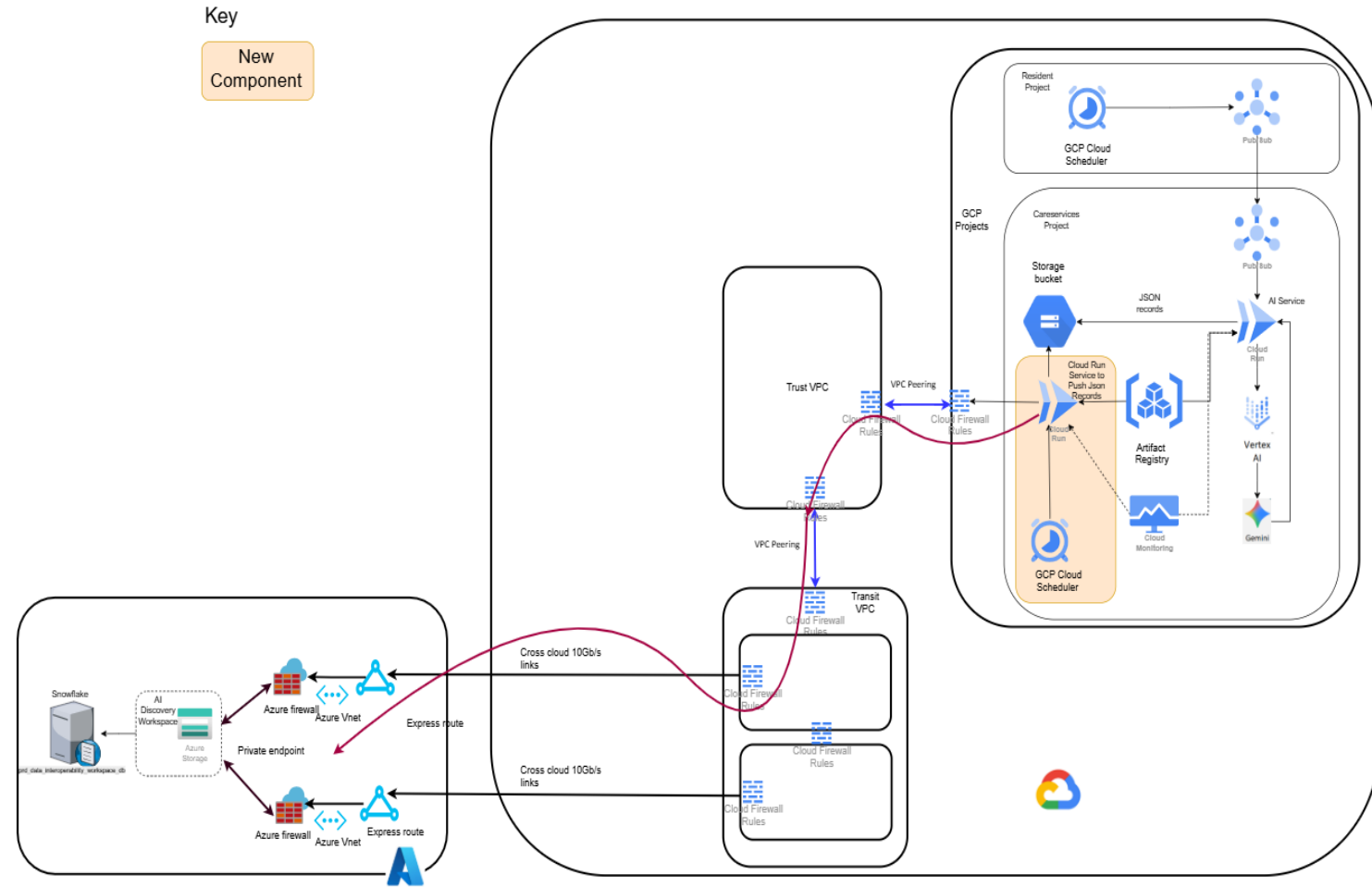
Align with feedback tracker

Sprint Goals

- **1 Adapt app to use the new Redis database**
Review and add code in confluence page to the repo.
- **2 Finalise and review data migration plan**
Ready to be presented to the DDA
- **3 Complete Shadow Traffic Change**
 - Code review and PR
 - Cloud resources (new Topic and Subscription)
 - Move to dev
- **4 Finish fix for Monitoring dashboard**
- **5 Backlog Reshuffle**
Align with feedback tracker

Data Migration Plan

What changed in the diagram



LLM Response Models

Use pydantic models for easier editing and communication of LLM response structures. New Vertex-specific method that converting into genai SDK objects.

```
CLASS_KEY = "class_value"
JUSTIFICATION_KEY = "justification"

class ClassificationResponse(BaseModel):
    """
    A model representing the result of a classification.
    """

    class_value: bool = Field(
        alias=CLASS_KEY,
        description="The boolean result of the classification (e.g., True if the criteria are met).")

    justification: str = Field(
        alias=JUSTIFICATION_KEY,
        description="A string explaining the reasoning behind the classification decision.")
```



```
Schema(
    description="A model representing the result of a classification.",
    properties={
        'class_value': Schema(
            description='The boolean result of the classification (e.g., True if the criteria are met).',
            type=<Type.BOOLEAN: 'BOOLEAN'>
        ),
        'justification': Schema(
            description='A string explaining the reasoning behind the classification decision.',
            type=<Type.STRING: 'STRING'>
        )
    },
    required=[
        'class_value',
        'justification',
    ],
    type=<Type.OBJECT: 'OBJECT'>
)
```

Backlog Refinement

Problem Statement

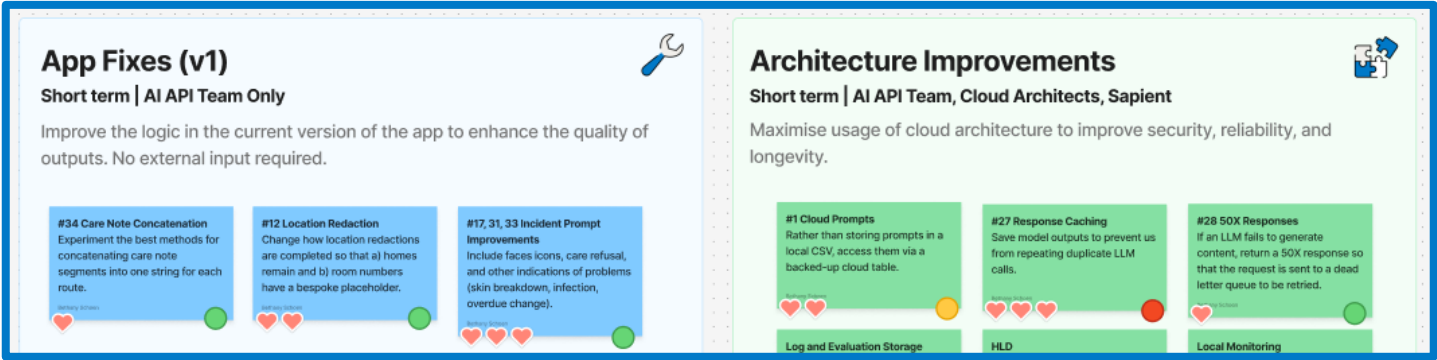
It was difficult finding existing tasks in ADO. It didn't match the structure of planning that had been completed in Bucharest in January.

Solution

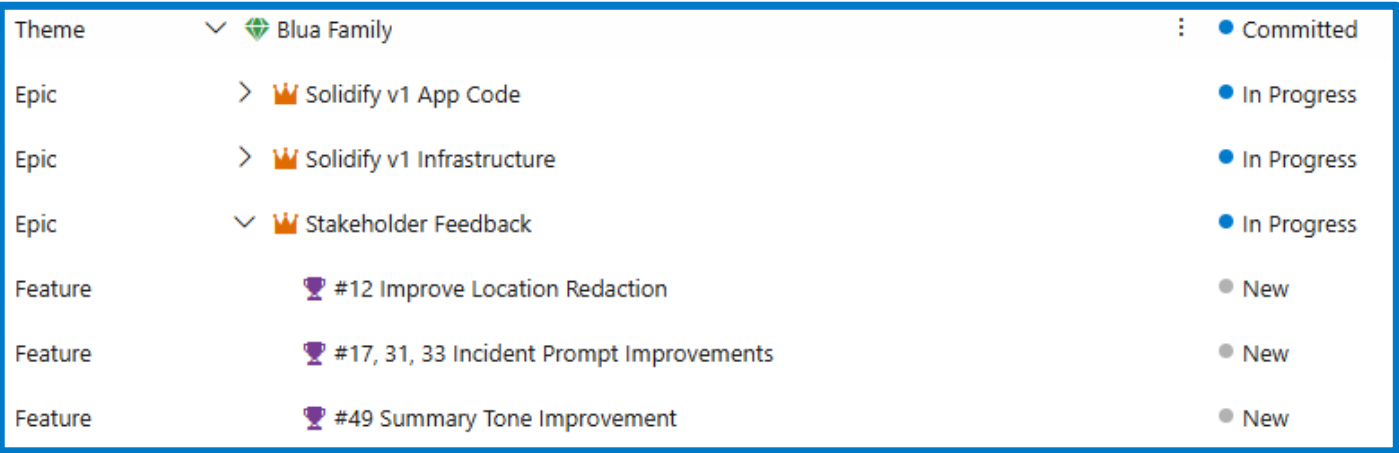
- New backlog with epics that follow the structure of the Figma board
- Align features with items in the feedback tracker so that we can capture ideas quickly in the spreadsheet, and prioritise them in the backlog

Result

- Much easier to visualise work to do
- Easier to plan and share PBIs
- Clearer vision of how much work is outstanding/team capacity



16	-	Specific 2.5 model versions	App logic	Low	Medium - more robust evaluation	Not started
17	-	Incident prompt rewording	Route logic	Medium - easy prompt change but requires evaluation	help with wording of non incident definition	In the backlog
18	-	Store log data in Snowflake	App logic	High	High	In progress
20	-	3rd party pronouns	Prompt update	Medium - easy prompt change but requires evaluation	Low - expect model to assume they/them	Completed



Sprint Goals



Deploy Shadow Traffic in Dev



Response Caching App-Side Changes



Text embedding models not provisioned

SPIKE: Classification v2
How can we adapt the existing methodology?



It's currently being reviewed

Approve Data Migration Plan



Troubleshoot connection to Redis Cluster

Resolve App Code Duplication
Shared code between the evaluator and app folders



SPIKE: Prompt Storage
Cloud storage for accessing and versioning prompts

Response Caching

Problem Statement

Add flask-caching functionality to the app

Solution

- A new caching decorator with custom key generation was created. It takes a specific field from within the request as well as app, model, and prompt versions to generate a cache value to be stored and checked
- (To help with conceptually understanding this, I explored stacking decorators)
- An invalidation script was created for removing specific content from the database

Result

- All app-side changes have been made and unit tested
- The cache is working with simple cache (local storage) in the workbench

Next Steps

- Integration test with a Redis instance

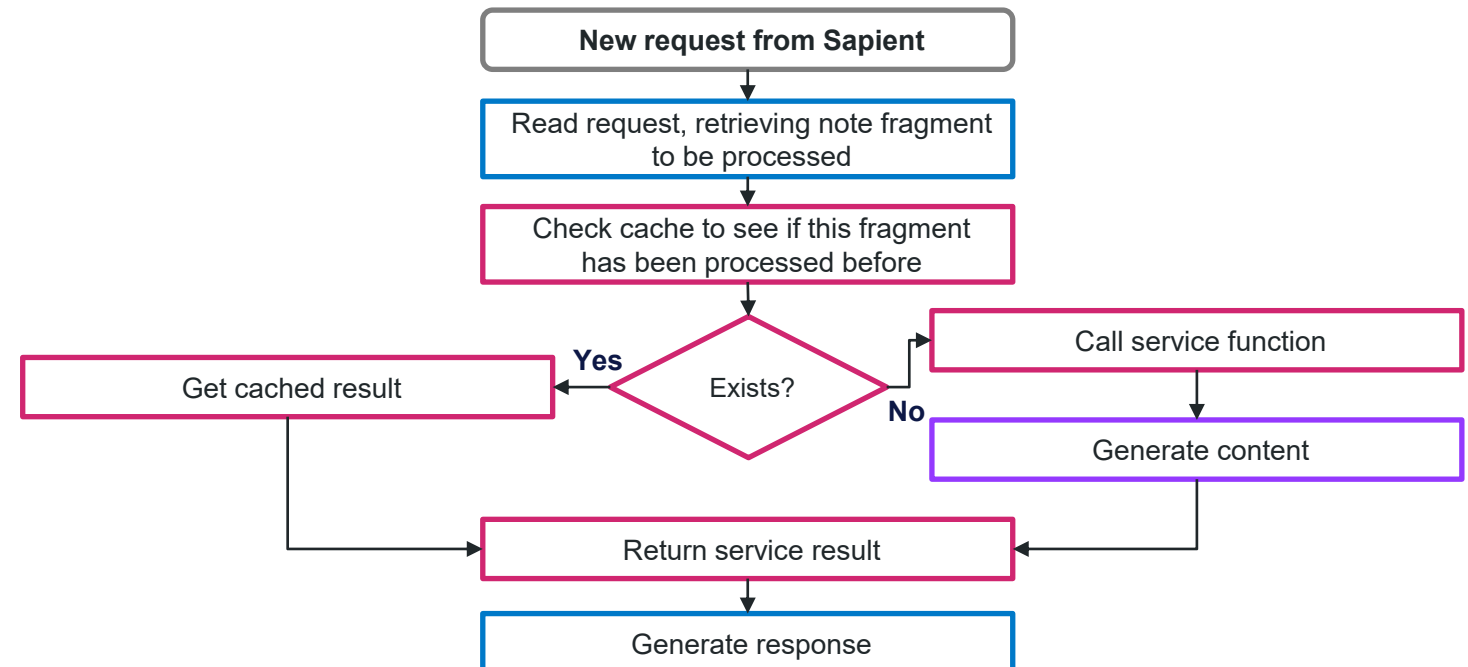
App Logic

```
@v1_bp.route(f"/notepreprocessing", methods=["POST"])
@api_route(
    json_request_model=NotePreprocessingRequestModel,
    avro_request_model=NotePreprocessingEvent,
    service_result_model=NotePreprocessingServiceResult,
    req_avro_schema_name='com.blua_family.ai.request.ObservationPIIEvent',
    res_avro_schema_name='com.blua_family.ai.response.ObservationPIIResponseEvent',
    response_pubsub_topic_name="NOTE_PREPROCESS_RESPONSE_TOPIC"
)
@api_cached(
    timeout=3600,
    route_name="notepreprocessing",
    field_name='observation', ← Field within the request to be cached
    prompt_versions=pv.NOTE_PREPROCESSING_PROMPT_VERSION_STRING,
    models_used=mv.NOTE_PREPROCESSING_MODEL_STRING
) } Used to create the cache key

def preprocess(validated_input: NotePreprocessingEvent | NotePreprocessingRequestModel, get_generator: Callable[[str], TextGenerator]):
    """
    API Endpoint for pre-processing care notes
    """
    return note_preprocessing_service(validated_input, get_generator)
```

Caching (New)

Business Logic



Classification v2

Problem Statement

The existing incident classification solution is working well but relies heavily on an LLM. Therefore, it is expensive and slow. Can we use ML tools to achieve the same goal?

Solution

Spike to explore LLM usage and ML techniques:

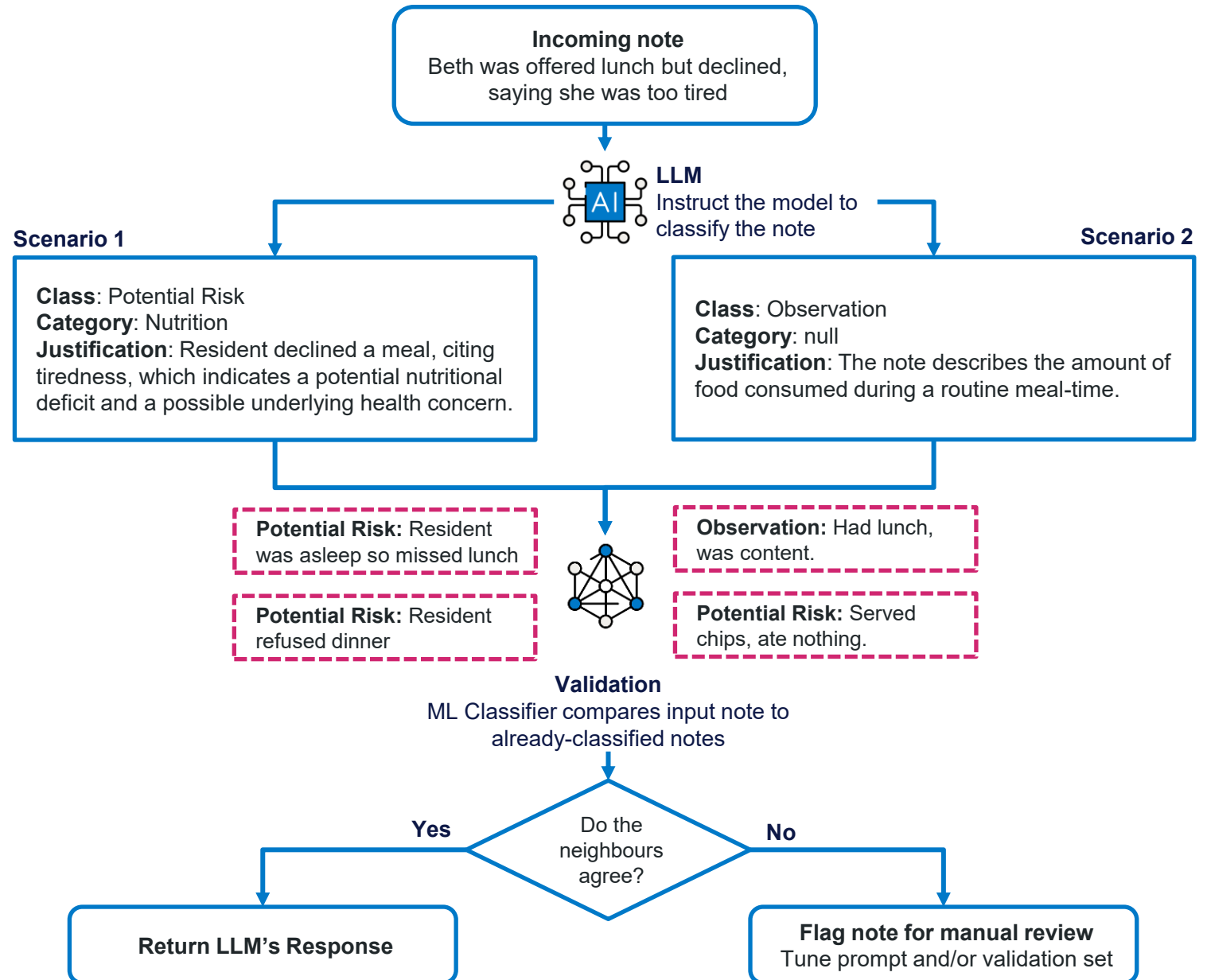
- Redefine problem to multi-class classification to help with clearer decision boundaries
- ML classifier for LLM decision validation – KNN with embedded notes strategically selected to cover edge cases
- Blocked: no text embedding models available

Result

- New [confluence page](#) detailing discovery work completed so far

Next Steps

- Provision text embedding model
- Create training dataset and classifier
- Finish confluence page



Spike: Prompt Storage

Problem Statement

Spike: Prompt Management for Cloud Run + Gemini

Solution

Identified limitations in the current approach where Gemini prompts are stored in an Excel file and embedded into the Cloud Run container image.

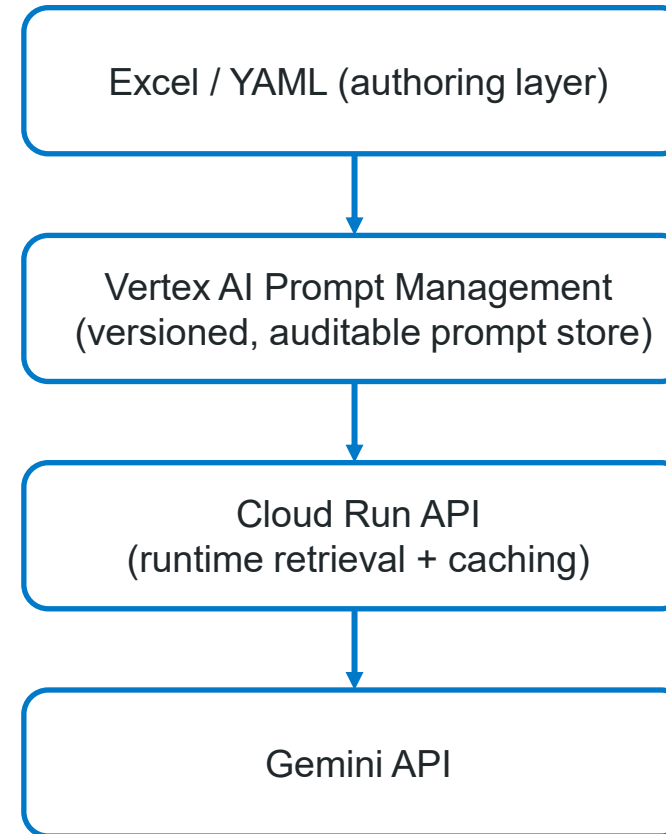
Assessed risks around scalability, operability, and governance due to tight coupling between prompt content and application deployments.

Investigated GCP-native alternatives to manage prompts more effectively in a production environment.

Result

- Evaluated **Vertex AI Prompt Management** as a managed, versioned prompt store.
- Designed a target architecture where:
- Prompts are externalised from the container image.
- Cloud Run retrieves prompts dynamically from Vertex AI at runtime.
- Prompt version selection is controlled via environment variables.
- Defined a promotion strategy (dev → test → prod) using fixed prompt versions.
- Identified a caching approach to mitigate runtime latency:
- Load prompt once per Cloud Run instance.
- Cache prompt in memory for the lifetime of the instance.

Target Architecture



Resolve App Code Duplication

Problem Statement

There is code duplication between the AI service and the AI Evaluation service.

Shared logic has been copied into both services, meaning any change made in one service must be manually replicated in the other, increasing the risk of divergence and maintenance overhead.

Solution

Two approaches were investigated:

Option 1 — Shared Folder

- Move common files into a shared directory within the repository
- Both services import the same source files
- Minimal setup and low complexity
- Suitable when the amount of shared code is small and still evolving

Option 2 — Local Shared Library

- Extract common logic into a reusable internal library
- Services install and depend on the library (e.g. via pip)
- Introduces explicit ownership, versioning, and clearer boundaries
- More setup and maintenance overhead

Result

At the current scale, adopting **a shared folder is the most appropriate approach**. This reduces duplication and maintenance risk without committing to premature abstraction. The option to extract a local library remains open if shared code grows or becomes more widely reused.

Shared Folder 	Local Library  (for now)
• < 10 files • Low complexity • Fast changes • No dependency management	• Extra setup • Versioning cost • Unknown reuse



Unplanned Work

Bug 1

Following the shadow traffic change, Pub/Sub requests were failing. This was because attributes were not being returned (this was an easy fix)

Bug 2

Problem Statement

What if a resident has the same first and preferred names? Currently, the voided route treats the preferred name as gospel truth (i.e. no variations are allowed). But if the preferred name is the same, then we don't really know if the resident has a nickname.

- 76% (319/421) of residents don't have a different preferred name after cleaning.
- In prod void logs, out of 1000 notes, 14 (belonging to 8 residents) used a nickname that didn't match the preferred name.

Solution

Check if the resident's first and preferred names are the same. If they are, don't send the preferred name to the LLM, permitting it to follow the instruction to identify common nicknames.

Result

- More flexible solution.

```
cleaned_first_name = pre_process_name(first_name)
cleaned_preferred_name = pre_process_name(preferred_name)
if cleaned_first_name == cleaned_preferred_name:
    logger.info(f"Preferred and first names are the same after pre-processing. Setting preferred name to None.")
    cleaned_preferred_name = None

...

# 1. Construct LLM input
content = {
    vr.first_name_param:first_name,
    vr.last_name_param:last_name,
    vr.preferred_name_param:preferred_name,
    vr.singular_note_param:care_note
}

# remove the preferred name if it's null
# do it after constructing the content dict so that the preferred name parameter is always present in the same place in the prompt
if not preferred_name:
    content.pop(vr.preferred_name_param)

content_str = json.dumps(content)

# 2. Call LLM to identify if note should be voided
text_gen_result = tg_client.generate_text(
    content_str,
    system_instruction=voided_prompt,
    response_model=VoidedResponse,
    task_str=vr.voided_task_str
)

...
```

```
{
  "resident_info": {
    "firstName": "Elizabeth",
    "lastName": "██████████",
    "preferredName": "Elizabeth",
    "sex": "Female"
  },
  "connection_id": "████████████████████████████████████████",
  "observation": "Liz's mattress: Liz's Mattress checked and found to be ok.",
  "response_data": {
    "correctResident": "INCORRECT",
    "justification": "The name 'Liz' is used in the note. Although a common nickname for 'Elizabeth', it does not match the resident's preferred name 'elizabeth', and thus falls under the rule of not accepting other nicknames when a preferred name is provided.",
    "extractedNames": [
      "Liz",
      "Liz"
    ]
  },
  "updated_response": {
    "correct_resident": "CORRECT",
    "justification": "The name 'Liz' is a common nickname for 'Elizabeth' and no preferred name has been provided. The note describes the resident's mattress being checked, indicating direct participation.",
    "extracted_names": [
      "Liz"
    ]
  }
}
```

Before fix

After fix

Sprint Goals



Connection to Redis DB established

Working on the workbench and Cloud Run



Explore ShadowTraffic

Create data to evaluate Voided changes

Shadow Traffic

Dummy Data Creation

Problem Statement

We needed to create a labelled dataset of notes to test the AI services more vigorously (create more scenarios that we don't see often in production data).

Solution

Use Shadow Traffic to generate labelled data:

1. Design a config for creating care note "scenarios": which resident is the note referring to, how has their name been used, does anyone else feature, etc.
2. Use Gemini in Vertex to parse these scenarios and create a care note for testing
3. Label the data using switches (e.g. if the note resident is "target" then expected label is "correct").
4. Process the fake data through the services and evaluate

Result

- A new config has been created for the voided service
- We understand how to run Shadow Traffic locally, using complex functionality and saving to the local file system
- The voided service was evaluated on 100 data points
- **97/100 classified correctly (3 understandably wrong)**
- **All extracted names were correct**

Next Steps

- Apply to the Note Pre-Processing route
- Use Shadow Traffic in Cloud Run via GitHub workflows

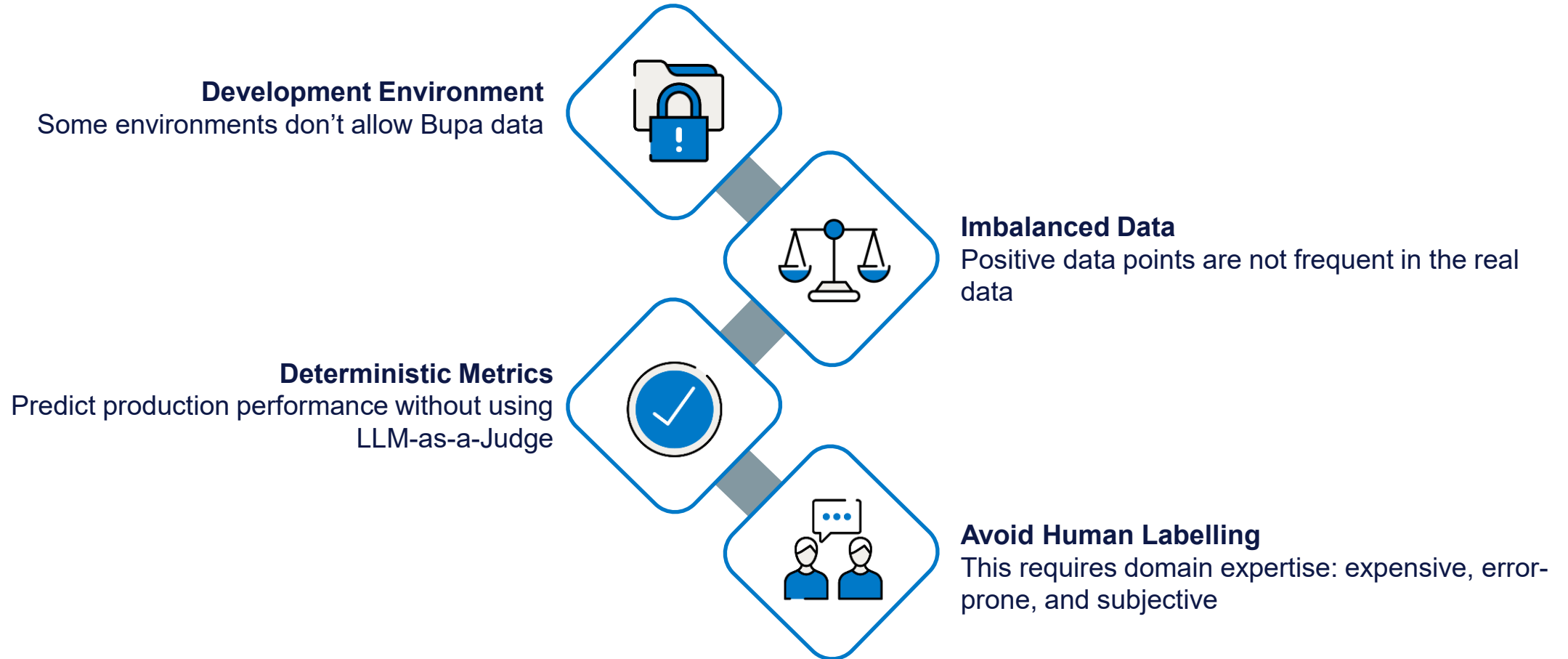
```
{
  "targetResident": { // the resident the note has been saved to
    "firstName": "Moshe",
    "preferredName": "Arthur",
    "lastName": "Weissnat",
    "sex": "Male"
  },
  "alternativeResident": { // an alternative resident in the same home
    "firstName": "Shoshana",
    "lastName": "Tillman",
    "sex": "Male",
    "preferredName": "Tyrell"
  },
  "noteScenario": {
    "noteResident": "alternative", // who is the note really about?
    "additionalNamesMentioned": "Reed", // is anyone else mentioned?
    "sentiment": "neutral" // to help with creative writing
  },
  "nameUsage": {
    "pronounUsage": "non-binary", // use the resident's pronouns?
    "type": "first", // which name fragment should be used?
    "misspelt": false // is the name spelt incorrectly?
  },
  "careNote": "Shoshana had their morning tea in the lounge. They chatted briefly with Reed before returning to their room.", // the AI-generated note (using Vertex in Workbench)
  "expectedLabel": "INCORRECT", // manually labelled using the config
  "expectedNames": [ // only captured when using first or preferred name, not misspelt
    "Shoshana",
    "Reed"
  ]
}
```

ShadowTraffic and Blue Family

Generating Testing Data

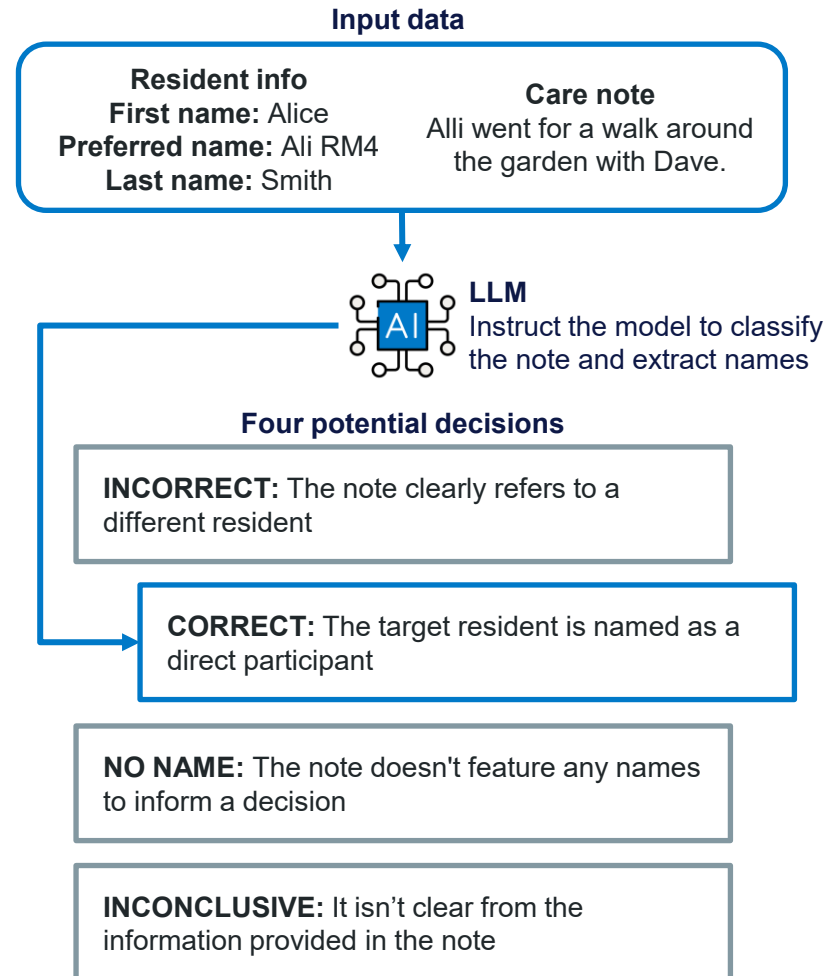
COPIED TO SEPARATE DOC

LLM Evaluation Constraints



We need a scalable way to generate realistic, labelled, privacy-safe evaluation data with known expected outcomes

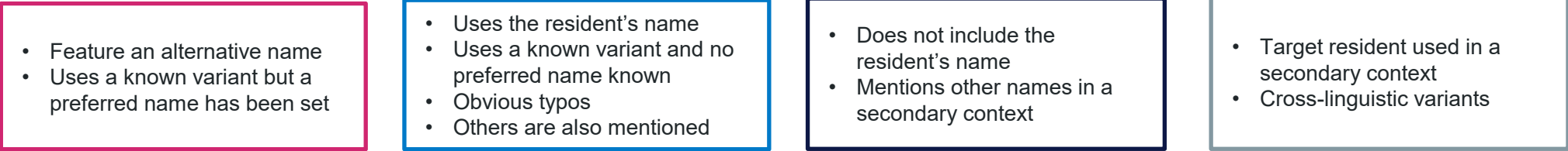
LLMs are used to identify care notes saved to the wrong resident



Output Classes



Linguistic patterns that produce the outcome



Represent as structured variables

Structured Variable	Datatype	Effect on AI Note
noteResident	enum: [target, alternative]	Which resident should the note be about: target or alternative
additionalNamesMentioned	string null	Is another name mentioned in the note? If so, what is their name?
pronounUsage	enum: [correct, incorrect, non-binary]	Should the pronouns that align with the resident's recorded sex be used?
nameFragment	enum: [first, preferred, variant, etc.]	Which part of the resident's name should be referenced in the note?
misspelt	boolean	Has the name been spelt correctly?

ShadowTraffic generates combinations of these variables.

We control the variables, rather than the text itself.

ShadowTraffic allows us to design a controlled test dataset

1

Generate structured scenario

Populate parameters defining the nuances of the note

2

Render to natural language

LLM converts the structured scenario into a realistic care note

3

Automatically derive labels

The label can be inferred deterministically

Design configurations to create varying care notes

Correct Scenario

☒	Note subject: Target resident (Alice)
☒	Name fragment: Preferred name
☐	Correct spelling: Nope
☒	Anyone else mentioned: Dave



Care note 

Alli went for a walk around the garden with Dave.

Incorrect Scenario

☐	Note subject: Alternative resident (Dave)
☒	Name fragment: First name
☒	Correct spelling: Yep
☒	Anyone else mentioned: Nope



Care note 

Dave had toast for breakfast

Here's an example ShadowTraffic created

```
{
  "targetResident" : {
    "firstName" : "Moshe",
    "preferredName" : "Arthur",
    "lastName" : "Weissnat",
    "sex" : "Male"
  },
  "alternativeResident" : {
    "firstName" : "Shoshana",
    "preferredName" : "Tyrell",
    "lastName" : "Tillman",
    "sex" : "Male"
  },
  "noteScenario" : {
    "noteResident" : "alternative",
    "additionalNamesMentioned" : "Reed",
    "sentiment" : "neutral"
  },
  "nameUsage" : {
    "pronounUsage" : "non-binary",
    "type" : "first",
    "misspelt" : false
  }
}
```

AI-Generated Note

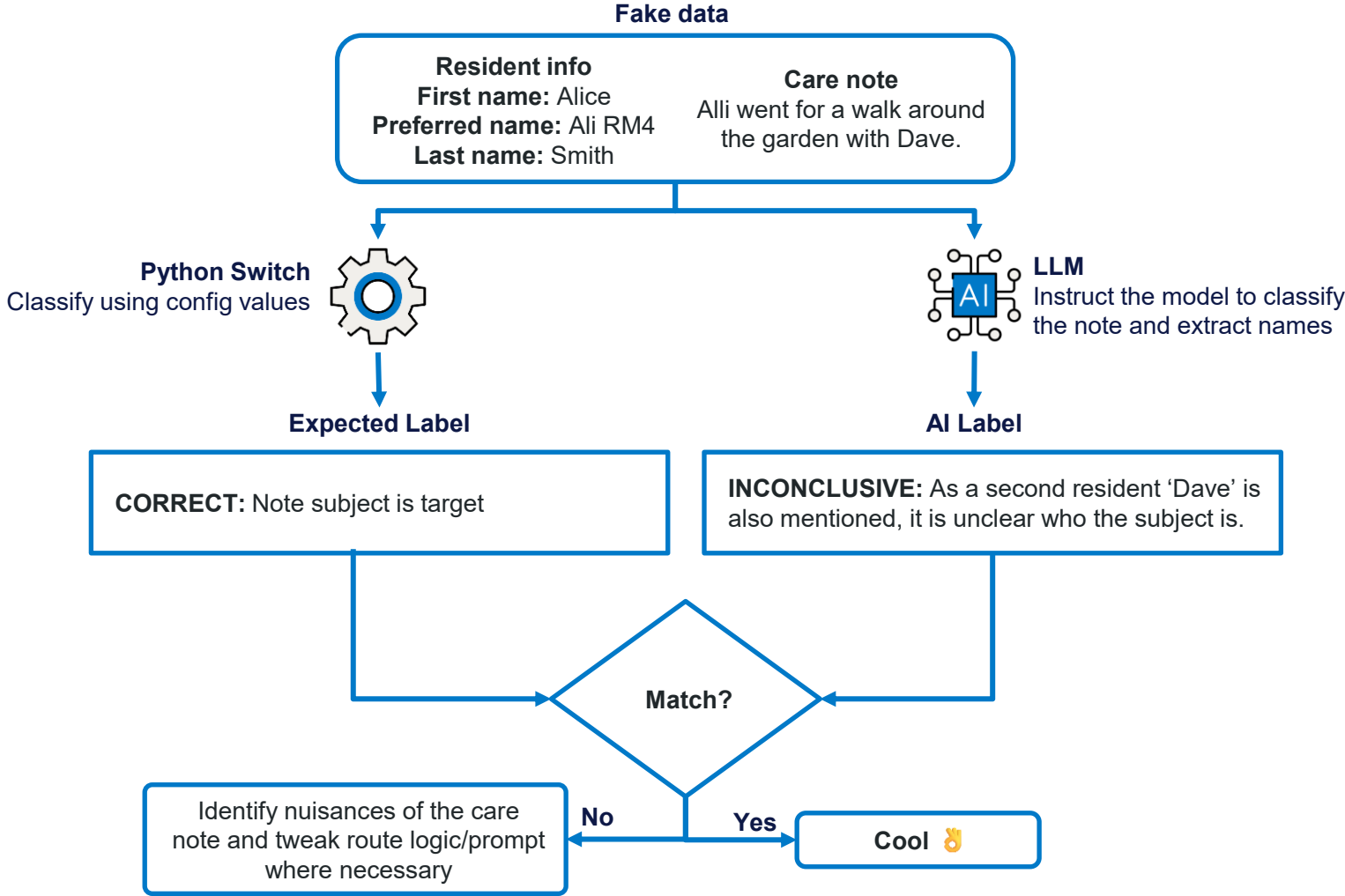
"Shoshana had their morning tea in the lounge. They chatted briefly with Reed before returning to their room."



Expected label: INCORRECT

Expected extracted names: ['Shoshana', 'Reed']

Process the fake data through our voided service



Correct Example

1. Shadow Traffic
2. AI Generated
3. App Output

Target Resident	Alternative Resident
Name: Jenelle D'Amore Preferred: Jenelle S1 Sex: Female	Name: Olin Huels Preferred: Hilma Sex: Male
Note Scenario	Name Usage
Resident: Alternative Additional names: None Sentiment: Negative	Pronouns: Correct Type: First Misspelt: True
Care note: Olen struggled with personal care this morning, refusing assistance with washing and becoming agitated. He seemed very unhappy.	

Voided Result
Label: Incorrect Justification: The note refers to 'Olen' as the subject of care, which is inconsistent with the resident's name 'Jenelle'. Extracted names: ["Olen"]
Evaluation Result
The production model correctly identified that the note refers to 'Olen' as the subject of care, which is a distinct name inconsistent with the resident 'Jenelle'.

Incorrect Example

Target Resident

Name: Marianela Schinner
Preferred: Marianela
Sex: Male

Alternative Resident

Name: Tanner Rempel
Preferred: Blaine
Sex: Female

Note Scenario

Resident: **Target**
Additional names: Alex
Sentiment: Positive

Name Usage

Pronouns: Correct
Type: Last & Title
Misspelt: True

Mr. Schiner enjoyed a pleasant conversation with Alex in the common room, he seemed very engaged and happy.

Voided Result

Label: Incorrect
Justification: The note refers to 'Mr. Schiner', which is inconsistent with the resident's first name 'Marianela' and the title 'Mr.', indicating a different resident.
Extracted names: ['Mr. Schiner', 'Alex']

Evaluation Result

The production model correctly identified that the note refers to 'Mr. Schiner', which is inconsistent with the resident's female first name 'Marianela' and the masculine title 'Mr.'. This clear gender mismatch (indicated by the title) firmly establishes that the note is about another resident, despite the similar last name (even with a minor misspelling).

Sprint Goals



Caching Working in Dev
Working in Cloud Run



Not yet running in GCP, requires cloud run

ShadowTraffic Refinements
Use AI models directly within the config
Run in GCP



Improve Incident Classification
Update prompt

Caching in Dev

Problem Statement

We initially implemented Redis-based caching using Flask-Caching to improve API performance and reduce repeated Gemini calls for duplicate requests.

However, the production Redis cluster requires TLS and IAM-based authentication, which Flask-Caching's Redis integration does not support, making the original approach unsuitable for production.

Solution

- Built a custom Redis caching implementation instead of relying on Flask-Caching.
- Added support for IAM token-based authentication when connecting to Redis.
- Enabled TLS-secured Redis connectivity using the required production certificates.
- Integrated the cache layer with the application running on Cloud Run.

Result

- Deployed the caching solution in DEV.
- Reduced response time from ~18 seconds to ~0.2 seconds for duplicate requests.
- Verified expected caching behaviour: cache miss on first request, cache hit on subsequent requests.

Next Steps

- Deploy to test and prod
- Build a custom dashboard to visualize cache performance, estimate cost savings.
- Add alerts for cache

Initial request (Not cached)

```
Time elapsed: 20.27s
{
  "metadata": {
    "connectionId": "5b8aa5a2d2c872e8321cf37308d69df2",
    "observationId": "270f15d3-3770-4fb3-8dd2-da30f45f5a75",
    "field": "note",
    "timestamp": 1759517418842
```

Duplicate request (After Caching)

```
Time elapsed: 0.02s
{
  "metadata": {
    "connectionId": "5b8aa5a2d2c872e8321cf37308d69df2",
    "observationId": "270f15d3-3770-4fb3-8dd2-da30f45f5a75",
    "field": "note",
    "timestamp": 1759517418842
```

Shadow Traffic

Refinement

Problem Statement

Improve the current ShadowTraffic setup

1. Use AI models within the Shadow Traffic config
2. Better reflect the nuances of production data to test additional functionality added recently to the voided route

Solution

1. Edit the config file to use LLMs and test locally
2. Allow the preferred name to occasionally be the same as the first name – achievable via Custom Functions
3. Add noise to the preferred name (e.g. room numbers) – simple python function

Result

- Config can be executed in Cloud Run without additional LLM connectivity
- More reliable data available for route testing

Next Steps

- Use Shadow Traffic in Cloud Run via GitHub workflows

```
"careNote": {
  "_gen": "ai",
  "service": "ollama",
  "ollama": {
    "url": "http://host.docker.internal:11434",
    "model": "llama3.2:3b",
    "prompt": {
      "_gen": "env",
      "var": "VOIDED_PROMPT"
    }
  }
}
```

Additional instruction for
care note generation
within the config

```
def name_noise_generator():
    """
    Randomly create room number noise to add to a name
    """
    name_space = random.choice(["", " ", " ", " "])
    room_types = ["", "room", "rm"]
    punctuation = ["", " ", "-", "_"]
    additional_letters = ["", "S"]

    room_noise = name_space + random.choice(room_types) +
    random.choice(punctuation) + random.choice(additional_letters) +
    str(random.randint(1, 50))

    return room_noise
```

Create a pseudo
random string to
add to a name

Incident Classification

Prompt Improvement

Problem Statement

1. Continue Ronak's work, refining the prompt to incorporate domain-expert feedback
2. Create a reusable system for testing the classification prompt

Solution

1. Worked iteratively with Care Services to understand the expected behaviour of the prompt – what should and shouldn't be an incident
2. Redesigned the prompt structure, guiding the model on how to interpret the provided information
3. As with other routes, we setup notebooks for exploring and testing the updated prompt on fake and real data

Result

- On 200 datapoints, average justification coherence score (LLM-as-a-Judge) was 4.9/5
- Notebooks are available for rerunning when required
- Classified data points and the updated prompt have been shared with Care Services for final sign-off.

Next Steps

- PR

New Structure

Context

The production model's role

Task

Incident definition

Conditions for True classification (Updated)

e.g. "Physical or Emotional Distress: Physical pain or emotional/psychological distress requiring intervention, monitoring, reassurance, or follow-up, regardless of resolution."

Conditions for False classification (New)

e.g. "Routine, neutral, observational, or planned notes without evidence of risk, harm, deterioration, escalation, or intervention."

Classification Instructions (Sequential Reasoning) (New)

Step-by-step guide instructing the model to read the entire note, look for explicit, high-severity events, look for procedural incidents, check for ambiguous situations, or classify as a non-incident.

Output Structure (New)

What the model must return

Examples (Updated)

Real notes classified

Data Journey Report

View care notes relating to an AI-generated summary and explore their transformation steps

1. Select a resident

Can also use other fields to help filter (e.g. home)

Resident

Joanna Smith

2. Select a date

This could also be a date range

Date

11 May 2026

3. Select a summary

Options will only appear once the slicers have been used to prevent the report from loading all summaries (large, slow query).

Time	Summary
12:05	Jo was helped out of bed. She then brushed her teeth, requiring little assistance, and was content. She was assisted with a warm shower and getting dressed for the day. Her daily blood pressure medication was administered by a staff member. For breakfast, Jo had a bowl of cereal with a cup of tea in the dining room, eating most of it and being content. She also had a conversation with two other individuals over breakfast, which lasted 30 minutes and was noted as being socially beneficial, with her appearing very engaged.
17:05	Jo's daughter called to update her home address. A caregiver offered her a cup of tea and pink wafers, which she ate. She then had a friendly chat with two other individuals, discussing her upcoming birthday. One of the individuals laughed about their cat, and they reminisced about earlier days. Jo had a smooth medical appointment with an individual, who provided emergency contact information. Later, she ate a good lunch of chicken casserole with vegetables and cleared her plate.

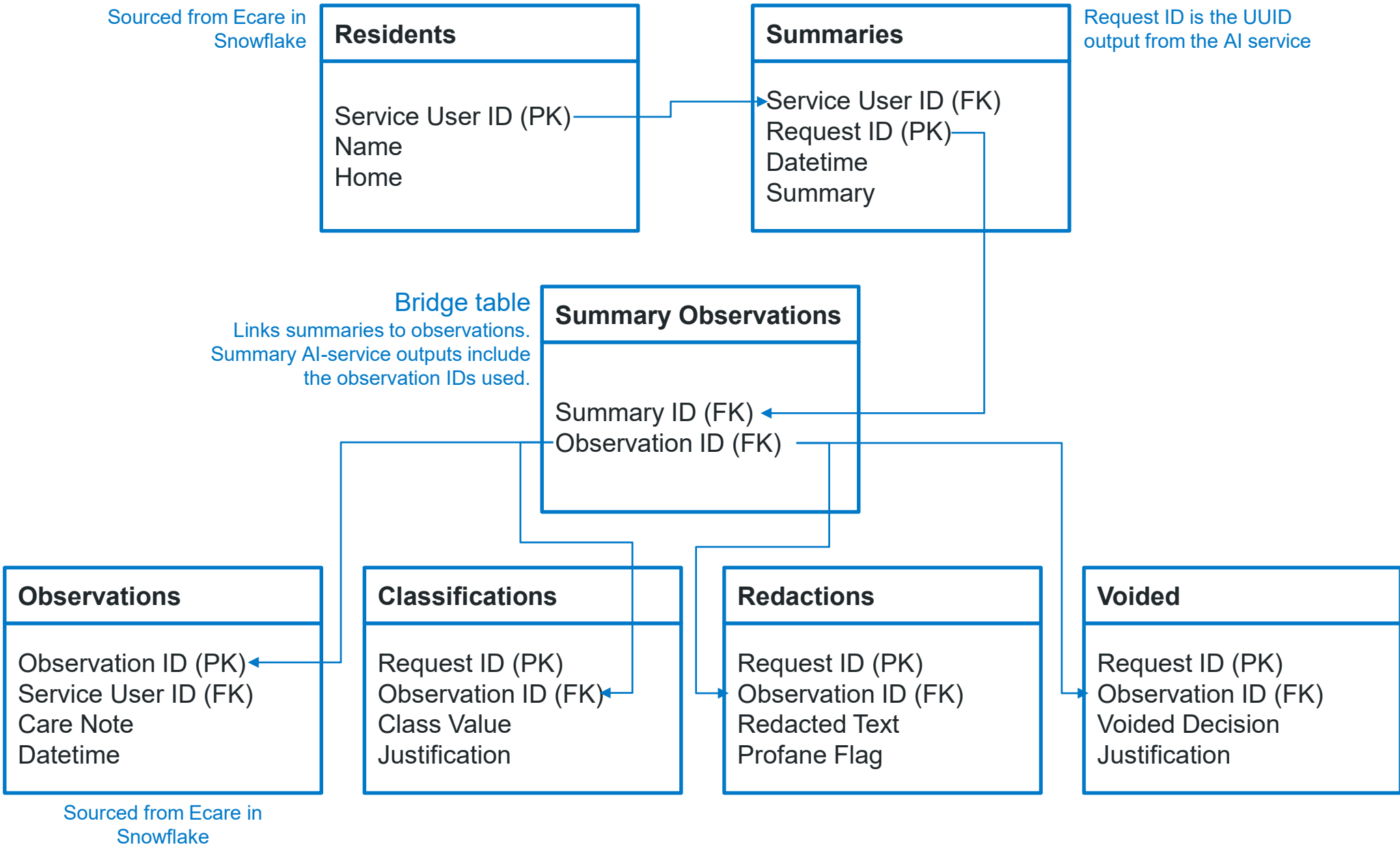
To control visibility, use DAX measures as visual filters, checking if the resident, date, and/or summary tables only have one value each

Obs. ID	Time	Care Note	Incident Justification	Redacted Text	Profane	Voided	Voided Justification
a1	06:30	Mobility: Helped out of bed	No incident mentioned.	Mobility: Helped out of bed	False	NO_NAME	No name is mentioned
b2	06:40	Personal care: brushed teeth, little to no help required, was content	No incident mentioned.	Personal care: brushed teeth, little to no help required, was content	False	NO_NAME	No name is mentioned
c3	06:55	Personal Care: Assisted with a warm shower and getting dressed for the day.	No incident mentioned.	Personal Care: Assisted with a warm shower and getting dressed for the day.	False	NO_NAME	No name is mentioned
e5	08:00	Medical: Nurse Dave administered daily blood pressure medication.	No incident mentioned.	Medical: [REDACTED_PERSON_NAME_1] administered daily blood pressure medication.	False	NO_NAME	No name is mentioned
f6	08:10	Nutrition eating and drinking: Jo had a bowl of cereal with a cup of tea for breakfast in the dining room, ate most of it, was content	No incident mentioned.	Nutrition eating and drinking: Jo had a bowl of cereal with a cup of tea for breakfast in the dining room, ate most of it, was content	False	CORRECT	Jo is mentioned
g7	09:30	Social: Had a conversation with Brian and Stephen over breakfast, seemed very engaged, lasted 30 minutes, there was a social benefit	No incident mentioned.	Social: Had a conversation with [REDACTED_PERSON_NAME_1] and [REDACTED_PERSON_NAME_2] over breakfast, seemed very engaged, lasted 30 minutes, there was a social benefit	False	NO_NAME	No name is mentioned

4. Observe care notes used in the selected summary

This will **not** show care notes flagged as voided or incidents as these weren't included in the summaries. Notes will only appear when a summary has been selected. We can display transformation steps completed (redactions, profanity flag, etc.) to allow users to identify errors.

Data Journey Report – Data Model



Sprint Goals



Note Order Consistency Metric

Fix logic and include with summary evaluation



Summary Tone Improvement

Investigate tones and prompt changes



ShadowTraffic in GCP

Run via Cloud Run



Redis/Caching Monitoring

Justify value of DB

Deprioritised due to environment migration

Note Order Consistency

Bug Triage

Problem Statement

A metric had previously been programmed to identify how well the production LLM maintains chronological order when generating a summary ([more info here](#)). However, it wasn't working on production data.

Solution

1. Recover metric logic from last year
2. Solidify code – pydantic to validate LLM inputs and outputs
3. Unit test
4. Run on real data

Result

New class has been programmed and tested. Errors have been identified: LLM says a note appears more than once; only one note to analyse.

Blockers

No workbench access to run on production data.

Next Steps

- PR

Errors Identified

Only one note going in

```
notes_list = [
    ("Nutrition", "Enjoyed a full breakfast.")
]

summary = "The resident enjoyed a full breakfast this morning and later had their hair brushed."

order_consistency_with_differing_strategies(tg_client, n_samples=3, notes_list=notes_list, summary=summary)
```

```
=== None ===
{
  "score": null,
  "justification": "{\\"kendall_tau\\": null, \\"std\\": null, \\"internal_order_score\\": null, \\"n_tau_scores_aggregated\\": null, \\"n_internal_scores_aggregated\\": null, \\"sampling_strategy\\": null, \\"n_samples\\": 1}",
  "prompt_version": 3
}
```

Score not calculated

```
notes_list = [
    ("Nutrition", "Enjoyed a full breakfast."),
    ("Personal care", "Had their hair brushed.")
]

summary = (
    "The resident had their hair brushed before enjoying a full breakfast this morning." \
    "Later, they had their hair brushed again."
)

order_consistency_with_differing_strategies(tg_client, n_samples=3, notes_list=notes_list, summary=summary)
```

Hair brushing is mentioned twice in the summary

```
=== None ===
{
  "score": -1.0,
  "justification": "{\\"kendall_tau\\": -1.0, \\"std\\": 0.0, \\"internal_order_score\\": null, \\"n_tau_scores_aggregated\\": 1, \\"n_ternal_scores_aggregated\\": 0, \\"sampling_strategy\\": null, \\"n_samples\\": 2}",
  "prompt_version": 3
}
```

Evaluator only considered the first appearance of the hair brushing note, not the second, resulting in a -1 score (perfectly opposite).

Summary Tone Improvement

Problem Statement

Our current "one-size-fits-all" summarisation prompt lacks context. It treats a busy afternoon the same as a quiet overnight period. The production LLM is also instructed to not infer any information from the notes. This leads to summaries that can be flat-toned, repetitive, and/or vague.

Can we improve the quality (readability and accuracy) of summaries for different times of day?

Solution

1. Identified potential tones and created prompt additions for each
2. Designed experiments for exploring prompt and example changes. These will allow us to choose which changes to make to optimise accuracy and readability.
3. Created a write-up of the experiments to be completed

Blockers

No workbench access to perform EDA and experiments on log data.

Next Steps

- Perform EDA on log data
- Design tone-specific metrics

Hypotheses



Time-specific examples improve readability more than time-specific prompts.



Tone examples improve readability more than tone instructions.



High-risk examples improve groundedness and note inclusion.



Common examples improve readability.



Temperature has minimal impact on accuracy metrics.

Shadow Traffic

Running in GCP

Problem Statement

We needed to generate realistic synthetic data to support ShadowTraffic testing on AI API, but manual approaches were not scalable or representative of real-world scenarios. Running Docker-based tools in our constrained environment presented additional challenges, alongside the need to securely manage prompts and integrate Vertex AI for LLM-driven data generation. The solution also required a reliable way to store and reuse the generated data.

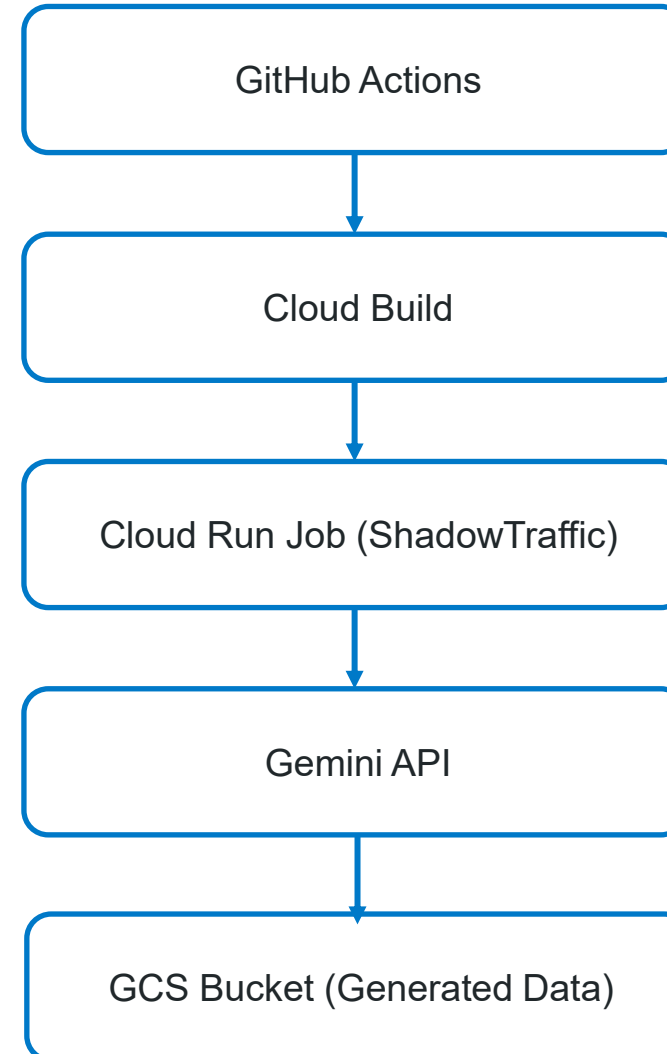
Solution

1. Built reusable Cloud Build pipeline to build & deploy ShadowTraffic image
2. Deployed as Cloud Run Job with controlled memory + execution settings
3. Integrated Secret Manager for secure prompt injection
4. Configured Vertex AI (Gemini) for LLM-based generation
5. Output written directly to GCS via ShadowTraffic bucketConfig

Result

1. Fully automated synthetic data generation pipeline
2. Secure, compliant handling of prompts and secrets
3. Scalable execution via Cloud Run Jobs
4. LLM-powered generation integrated with GCP services
5. Clean output stored in GCS for downstream use

Architecture Diagram (simple flow)



Sprint Goals



PRs to be reviewed

Caching Fixes

- Saving and accessing certificate
- Failing DB connections workflow



Data Migration

Create schemas for data to be loaded into



Summary Tone Improvement

Conduct EDA on historic summarisations



Redis/Caching Monitoring

Justify value of DB

Summary Tone Improvement

Problem Statement

Conduct exploratory data analysis (EDA) on historic requests to understand links between note frequency, content duplication, time of day, and summary quality.

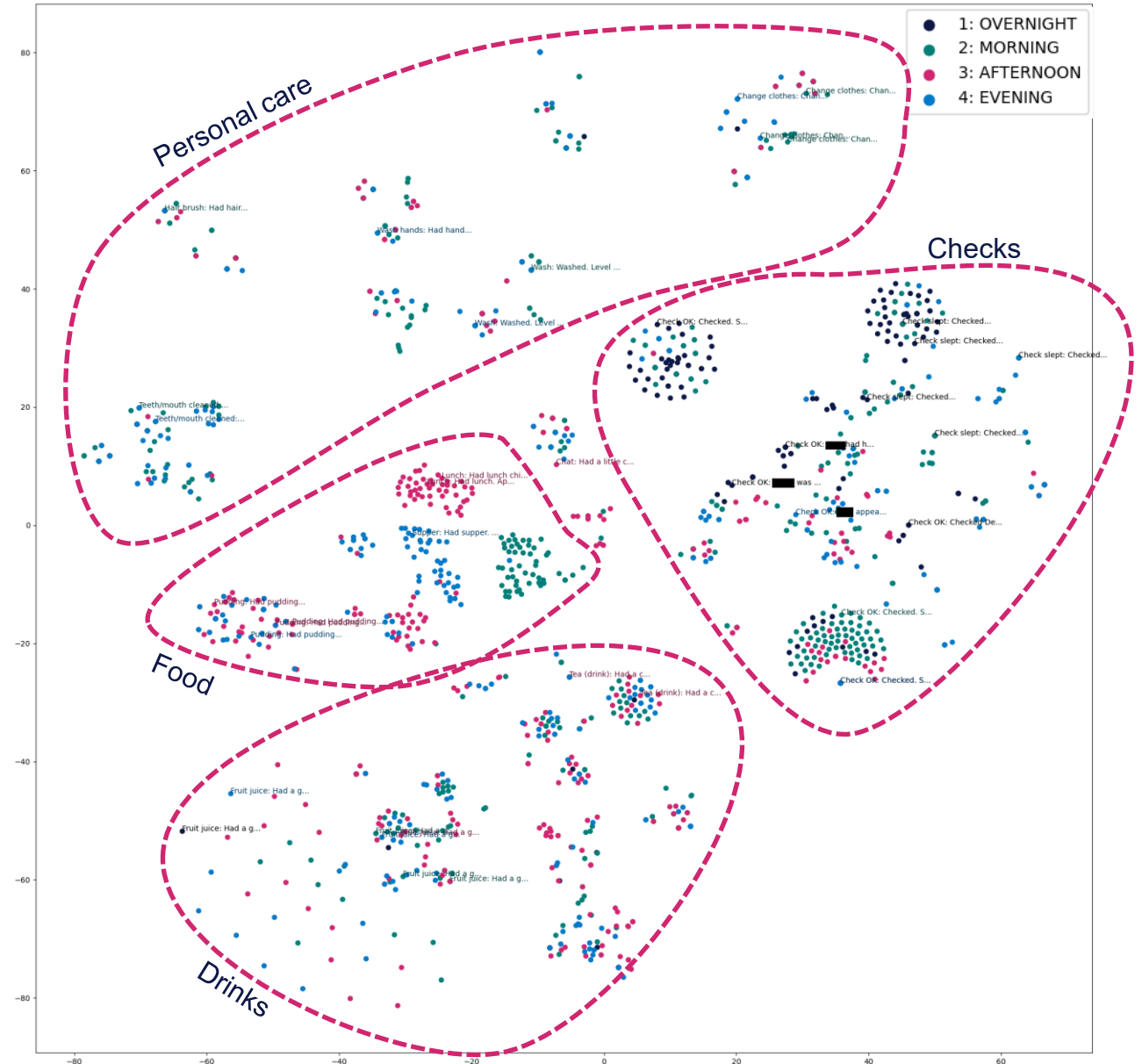
Findings

1. The morning typically has the most notes per request
2. Very small negative trend between number of notes and summary performance.
3. When observing similarity of notes within each request and aggregating scores by time of day, all time periods except overnight has diverse content
4. Content is less diverse/more overlapping per resident.
5. There wasn't a relationship between the amount of similarity in notes in a request and the summary performance.
6. All time periods are performing well with high metric scores. Morning is performing the worst, overnight is performing the best.

Added to the existing [write-up](#) more detailed results.

Next Steps

- Design tone-specific metrics
- Test prompt improvements



Caching Fixes

Problem Statement

Redis caching was brittle in two areas:

1. TLS certificate handling depended on static config / secrets and was harder to maintain across dev and runtime environments
2. Redis failures (auth / connection / runtime issues) could disrupt request flow instead of degrading gracefully

Solution

- Added dynamic Redis CA certificate handling
 - fetches the active CA bundle from the Memorystore API
 - supports override via mounted file or inline PEM
 - reuses a cached temp file and supports refresh/reset paths
- Hardened cache runtime behaviour
 - added retry-on-authentication-error
 - introduced fail-open behaviour by disabling cache after unrecoverable Redis failures
 - updated decorator flow to skip caching cleanly when cache backend is disabled
- Expanded unit tests to cover new functionality

Redis Cache Initialization Logs

```
Datetime 2026-06-16 10:23:00,638 - Level INFO - Session ID unknown: - Successfully parsed all 30 schemas in specified order.
Datetime 2026-06-16 10:23:08,489 - Level INFO - Session ID unknown: - Redis IAM token acquired using gcloud.
Datetime 2026-06-16 10:23:08,798 - Level INFO - Session ID unknown: - Redis CA bundle written to /tmp/redis-ca-ti431fcc.pem
Datetime 2026-06-16 10:23:08,798 - Level INFO - Session ID unknown: - Redis client initialised. host=10.212.102.138 port=6379 tls=True
a_cert=True
Datetime 2026-06-16 10:23:08,798 - Level INFO - Session ID unknown: - Caching enabled: app.common.iam_redis_cache.IAMRedisCache
Datetime 2026-06-16 10:23:08,798 - Level INFO - Session ID unknown: - Cache backend initialised: IAMRedisCache
```

Cache Disable on Redis Failure

```
File "/home/jupyter/.local/lib/python3.10/site-packages/redis/connection.py", line 280, in connect
    raise TimeoutError("Timeout connecting to server")
redis.exceptions.TimeoutError: Timeout connecting to server
Datetime 2026-06-16 13:46:08,159 - Level ERROR - Session ID 326d1c05-7c42-43dc-9299-48f633acc817: - Caching disabled for this app instance. Reason: Redis error during GET
Datetime 2026-06-16 13:46:08,460 - Level WARNING - Session ID 326d1c05-7c42-43dc-9299-48f633acc817: - Could not acquire cache lock after multiple attempts. Proceeding without caching.
```

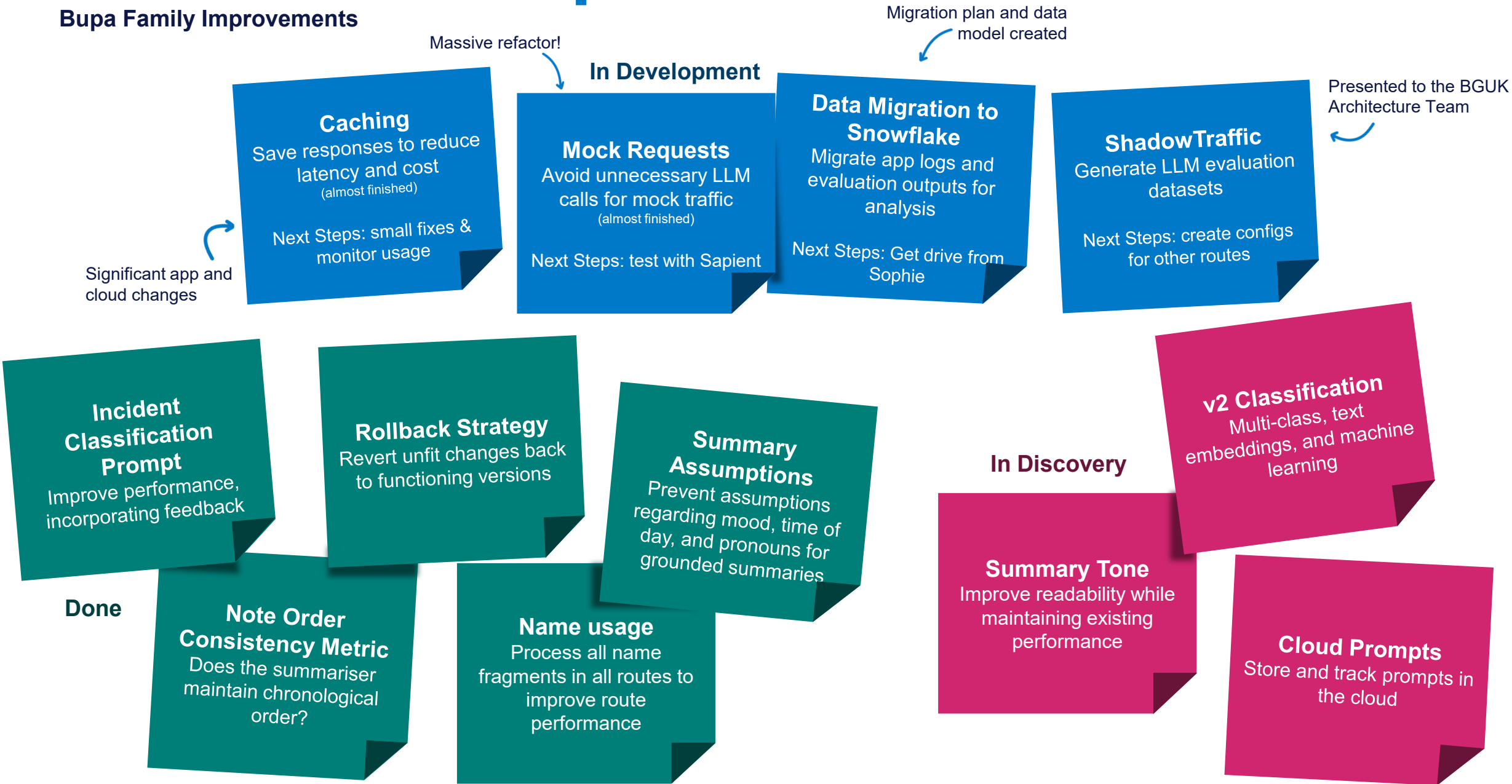


AI & Care Services

Quarterly Planning

What have we been up to?

Bupa Family Improvements



And the feedback tracker?

We're working on it.

Completed	27
In Progress	5
In the Backlog	7
Not Started	10
Not Following	3
Total	52

+8 this last quarter



Benchmarks

Average per day in May

How many evaluation files did we read for this analysis?

Prompt tokens (\$0.30 per 1M tokens)

Candidate and thought tokens (\$2.50 per 1M tokens)

Difference between time received and time response was generated

Route	Requests	# Evals	Input Tokens	Output Tokens	Duration (s)
Summarise	1670	210	1779 (\$0.00053)	920 (\$0.00023)	5.24
Classify	54179	200	770 (\$0.00023)	448 (\$0.00011)	3.18
NPP	62680	200	3996 (\$0.0012)	1270 (\$0.0032)	7.97
Voided	54174	100	3378 (\$0.001)	467 (\$0.00014)	2.93

I was running out of time

Benchmarks

Summarise

Note Inclusion (%)

1

Groundedness
(binary)

0.91

Coherence (1 to 5)

4.7

Fluency (1 to 5)

4.75

Voiced

Name Extraction
Accuracy (binary)

0.99

Voiced Justification
Coherence (1 to 5)

4.85

Classify

Incident Justification
Coherence (1 to 5)

4.83

Note Pre-Processing

Profanity Removal
Evaluation (binary)

1

PII Removal
Evaluation (binary)

0.99

Text Similarity (before
vs after) (%)

0.98

What next?



Bupa Family

Description

Fixes and improvements to create a faster, cheaper, and more accurate AI-service. This includes resolving feedback and developing v2 features.

Status

In Progress (to continue throughout 2026)

What's Next

- Finish in-progress tickets
- Create monitoring reports
- Onboard Sapient team members to enable them to complete fixes.

Clarity

High



Dependency Assessments

Description

Completing dependency assessments takes 3-4 hours per resident. Can parts of this be automated to save time?

Status

New (starting July)

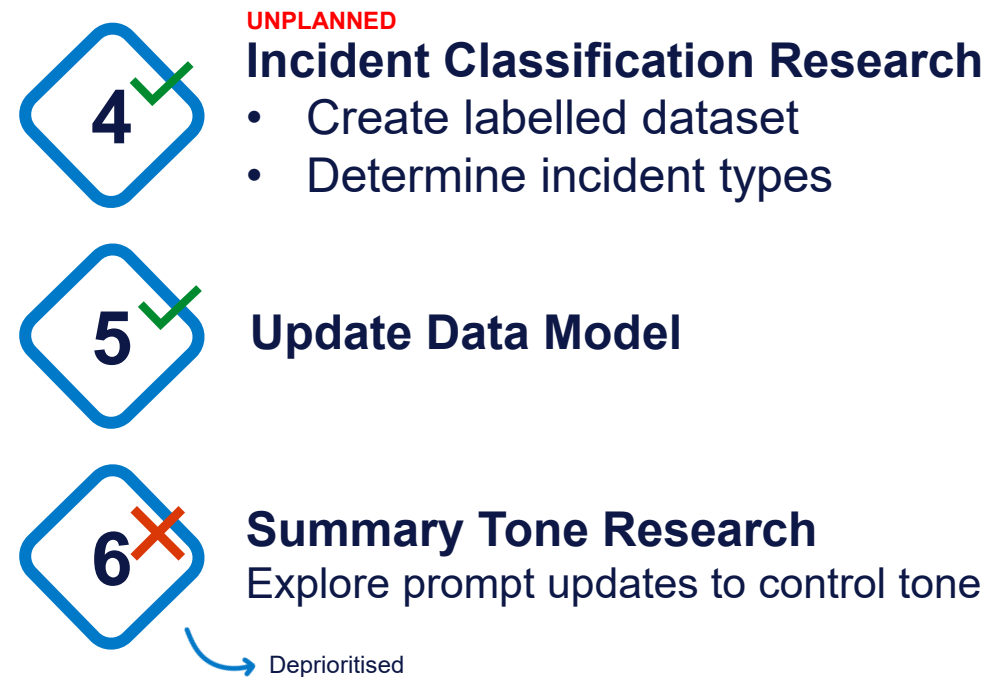
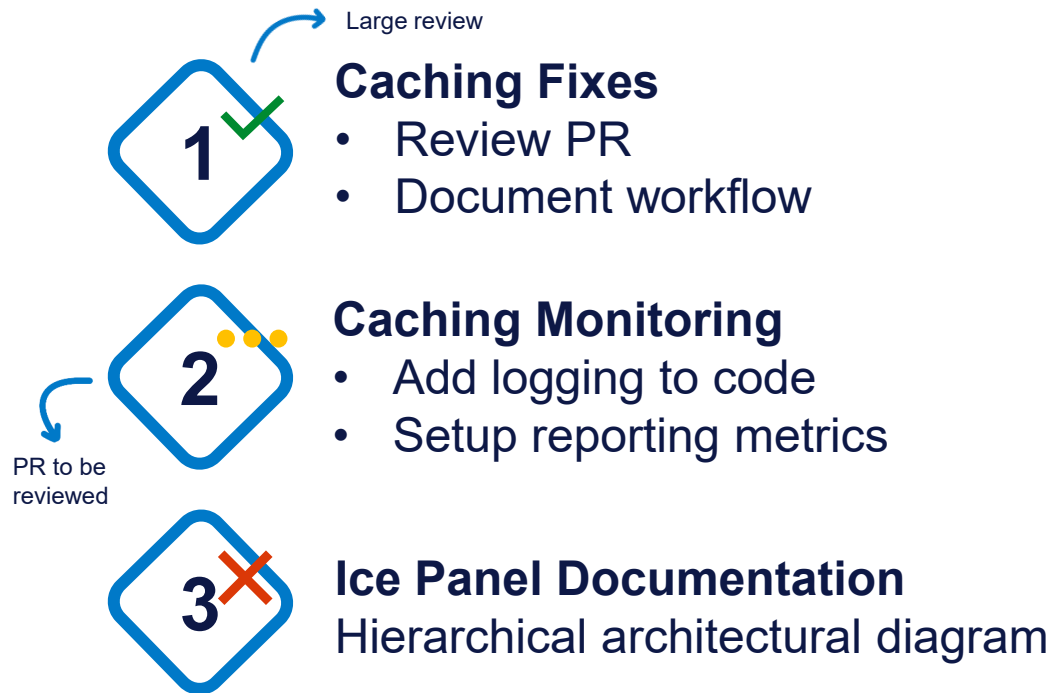
What's Next

Funding confirmed. Conduct feasibility study, meeting with care home staff to witness how 'Resident of the Day' is currently completed.

Clarity

Low

Sprint Goals



Caching PR Review

Problem Statement

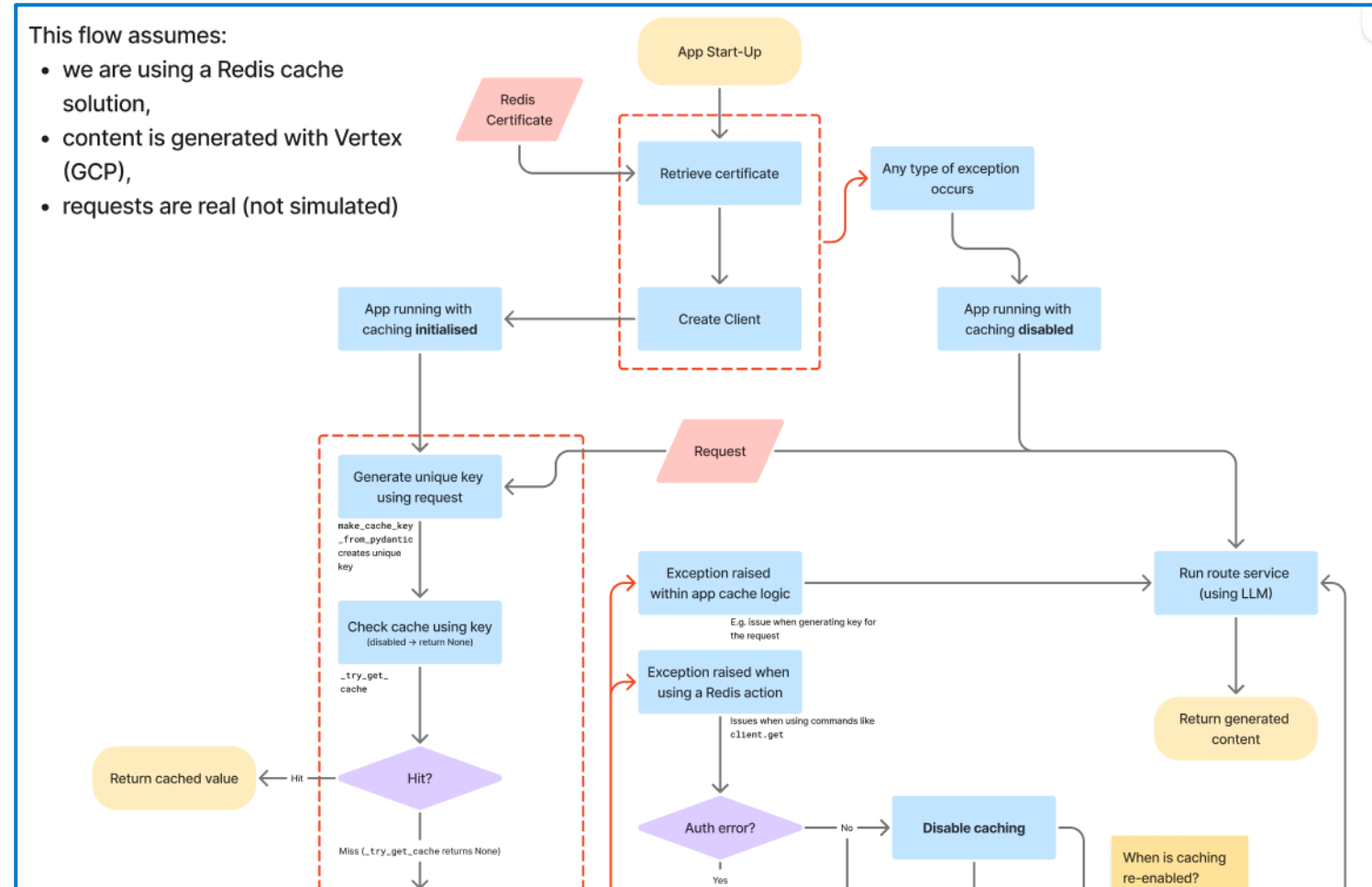
- Need to review the most recent caching changes
- Changes are significant, impacting app workflow
- Want to visualise expected and actual behaviour

Solution

Create a diagram that outlines the current workflow of the caching system, identifying dependencies, decisions, how the caching is disabled, and consequences of disabling.

Benefits

- Agree on the expected and actual behaviour as a team
- Identify potential flaws, bugs, or inconsistencies
- Share the logic externally with teams wanting to implement caching in their systems



Caching Monitoring – Adding structured logging

Problem Statement

- The caching layer had operational logs, but it was difficult to measure cache effectiveness, failure patterns, and lock contention in a consistent way.
- This limited our ability to understand cache hits vs misses, Redis failures, token savings, and whether locking behaviour was helping or harming request performance.

Solution

- Added a centralised cache observability helper to emit structured cache events in a consistent JSON format.
- Instrumented the caching flow to log:
- cache hit / miss / bypass outcomes
- request duration by cache outcome
- exact token usage and savings from response metadata
- lock wait start / resolve / fallback events
- Redis/cache errors by operation and error type
- cache backend disable events
- Redis client lifecycle events (initialisation, reset, token failures)
- application startup / shutdown and cache initialisation events

Result

- We now have a single, structured source of truth for cache behaviour across the service.
- Cache events are consistent, machine-readable, and ready to be converted into log-based metrics, dashboards, and alerts.
- This improves operational visibility, incident debugging, and future scalability of the caching platform.

Incident Classification

Data Labelling

Problem Statement

- Want to treat individual incident categories differently in the front end (e.g. display confusion for dementia residents)
- Need to determine incident types but need labelled data to do this.

Solution

Create a lightweight but representative dataset of incidents and non-incidents so that we can learn about incident patterns.

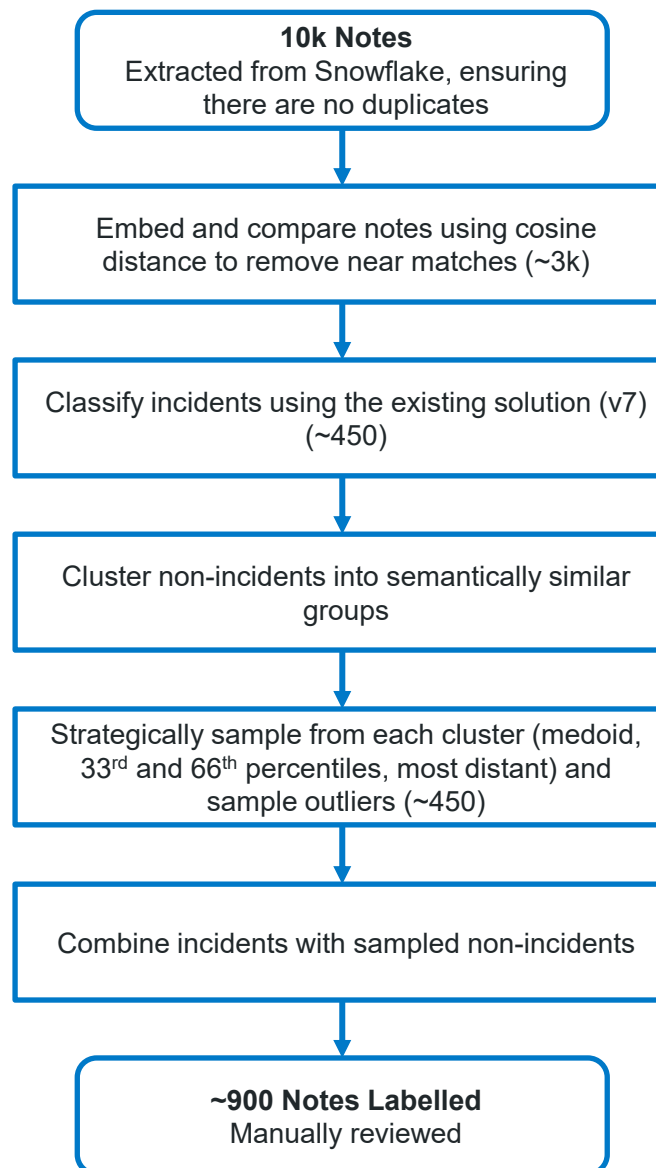
Outcome

- Incident categories and criteria defined and shared with stakeholders
- Guided classification methodology

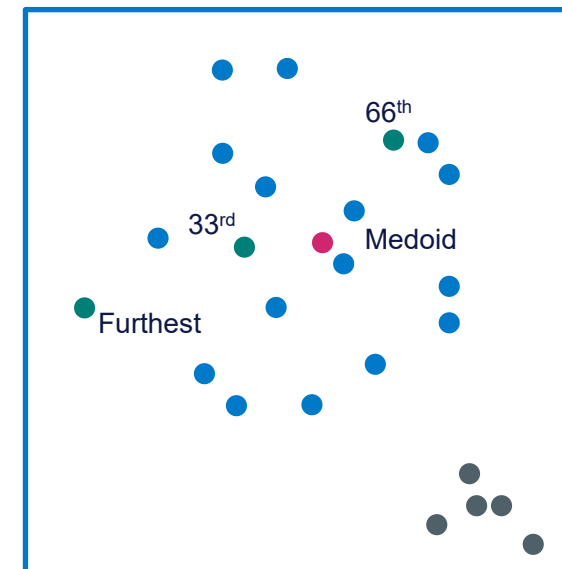
Next Steps

- Review and approve categories
- Write new incident triage prompt

Creating a representative dataset



Cluster of embedded notes visualised in 2D



Categories

- Mobility & Falls
- Nutrition & Swallowing
- Medication
- Skin Integrity & Wounds
- Continence
- Behaviour & Emotional Wellbeing
- Physical Observations/Reports
- Personal Care
- Personal Safety
- Safeguarding

Evaluation Report

User Story

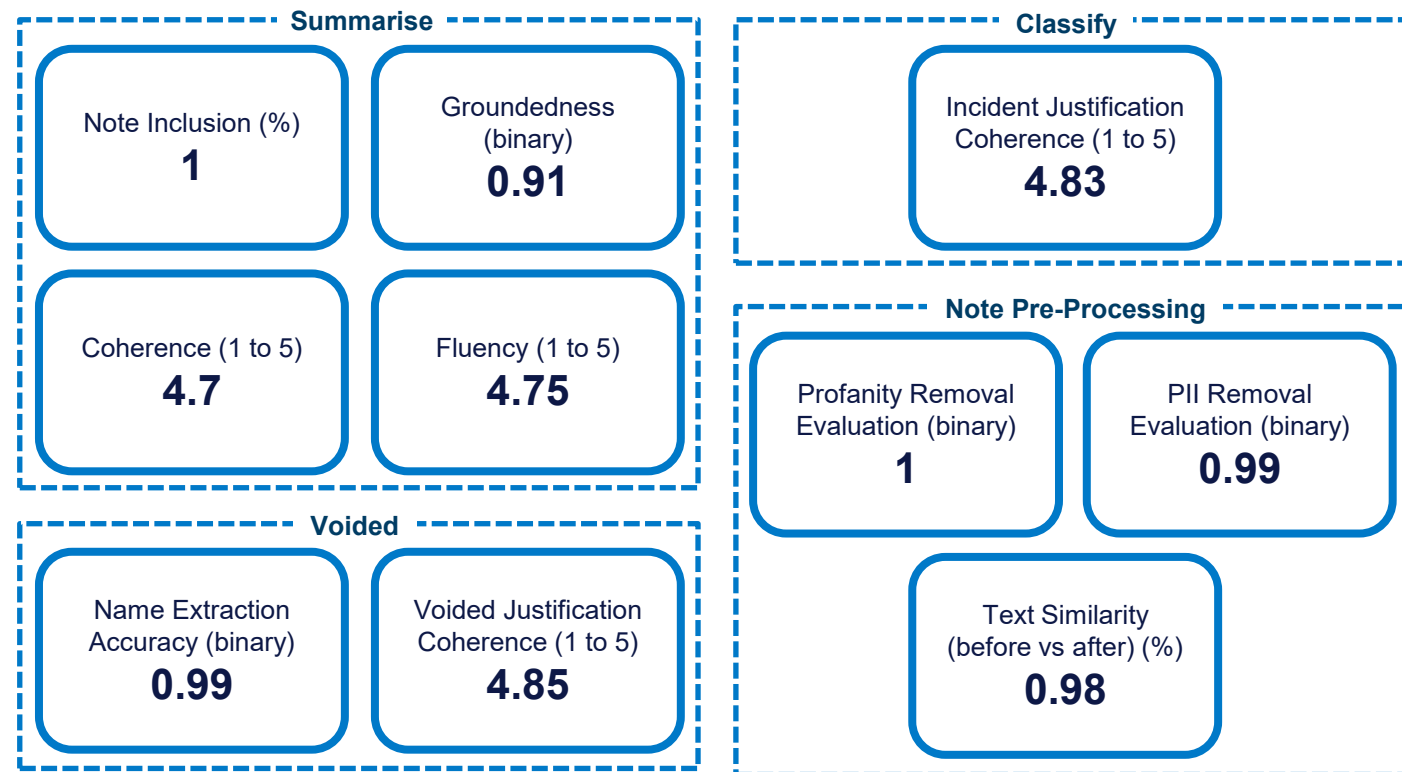
As an exec/directory level, I need a high-level view of AI performance and adoption so that I can be confident that AI is delivering value, identify significant risks, and understand when investment or intervention is required.

Description

- Cards summarising metric scores for each service during a select period
- Quick to digest

Features

- Traffic light colours
- Change in scores compared to last [period]



Doesn't need to look like this, but these are the metrics. Number of metrics calculated may increase.

Evaluation Report

User Story

- As a product owner, I want to monitor AI performance over time so that I can detect regression, understand the impact of changes, and prioritise improvements.
- I need to know when AI performance falls below an acceptable threshold so that I can investigate potential issues before they affect app users or team members.

Description

- Line graphs showing stats over time
- Vertical lines to show when prompt version was updated
- Horizontal lines/traffic light colours to show metric thresholds (static)

Features

- Filter by care home and timeframe (sliders)
- Select service and metric (buttons)
- Filter by prompt version, model (sliders – dependent on service and metric)

Response stats over time: Is what we're returning changing?

Stacked area chart, 1-2 per service. Top value is number of requests received at a given time. Areas are:

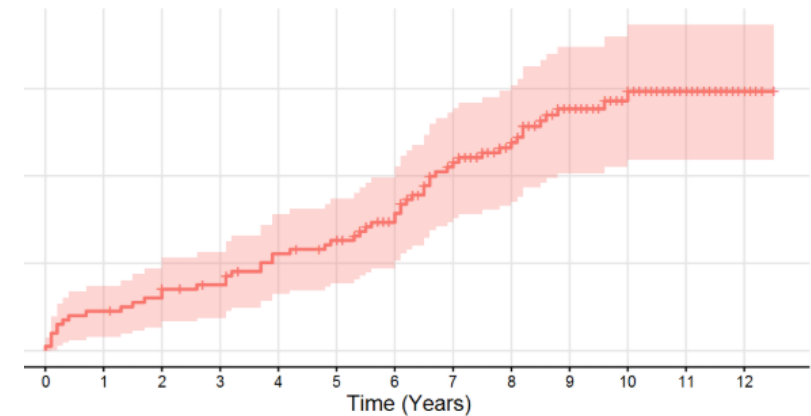
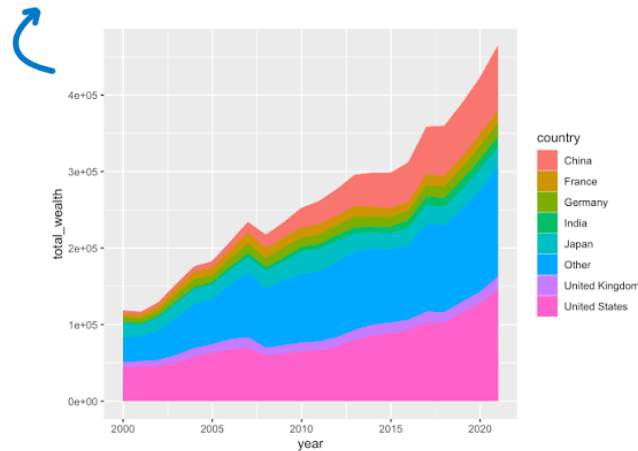
Summarise: Number of notes received (binned)

Classify: Proportion of notes classified True/False

Voided: Proportion of decisions returned (correct, incorrect, no name, inconclusive)

Note Pre-Processing (two), toggleable by buttons, tool tip (% and count):

- Proportion with PII redactions
- Proportion with profanity redactions



Metric scores over time: Is the response quality changing?

Line chart with shaded area representing range (min and max scores at a given time). Line is mean. One graph per metric (must have metric selected).

Evaluation Report

User Story

- As an AI engineer, I want to identify low-quality AI outputs and understand why they performed poorly so that I can improve prompts, evaluation criteria, or underlying system behaviour.
- As a data scientist, I want to understand which evaluation metrics contribute most to low overall scores so that I can improve evaluation methodology and ensure metrics remain meaningful.

Description

- Table of worst-performing outputs to be manually analysed

Features

- Show number of evaluations calculated
- Service specific health

Datetime	OG Input	AI Output	Metric Scores	Evaluator Reasoning	Prompt and model (Prod)	Prompt and model (Eval)
When did we service this request	What info did we receive (this may be hard for summarise as this input is long)	What did the AI generate? This could include more than one field (e.g. classify returns class and justification)	What were the metric scores (over several columns)	Evaluator will also return reason for the score – what about the output it did/didn't like	Which prompt and model did the production service use to generate the output?	Which prompt and model did the evaluator use to evaluate the production service's output?
25/06/2026	"Witness a fall"	False – no mention of incident	Classification justification: 1	This is objectively wrong	V7, Gemini-2.5-Flash	V3, Gemini-2.5-Pro

Sprint Goals



Caching Monitoring

- Resolve PR comments
- Create dashboard



Create Plan for Integration Test Caching

Caching working and failing as expected in GCP



Ice Panel Documentation

Hierarchical architectural diagram



Explore Incident Triaging

Identify note type before classifying

Incident Classification

Note Triage

Problem Statement

Current incident prompt includes criteria for **all** types of incidents, making it long, complex, and missing the intricacies of each incident type.

Solution

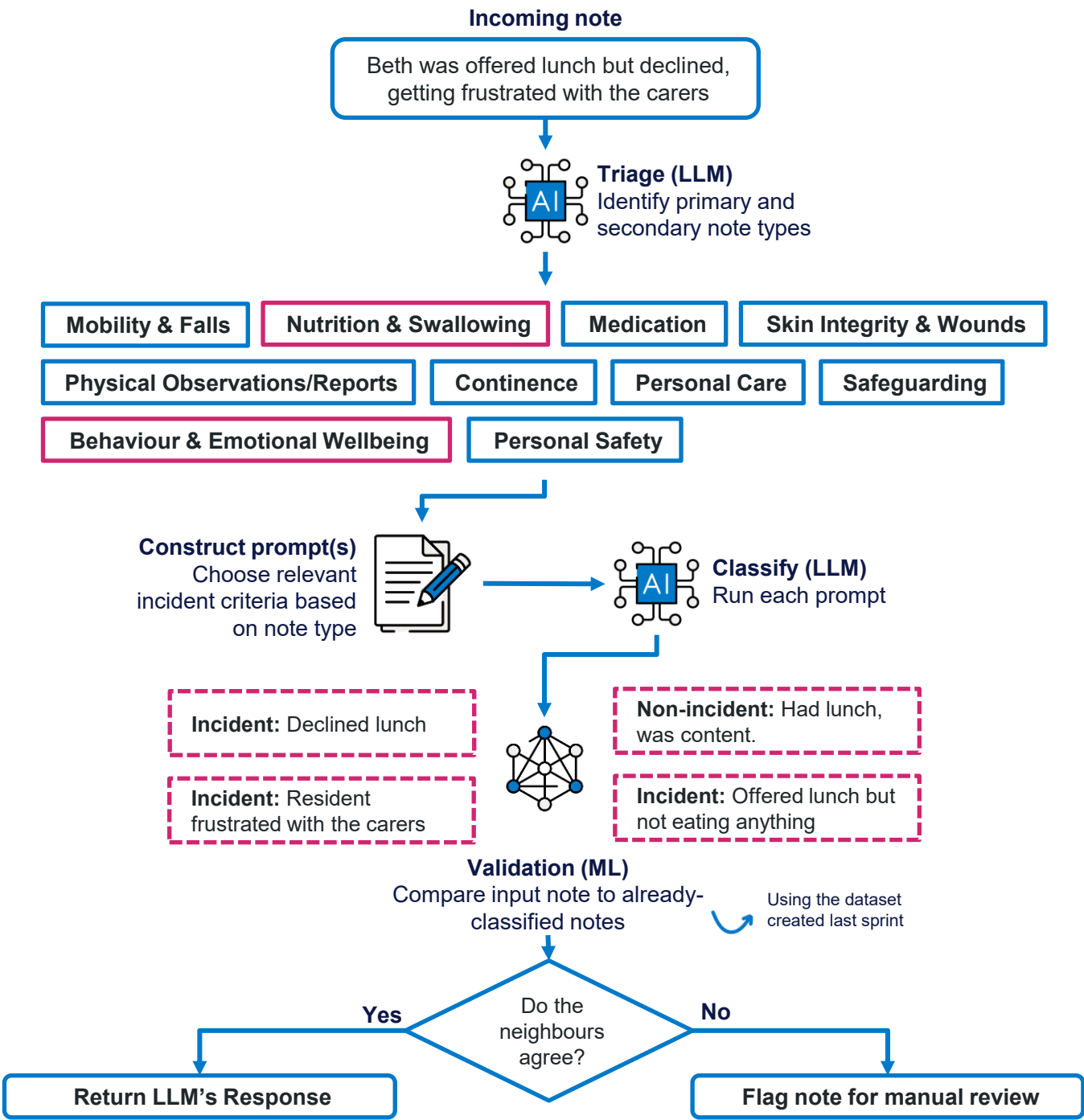
- Identify note type **before** classifying (triage)
- Define an incident criteria lookup with positive and negative measures for each incident type
- E.g. Mobility -> positive: declined repositioning; confirmed fall. negative: the placement and checking of mats; does not matter how the resident was repositioned.
- Construct prompt(s) using prompt template, triaged note type, and incident criteria lookup

Result

- New triage prompt experimented with in Jupyter Notebooks

Next Steps

- Tailor lookup with clinicians/domain experts



Caching Monitoring Metrics and Dashboard

Problem Statement

We had added structured cache events, but there was still no simple operational view to track cache behaviour over time.

Solution

- Converted the structured cache events into GCP log-based metrics
- Built a dedicated cache monitoring dashboard to track:
 - cache hit / miss / bypass trends
 - request duration by cache outcome and phase
 - token usage and token savings
 - lock wait events and wait duration
 - Redis / cache errors by type
 - Redis client lifecycle and cache backend disable events

Result

- We now have a single operational view of cache behaviour across the service.
- Cache effectiveness, Redis health, and lock behaviour can be monitored without manually reading logs.

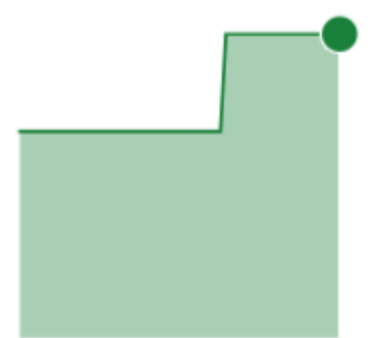
Total tokens saved (30d)

74.468k



Total tokens used (30d)

28.723k



Spike: Integration Testing

Problem Statement

- We had good unit test coverage, but no repeatable way to validate the real API deployed in dev.
- We also needed a plan to test cache and Redis behaviour in a realistic environment.
- This made it difficult to test repeated requests, concurrent requests, and locking behaviour consistently.

Solution

- Spiked an integration testing approach for the deployed dev API.
- Proposed:
 - pytest for correctness-focused API integration tests
 - Locust for concurrency, repeated-request and locking scenarios
- Planned to use ShadowTraffic for safe, repeatable synthetic test data.

Result

- We now have a clear approach for testing the real API in dev.
- The plan covers both current API validation and future cache / Redis scenarios.
- This gives a practical path to test behaviour more consistently outside local unit tests.

Test scenarios

•Valid request

- Send a request to the dev API
- Assert success response and expected response structure

•Invalid request

- Send an invalid or malformed request
- Assert expected validation / error response

•Same request twice

- Send the same request twice sequentially
- Assert first request = **MISS**, second request = **HIT**

•Unique request

- Send a new request that has not been seen before
- Assert response = **MISS**

•Same request in parallel

- Send the same request at the same time from multiple users
- Assert successful responses and expected cache / locking behaviour

•Different requests in parallel

- Send multiple unique requests at the same time
- Assert successful handling under concurrent traffic

•Mixed traffic

- Run a mix of repeated and unique requests
- Validate cache reuse and normal miss behaviour together

•Authentication failure

- Request without valid auth
- Confirm expected rejection

•Burst traffic

- Sudden spike of repeated requests
- Useful for observing lock contention and cache behaviour under load

•Repeated same request over time

- Send the same request many times over a short period
- Useful to confirm hit ratio improves after initial miss

•Fallback / resilience scenario

- If possible, test behaviour when cache or Redis is unavailable
- Useful for confidence in degraded-mode behaviour

Ice Panel Documentation

Task

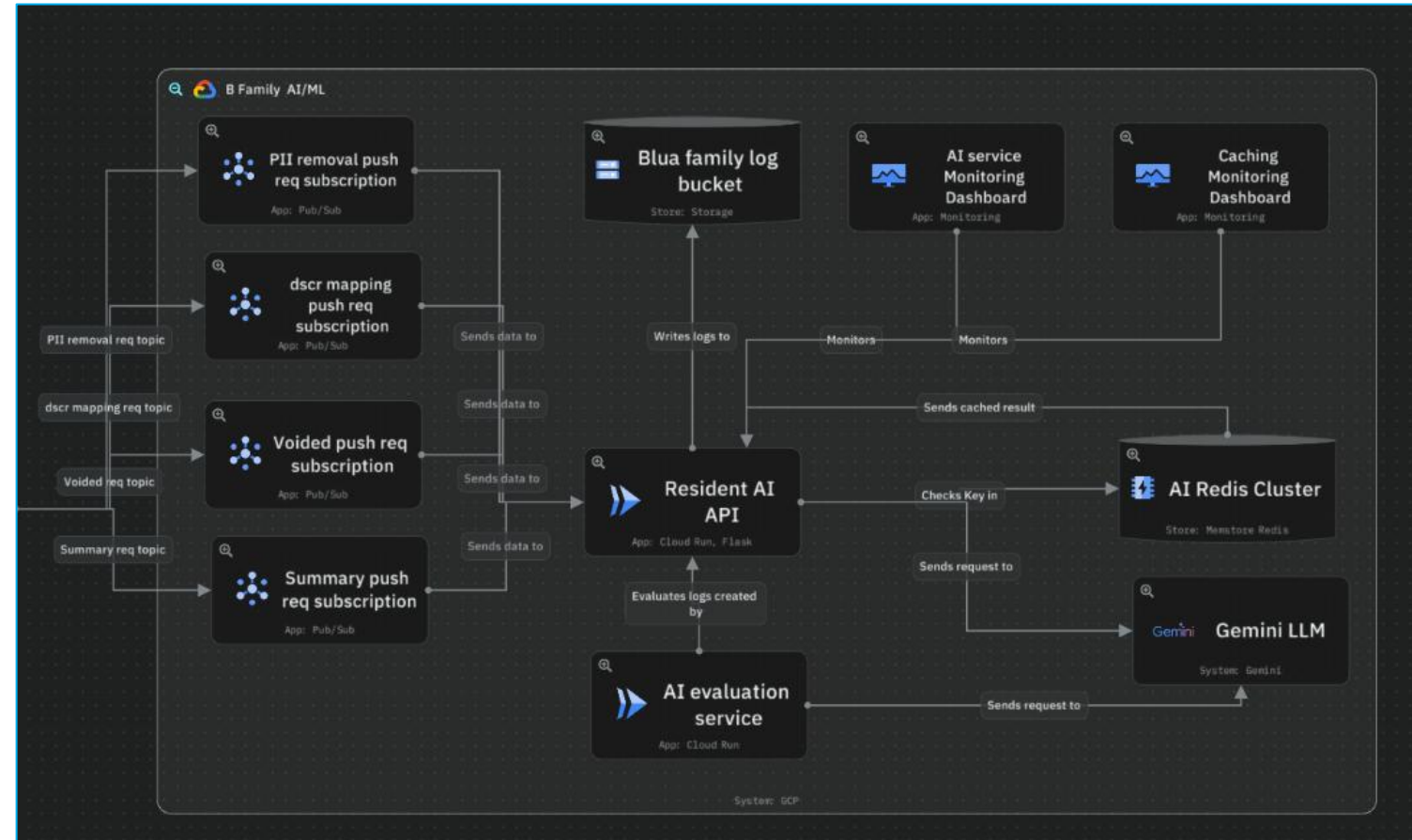
- Asked to produce the ICE panel architecture documentation for the AI service running in GCP.
- The aim was to create a clear high-level view of the service, its main components, and how requests flow through it.

What the ICE panel is used for

- The ICE panel is used to document the service architecture in a clear, structured way.
- It shows:
- the main system building blocks
- the relationships between them
- how requests and data move through the service
- The aim is to make the architecture easier to understand, review and maintain.

What C4 means

- C4 is a way of structuring architecture diagrams into levels of detail.
- The main levels are:
- Context – what the system interacts with
- Container – the main applications / services / data stores
- Component – the key internal parts inside a container
- Code – the implementation level



Sprint Goals



Integration test caching

Ensure caching is working as expected



Deploy caching to prod

Let's start saving some money 



Deploy all changes to prod

All recent fixes that have been tested in Dev and Test can be promoted

Early impact of Redis Caching

**Approx. cost avoided
First 24 hours after rollout:**

Thought tokens:
 $865.5\text{M} \times \$2.50 / 1\text{M} = \259

Prompt tokens:
 $234.5\text{M} \times \$0.30 / 1\text{M} = \586

Total:
 $\approx \$845 \rightarrow \text{£}629!!!$

Based on Gemini 2.5 Flash token pricing

212.5K

Total requests



178.7K

Cache Hits



25.4K

Cache misses



87%

Cache hit ratio



1.1B

Tokens saved



169.6M

Tokens used



Sprint Goals



- Subject to PR
- 1** ✓ **Response Caching Attributes**
Return information with a Pub/Sub response with caching usage info (for testing)
 - 2** ✓ **Caching Usage in JSONs**
Record caching usage in JSON logs for long term analysis
 - 3** ✓ **Cache Original Request ID**
Save the request ID with the AI-generated content for traceability
 - 4** ●●● **Integration test caching (continued)**
Use response attributes to finish testing
 - 5** ✗ **PII Redaction & Validation Fix**
Resolve ambiguities in PII redaction

Additional Caching Monitoring

Problem Statement

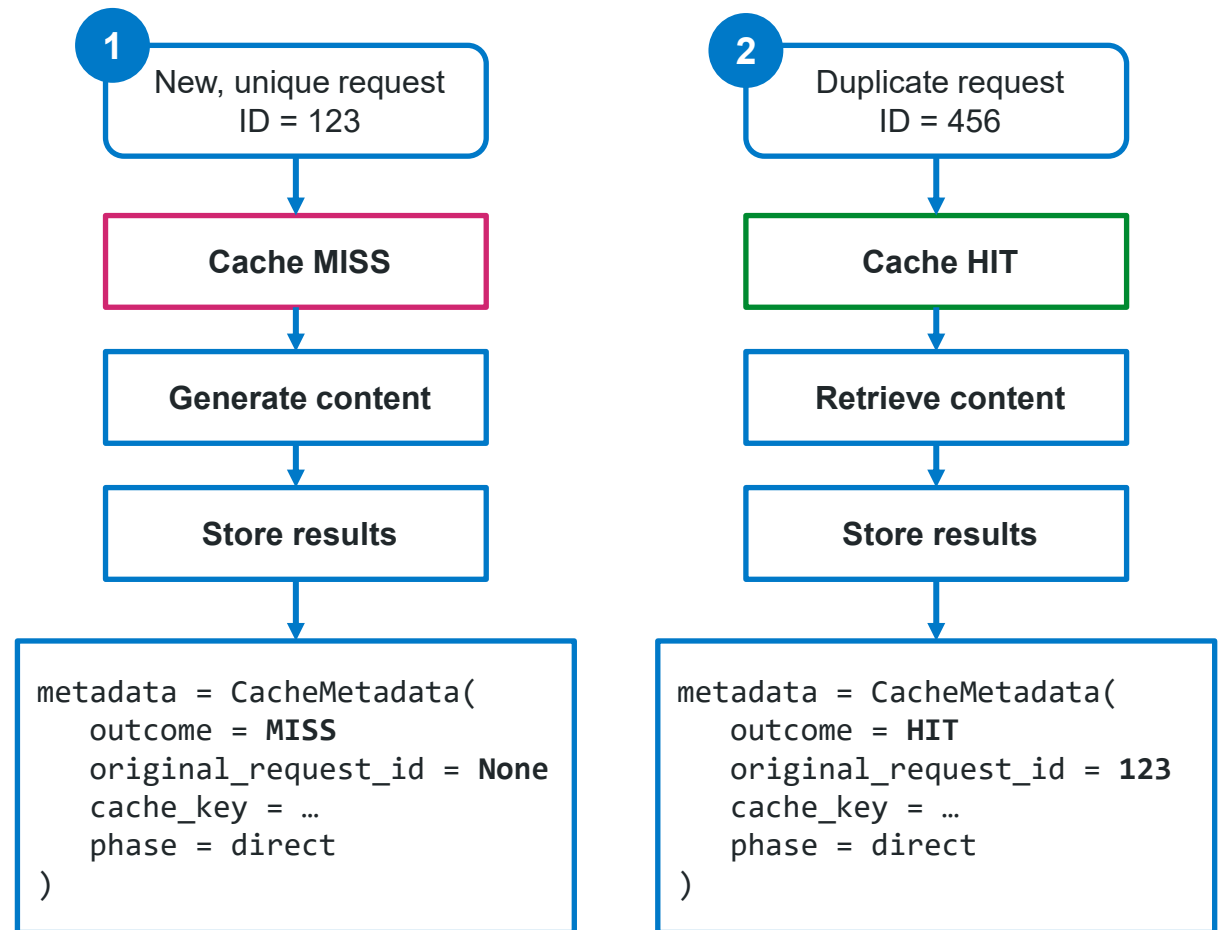
We want to analyse long-term caching usage, building more custom reports, but info on usage is currently only in the Cloud Run logs.

Solution

- Store caching usage metadata within the JSON logs saved in the GCP bucket
- This data will be migrated to Snowflake for custom reporting.
- For cache misses, store the original request ID with the AI-generated content for auditability

Next Steps

- PR and review



Integration Testing: Response headers

Problem Statement

We needed confidence that deployed API caching behaves correctly (miss, hit, concurrency lock behaviour, and fail-open paths).

Approach

We run integration tests against deployed Cloud Run using Pub/Sub-style requests with Avro payloads, then validate cache behaviour through response metadata and summary logs.

What we validate

- First-time unique requests are cache misses.
- Repeated identical requests become cache hits.
- Concurrent duplicates collapse through lock/wait path.
- Concurrent unique requests do not cross-contaminate cache.
- Invalid envelopes fail with expected validation errors.
- Redis-failure scenario fails open (when run against Redis-broken revision).

Outcome

Higher confidence in deployed caching correctness, with clear diagnostics in passing test logs.

Sprint Goals



Investigate caching other routes

- Understand prevalence of duplicate requests
- Create strategy for customisation



PR Cache JSON logging

Record caching usage in JSON logs for long term analysis



PR Integration Tests

Dependent on cache JSON logging PR



Investigate restarting cache

Use response attributes to finish testing



PII Redaction & Validation Fix

Resolve ambiguities in PII redaction

Disrupted by
illness



Duplicate Classification Requests

Analysing all classification requests on 1st June 2026 (observation ID removed from the incoming data)

Metric	Strategy	Value
Total number of requests	<code>len(df)</code>	50,678
Number of unique requests	<code>request_count = df.groupby(["input_data_text"]).observation_id.count()</code> <code>len(request_count)</code>	14,704
Inputs only requested once	<code>len(request_count[request_count.count == 1])</code>	8970
Number of redundant requests	<code>more_one_requests = request_count[request_count.count > 1]</code> <code>redundant_requests = more_one_requests.request_count.sum() - len(more_one_requests)</code>	35974
% Avoidable	<code>redundant_requests/len(df)</code>	71%

Duplicate Voiced Requests

Analysing all voided requests on 1st June 2026 (observation ID removed from the incoming data)

Metric	Strategy	Value
Total number of requests	<code>len(df)</code>	49,895
Number of unique requests	<code>request_count = df.groupby(["input_data_text"]).observation_id.count()</code> <code>len(request_count)</code>	14,688
Inputs only requested once	<code>len(request_count[request_count.count == 1])</code>	9070
Number of redundant requests	<code>more_one_requests = request_count[request_count.count > 1]</code> <code>redundant_requests = more_one_requests.request_count.sum() - len(more_one_requests)</code>	35207
% Avoidable	<code>redundant_requests/len(df)</code>	71%

Most common requests

1537 Class. Requests

```
{
  "fragment": "Checked",
  "correction": null,
  "actionIconName": "Check OK",
  "isAccident": false,
  "isIncident": false,
  "isSignificant": false,
  "isReminder": false,
  "enumSliders": [
    {
      "slideName": "Happiness",
      "sliderInfo": [
        {
          "sliderName": "OK",
          "overridenMeaning": "was content"
        }
      ]
    }
  ],
  {
    "slideName": "Sleeping state",
    "sliderInfo": [
      {
        "sliderName": "Awake",
        "overridenMeaning": "was awake"
      }
    ]
  }
],
  "fluidSliders": [],
  "levelOfHelp": "",
  "happiness": "was content",
  "result": "Done"
}
```

712 Class. Requests

```
{
  "fragment": "Adjusted the curtains to suit the time of day",
  "correction": null,
  "actionIconName": "Adjust curtains",
  "isAccident": false,
  "isIncident": false,
  "isSignificant": false,
  "isReminder": false,
  "enumSliders": [],
  "fluidSliders": [],
  "levelOfHelp": "",
  "happiness": "",
  "result": "Done"
}
```

597 Class. Requests

```
{
  "fragment": "Checked the room to ensure everything is OK",
  "correction": null,
  "actionIconName": "Check room",
  "isAccident": false,
  "isIncident": false,
  "isSignificant": false,
  "isReminder": false,
  "enumSliders": [],
  "fluidSliders": [],
  "levelOfHelp": "",
  "happiness": "",
  "result": "Done"
}
```

Expanded Caching Strategy

Problem Statement

Cache Voided and Classify outputs

- Alternative field in the input to check (not `observation` but `fullCareNoteInfo`)
- Need to remove observation ID from the input
- Conditional cache Voided – only those with “NO_NAME” decision. We don’t want one resident’s “INCORRECT” decision, to impact another resident’s “CORRECT” decision.

Solution

1. Create route-specific cleaning functions. For Classify and Voided, this will remove the `observation\_id` field from the `fullCareNoteInfo` object.
2. Implement a switch that will choose the cleaning function and implement it once the relevant field has been extracted from the request input
3. Add a conditional function that takes the generated output and route name to check whether the content should be saved

Next Steps

- Create a new branch with these changes
- Test and review

New helper functions

1

```
def fcni_clean(field_value: FullCareNoteInfo) -> dict:
    """
    Remove observationId from the FullCareNoteInfo model to ensure that the cache key is not affected by this unique identifier.
    """
    cleaned_dict = field_value.model_dump(by_alias=False) # Convert Pydantic model to a dictionary
    cleaned_dict.pop("observation_id", None) # Remove observationId if present
    return cleaned_dict

def route_specific_clean(route_name: str) -> callable:
    if route_name == "classify":
        return fcni_clean
    elif route_name == "notepreprocessing":
        return lambda x: x # No cleaning needed for note preprocessing
    else:
        raise ValueError(f"Unknown route name: {route_name}. Please provide a valid route name for cleaning.")
```

Within _make_cache_key_from_pydantic

2

```
# Ensure the field has a value to hash.
if not field_value:
    raise ValueError(
        f"The field '{field_name}' in model '{type}'
    )

# Clean the field value based on the route name.
clean_function = route_specific_clean(route_name)
field_value = clean_function(field_value)

# Combine the static version string with the dynamic
# This ensures that a change in logic (version) or
combined_key_string = _create_cache_string(
```

Demo

```
key_generator = _make_cache_key_from_pydantic(
    route_name="classify",
    pydantic_models=[ClassifyEvent, ClassifyRequestModel],
    field_name="full_care_note_info",
    prompt_versions=json.dumps({"prompt": "test"}),
    models_used=json.dumps({"model": "test"})
)

classify_key = key_generator(validated_input)

print(classify_key) # with hashing removed

classify::unknown::{"prompt": "test"}::{"model": "test"}::{'fragment': 'Helped out of bed',
```

Within api_cached

3

```
def conditional_cache(route_name: str, service_result):
    if route_name == "voided":
        # check LLM decision
        if service_result.data.correct_resident == "NO_NAME":
            return True
        else:
            return False
    else:
        return True

service_result = _execute_and_log(cache_summary, service_func, validated_input, *args, **kwargs)
if conditional_cache(route_name, service_result):
    cache_success = _cache_result(g.request_id, service_result, delimiter, cache_key, timeout)

return service_result
```

Pseudo code

Redaction Improvements

Problem Statement

Many note pre-processing requests were returning errors with code 10003: failed redaction. These requests were repeatedly retried, failing in some instances, more than 50 times. This is unnecessarily expensive, both financially and in terms of the number of requests the app is concurrently handling.

Causes

- “Seen nurse” notes. They frequently mention a role followed by a name without punctuation, creating ambiguity for the model.
- Redacted role titles (not names). Removing “doctor” or “nurse” unnecessarily.
- Ambiguous locations. Sometimes “care home” was being removed.
- Typos. The model was not recognising the resident’s name when it was misspelt.

Solution

- Prompt fix to include clearer instructions
- Solution validated on the failed requests, dummy dataset, and 100 real requests.

Redactor

PII Categories to Redact

You must find and replace the following PII categories with placeholders:

- **PERSON_NAME**: Full names, first names, or last names of anyone who IS NOT the resident (e.g., doctors, nurses, family members). **(NEW)** Do NOT redact role titles, only their names (e.g., "Dr. Smith" => redact. "Doctor" => no name therefore, don't redact).
- **PHONE_NUMBER**: All phone number formats.
- **ADDRESS**: Full or partial personal addresses. **(NEW)** Do not redact mentions of place names such as care homes, hospitals, or other public places (e.g. "hospital")
- **EMAIL**: Email addresses.
- **DATE**: Specific dates (e.g., dates of birth, appointments, admissions). This DOES NOT include relative time frames (e.g. "yesterday")
- **ID_NUMBER**: Unique identifiers (e.g., NHS numbers, patient IDs).
- **(NEW) ROOM_NUMBER**: Any explicit mentions of specific room numbers in the care home. This does not include communal spaces such as "lounge" or "dining room", or generic room names e.g. "bedroom".

Final Rules

1. **(NEW) RESIDENT NAME TAKES PRECEDENCE**: If a person's name matches the resident's name (or is an obvious typo), treat that person as the resident. Do not redact this name. This rule takes precedence over contextual interpretations such as titles, occupations, roles, or grammatical ambiguity. Only treat the matching name as a different person when the text explicitly identifies a different individual with that exact name (e.g. "Beth, who is not the resident, came to visit the resident Beth" => two "Beth"s clearly identifiable).
2. **(NEW) RESIDENT NAME TYPOGRAPHICAL VARIATIONS**: If a name is an obvious typographical variation of the resident's name (e.g., typo "Margret" or nickname "Maggie" for "Margaret"), treat it as the resident's name and do not redact it.
3. **PRESERVE THE ORIGINAL TEXT**: Do not alter, rephrase, or correct any part of the text that is not a PII entity being replaced. Preserve all original grammar, spelling, and spacing.
4. **BE CONTEXT-AWARE**: Do not redact common words that might also be names (e.g., "Brown" in "the coat was brown" is not PII, but "Dr. Brown" is).

Validator

(NEW) Your role is **not** to reperform the redaction, re-intepret the redactor's rules, or independently determine how ambiguous people or entities should be classified. Instead, **audit whether the redactor's output is consistent with the rules it was instructed to follow.**

Additional code changes

To support the redaction fixes, some improvements were made to the app.

```
cleaned_first_name = pre_process_name(first_name)
cleaned_preferred_name = pre_process_name(preferred_name)
if cleaned_first_name == cleaned_preferred_name:
    logger.info(f"Preferred and first names are the same after pre-processing. Setting preferred name to None.")
    cleaned_preferred_name = None

return {
    vr.first_name_param: cleaned_first_name,
    vr.last_name_param: last_name,
    vr.preferred_name_param: cleaned_preferred_name,
    vr.note_param: notes
}
```

No preferred name: As with the voided route, if the note pre-processing endpoint is provided with the same first and preferred name for a resident, then no preferred name is provided to the model.

```
# 0. Check prompt data
for prompt in [
    vr.profanity_removal_prompt_name,
    vr.profanity_removal_validation_prompt_name,
    vr.profanity_removal_retry_prompt_name,
    vr.pii_removal_prompt_name,
    vr.pii_removal_validation_prompt_name,
    vr.pii_removal_retry_prompt_name
]:
    if prompt not in prompt_map:
        # use default version if not provided in prompt_map
        logger.warning(f"Prompt {prompt} not found in provided prompt_map,")
        prompt_map[prompt] = pv.NPP_PROMPT_MAP[prompt]
```

```
Datetime 2026-08-17 08:10:14,775 - Level WARNING - Session ID unknown: - Prompt profanity_removal not found in provided prompt_map, using default version from NPP_PROMPT_MAP
Datetime 2026-08-17 08:10:14,776 - Level WARNING - Session ID unknown: - Prompt profanity_removal_validation not found in provided prompt_map, using default version from NPP_PROMPT_MAP
```

```
from app.common.clients.vertex_text_generation import VertexTextGenerator
import variables as vr
from app.v1.services import note_preprocessing_service
from app.v1.model_versions import MODEL_MAP, VERTEX_PROVIDER_IDENTIFIER
from app.v1.schemas import NotePreprocessingRequestModel

def notebook_generator(task):
    return VertexTextGenerator(model_name=MODEL_MAP[VERTEX_PROVIDER_IDENTIFIER][task])

test_data = NotePreprocessingRequestModel(
    observation={"123": "Seen nurse beth was settled throughout the night"},
    first_name="Bethany",
    last_name="Ryan",
    preferred_name="Beth"
)

testing_prompt_versions = {
    vr.pii_removal_prompt_name: 5,
    vr.pii_removal_validation_prompt_name: 4
}

service_result = note_preprocessing_service(
    validated_data=test_data,
    get_generator=notebook_generator,
    prompt_map=testing_prompt_versions
)

print(json.dumps(service_result.model_dump(), indent=2))
```

```
{
  "data": {
    "123": {
      "redacted_text": "Seen nurse beth was settled throughout the night",
      "profane": false,
      "redactions": []
    }
  },
  "metadata": {
    "model": "\\\"profanity_removal\\\": \\\"gemini-2.5-flash\\\", \\\"profanity_removal_validation\\\": \\\"gemini-2.5-flash\\\", \\\"pii_removal_retry\\\": \\\"gemini-2.5-flash\\\", \\\"pii_removal_validation\\\": \\\"gemini-2.5-flash\\\", \\\"pii_removal_retry\\\": \\\"gemini-2.5-flash\\\"",
    "prompts": {
      "profanityRemovalPromptVersion": 3.0,
      "profanityRemovalValidationPromptVersion": 3.0,
      "profanityRemovalRetryPromptVersion": 2.0,
      "piiRemovalPromptVersion": 5.0,
      "piiRemovalValidationPromptVersion": 4.0
    }
  }
}
```

Prompt input into services: to examine a selection of new prompts whilst using the entire service, a partial or complete prompt map can be inputted into the service. If any prompt are missing, they are taken from a default map.

Sprint Goals



1 ✓

PR PII Redaction & Validation Fix

Resolve ambiguities in PII redaction

2 ✓

PR Cache JSON logging

Record caching usage in JSON logs for long term analysis

3 ✓

PR Integration Tests

Dependent on cache JSON logging PR

4 ✓

Caching Bug Fixes

PR to be reviewed

5 ✓

Coordinate Bupa Family Handover

Workshop scheduled for 10th Sep

Sprint Goals



Cache Other Routes
Classification and Voided



Evaluate new Gemini models
2.5 being deprecated



Bupa Family Handover
Workshop held on 10th Sep

Update LLMs

Speed and Cost

Problem Statement

Gemini-2.5-Flash will soon be depreciated. We need to understand how Gemini-3.1-Flash-Lite is expected to perform in its place.

Solution

Run each service several times to estimate latency and token usage

Results

- 3.1-Flash-Lite was significantly faster
- 3.1-Flash-Lite used no thought tokens and less tokens overall
- With cheaper input and output costs, 3.1 was also cheaper overall.

Gemini-3.1-Flash-Lite

- Input: \$0.25 per million
- Output: \$1.50 per million

Gemini-2.5-Flash

- Input: \$0.30 per million
- Output: \$2.50 per million

Times

	NPP		Summarise		Classify		Voided	
Model	2.5	3.1	2.5	3.1	2.5	3.1	2.5	3.1
mean	17.8	4.6	4	1.1	2.3	1	3.8	1.1
std	0.3	1	0.1	0.1	0.1	0.2	0.2	0.2
min	17.3	3.8	3.7	0.9	2.2	0.8	3.6	0.8
max	18.4	6.8	4.2	1.2	2.6	1.4	4.1	1.6

Total Token Count

	NPP		Summarise		Classify		Voided	
Model	2.5	3.1	2.5	3.1	2.5	3.1	2.5	3.1
mean	11977.6	7638	2546	1874	2188	1928	4093	3628
std	61	0	0	0	0	0	0	0
min	11878	7638	2546	1874	2188	1928	4093	3628
max	12130	7638	2546	1874	2188	1928	4093	3628

Update LLMs Performance

Results

Summary

2.5 had more grounded responses (less hallucinations), but performance was otherwise similar

Classify

2.5 marginally outperformed 3.1

Note Pre-Processing

Performance was the same, but there were redaction failure errors with 3.1

Voided

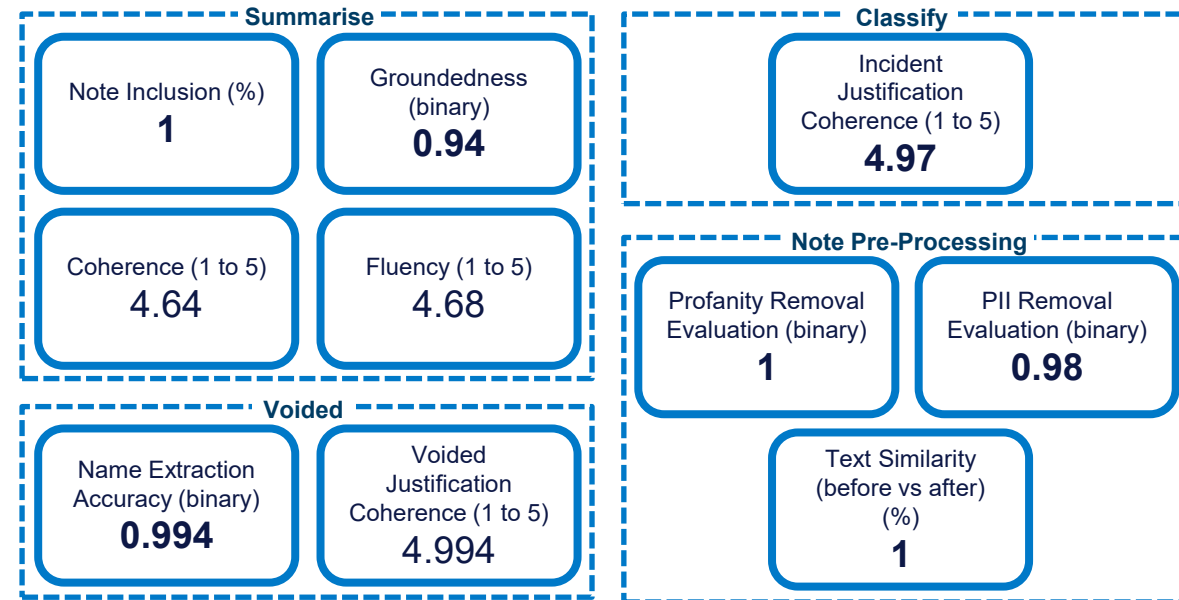
Performance was the same, but there were occasional errors when running with 3.1 and 3.1 made more errors on the ShadowTraffic data

All outputs were evaluated with 2.5-flash

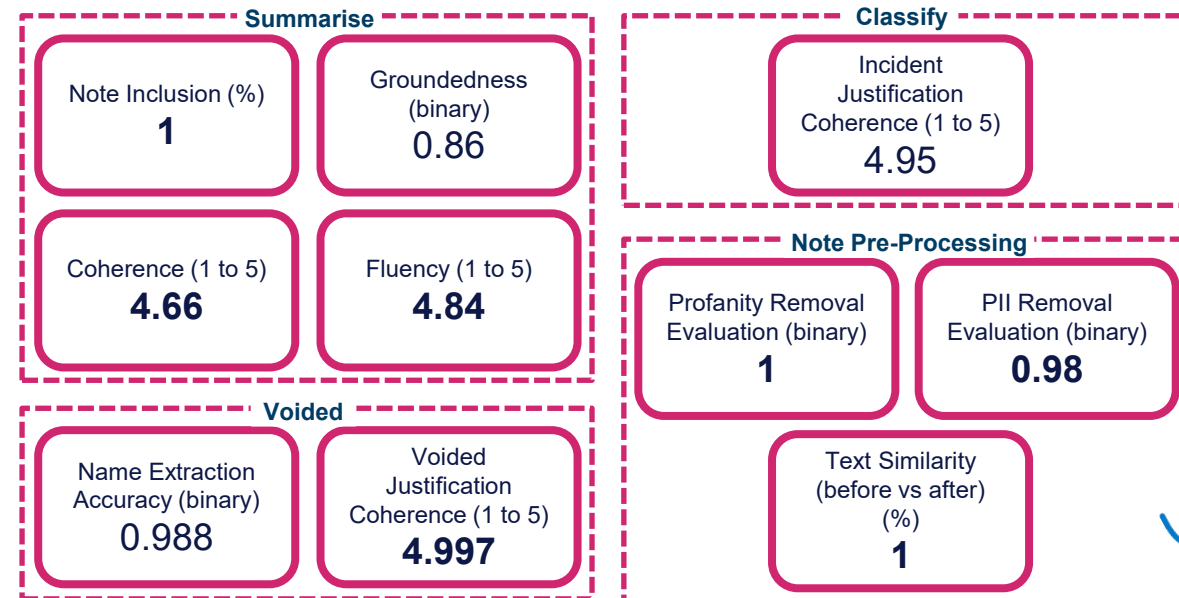
Verdict

3.1-flash-lite will be faster, cheaper, and perform at a similar level to 2.5-flash. However, we may see more API exceptions raised – more failed requests.

Gemini-2.5-Flash



Gemini-3.1-Flash-Lite



More redaction failures with this model (10003)